Real-Time LiDAR-Inertial SLAM for Resource-Constrained UAV Subterranean Mapping

By
Stepan Letsko

The thesis is submitted to University College Dublin in part fulfilment of the requirements for the degree of

Master of Engineering in Electronic and Computer Engineering

School of Electrical & Electronic Engineering

Supervisor: Dr. Pasika Ranaweera

April 2026

Hazard Form

SCHOOL OF ELECTRICAL & ELECTRONIC ENGINEERING

HAZARD IDENTIFICATION FORM

ME ELECTRICAL POWER ENGINEERING, ME ELECTRONIC & COMPUTER ENGINEERING, BE ELECTRICAL ENGINEERING OR BE ELECTRONIC ENGINEERING

Please complete the form using **BLOCK CAPITALS.** Note that you will **not** be allocated a space in the project lab (if required) until you supply a fully signed off version of this form.

Student Name:	Stepan Letsko	**Student Number:**	21407284		
Email:	stepan.letsko@ucdconnect.ie				
Project Title:	On-Board 3D LiDAR–IMU SLAM for UAV Cave Mapping				
Programme:	ME Electronic & Computer	**BE**		**ME**	✓

Health and Safety Aspects
Identify any potential safety hazards for your project, e.g. dangerous voltages, rotating machinery, radiation, soldering fumes, sitting in front of a computer for prolonged periods
• Prolonged computer use leading to eye strain and back/neck strain • Potential use of embedded hardware platforms (Jetson / Raspberry Pi) that may require electrical connection thus creating risk of electrical hazards • Handling LiDAR sensors: potential laser eye exposure
Indicate how all safety hazards identified above will be managed to achieve a safe working environment for your project

- Take regular breaks from screen use
- Take necessary electrical precautions and use PPE
- Follow LIDAR manufacturer's eye-safety guidelines

Indicate procedures in place for on-campus activities relating to the project, particularly in relation to unsupervised access to the project laboratory

- All project activities in the laboratory will follow UCD's health and safety procedures
- only access the project laboratory during permitted hours
- Unsupervised access will be limited to safe activities
- sign-in/out procedures will be followed
- ensure that the workspace is left tidy

Indicate how project activities and final objectives will be revised, if appropriate, if there is an extended lockdown period on the UCD campus

- Shift to software-only simulations using LiDAR/IMU datasets
- Focus on algorithm development, evaluation, and simulation metrics.

Ethical and Sustainability Aspects

Identify any ethical considerations which may affect the progress of your project.

- **Privacy Concerns:** LiDAR scanning and mapping could unintentionally capture sensitive details of people or private property. Testing will therefore be restricted to controlled indoor environments or public places

Identify any sustainability considerations which may affect the progress of your project.
• Reuse and repurpose available embedded platforms when possible, instead of acquiring new ones.

Student Project Laboratory Access

Laboratory desk space is **limited**. Each allocated desk includes **one monitor (no additional monitors are available), one HDMI cable, and a power outlet** for your laptop connection.

A small number of desks have **ethernet connections**, but these are also limited.

Wi-Fi is available. Students are advised to connect to the **eduroam** network for safety and stability.

Students are advised to bring their own HDMI–USB-C adapters, as the laboratory does not provide them.

If a PC is required, it must be requested by the project supervisor with proper justification. **The number of PCs is limited and restricted**.

Do you require a desk in the project laboratory?	Yes	✓	No	
Will your supervisor instead provide a desk for you?	Yes		No	
If so, please indicate the location				

Supervisor:	Name	Signature

Submit the completed form to your project module on Brightspace before the end of week 3 of the Autumn trimester.

In addition, email the completed form to the Head of the Electronic Workshop, Andrea Ortiz-Cuadros (andrea.ortizcuadros@ucd.ie).

Acknowledgments

I would first like to thank my supervisor Dr Pasika Ranaweera for his continued support and guidance throughout this project and for sourcing and acquiring all the necessary hardware. In addition I would also like to say a big thank you to Andrea from the UCD electronics workshop for her help regarding the more practical aspects of my project such as 3D printing, wiring and soldering. In particular I would also like to say thank you to my friends Donal Monahan and Killian Mullins for helping me with the field testing of my project.

Of course this thesis would not be possible without the help and support of my family and friends. Thank you for providing the emotional support and encouragement that was necessary to complete this thesis.

Abstract

Subterranean environments such as natural caves and mining tunnels provide invaluable data for geological research and archaeological discovery, and present critical operational challenges for search and rescue missions. However, traditional manual surveying of these sites is inherently hazardous and time-consuming, necessitating the deployment of autonomous Unmanned Aerial Vehicles (UAVs). While LiDAR-Inertial sensors provide the required perception for GPS-denied subterranean navigation, state-of-the-art SLAM algorithms operating in full global mapping mode exceed the strict Size, Weight, and Power (SWaP) constraints of lightweight aerial platforms leading to processing bottlenecks and memory saturation as shown in this work.

This thesis proposes and implements a functional edge-cloud split-compute architecture for real-time LiDAR-inertial SLAM on a resource-constrained UAV payload. The framework confines the Raspberry Pi 5 onboard computer to survival odometry using the FAST-LIO2 algorithm, maintaining a stable peak RAM footprint of under 500 MB during flight while offloading global map construction and pose graph optimisation to a remote base station. The work encompasses the full porting and modernisation of FAST-LIO2 from ROS 1 to ROS 2 Humble within a Docker containerised environment, the integration of a custom Hesai XT-16 LiDAR driver resolving fundamental point cloud format incompatibilities, hardware-level PPS synchronisation between the LiDAR and IMU, a cross-platform Foxglove Studio telemetry pipeline, and an offline loop closure module implementing Scan Context descriptor matching, ICP geometric verification, and Open3D pose graph optimisation.

System validation was conducted across simulation and three physical hardware deployments on custom UCD datasets. A systematic 25-configuration parameter sweep identified an optimal operating point of k = 5, v = 0.4 m, achieving zero frame drops and sub-100 ms per-frame latency across all environments. Physical deployments achieved sub-metre closing errors across all three environments, including a 0.53 m closing error under near-darkness and GPS-denied conditions in a featureless basement corridor, directly validating the system's suitability for subterranean deployment.

Lay Abstract

Mapping underground environments, such as deep natural caves or abandoned mines is essential for geological research and search and rescue missions. However, sending humans into these areas is often extremely dangerous due to risks like rockfalls and unstable ground. While small flying drones offer a safer way to explore these spaces, they face a major technical hurdle, they cannot use GPS for navigation underground and must carry heavy, expensive computers to see and navigate in total darkness.

This research has successfully developed a smart mapping system that can run on a tiny, low cost computer similar to the size of a mobile phone that is light enough for a small drone to carry. The system uses laser-based sensors that work perfectly in the dark to build highly detailed 3D digital maps of its surroundings in real time. The system is designed to livestream this data over radio to an operator for live viewing. The system was tested and validated both through simulation and physical hardware testing.

Table of Contents

List of Figures

List of Tables

Acronyms

UAV	Unmanned Aerial Vehicle
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
LiDAR	Light Detection and Ranging
SLAM	Simultaneous Localisation and Mapping
IMU	Inertial Measurement Unit
SWaP	Size, Weight, and Power
RAM	Random Access Memory
ROS	Robot Operating System
DDS	Data Distribution Service
QoS	Quality of Service
MEMS	Micro-Electrical-Mechanical System
EKF	Extended Kalman Filter
vSLAM	Visual Simultaneous Localisation and Mapping
LIO-SLAM	LiDAR-Inertial Odometry Simultaneous Localisation and Mapping
LOAM	LiDAR Odometry and Mapping
ICP	Iterative Closest Point
NIR	Near-Infrared
ToF	Time-of-Flight
DARPA	Defence Advanced Research Projects Agency
SubT	Subterranean (DARPA Challenge)
INS	Inertial Navigation System
PPS	Pulse-Per-Second
IESKF	Iterated Error-State Kalman Filter
ikd-Tree	Incremental KD-Tree
KPI	Key Performance Indicator
APE	Absolute Pose Error
RMSE	Root Mean Square Error
PCL	Point Cloud Library
CAD	Computer-Aided Design
PLA	Polylactic Acid
NMEA	National Marine Electronics Association
GPIO	General Purpose Input/Output
USB	Universal Serial Bus
UDP	User Datagram Protocol
MAP	Maximum A Posteriori
PGO	Pose Graph Optimisation
NCLT	North Campus Long-Term (dataset)
RTK	Real-Time Kinematic
TUM	Technische Universität München (trajectory format)
RGB	Red, Green, Blue
RGB-D	Red, Green, Blue — Depth
ENU	East-North-Up
PCD	Point Cloud Data

1 Introduction

Subterranean environments such as natural caves or man-made mining tunnels have significant value for geological research, hydrological studies and archaeological discovery. However, traditional manual surveying of caves is very time consuming and is inherently dangerous to humans due to the hazardous nature of underground formations where there are threats such as unstable terrain and rockfalls.

In addition to this cave like environments are frequently the setting of complex search and rescue operations such as the widely publicised Tham Luang cave rescue in 2018 in northern Thailand where a junior football association team was trapped for 17 days. During the rescue operations teams were initially forced to rely on incomplete decades-old 2D schematics (GIM International, 2018). This cave rescue incident highlighted how the inability to rapidly generate accurate 3D maps of caves severely hindered rescue operations. Consequently, the convergence of these safety constraints and the operational requirement for rapid data acquisition drives the motivation for an autonomous, UAV based mapping solution that can be operated in real time.

Figure 1.1: Tham Luang Cave, Northern Thailand

1.1 Motivation and Problem Statement

Deploying Unmanned Aerial Vehicles (UAVs) in subterranean environments gives rise to a unique engineering challenge that renders current commercial solutions ineffective. The first barrier is that deep subterranean environments are GPS-denied zones, meaning the use of the Global Navigation Satellite Systems (GNSS) is impossible. Secondly the complete absence of natural lighting in caves significantly limits the effective use of visual based mapping using RGB cameras (Zhang et al., 2017).

Instead, Light Detection and Ranging (LiDAR) has emerged as the superior sensor for cave like environments. Unlike passive visual sensors, LiDAR uses active laser beams to get precise range measurements regardless of lighting conditions, in fact it can operate in complete darkness. However simply acquiring accurate sensor data is not enough as without external GNSS signals to provide positional coordinates it is not possible for the UAV to navigate. In order for the UAV to navigate it must estimate its own position relative to its starting point. This requires the use of Odometry and Simultaneous Localisation and Mapping (SLAM) algorithms, which can construct a map of the unknown environment while simultaneously tracking the vehicle's trajectory within it.

However implementing LiDAR SLAM in caves presents its own challenges. First, the rapid and agile nature of UAV flight introduces motion distortion. Since a LiDAR's scanning rate (10 Hz) is much slower than the drones movement the resulting point clouds become skewed without high frequency inertial correction. Thus reliance on LiDAR alone is insufficient, the system requires the integration of a high sampling rate Inertial Measurement Unit (IMU) to de-skew scans. Second, caves or mining tunnels often exhibit geometric degeneracy with long featureless tunnels. This lack of distinct structural features can cause LiDAR only algorithms to slip and cause severe drift (Liao et al., 2024). Third, high accuracy SLAM algorithms require substantial computational power. Running such an algorithm in real time on board a resource constrained UAV represents a significant engineering challenge.

Finally there is also a significant practical barrier with system integration. While theoretical SLAM algorithms are well documented bridging the gap between hardware and high level software remains an engineering challenge. The integration of industrial grade sensors with consumer grade embedded computers requires the development and optimisation of custom middleware to ensure flawless operation.

1.2 Project Aims and Objectives

The primary goal of this project is to design, implement and validate a robust, real time Simultaneous Localisation and Mapping (SLAM) system capable of operating entirely onboard a resource constrained Unmanned Aerial Vehicle. The system will be specifically designed to operate in a GPS denied

subterranean environment with minimal lighting. Alongside this a rigorous evaluation workflow will be designed and implemented to test performance and quality trade-offs.

To achieve this the project will build upon and improve the FAST-LIO2 framework (Xu et al., 2021), evolving it from research into a fully deployable, state of the art embedded solution. Below are the specific engineering objectives that this project aims to achieve:

1. Embedded Computational Optimisation: To profile and optimise the SLAM pipeline for the Raspberry Pi 5 platform, ensuring peak memory allocation remains within physical hardware limits and CPU latency stays below the 100ms real-time threshold.
2. Containerised ROS 2 Software Architecture: To modernise the system architecture by porting the codebase to ROS 2 Humble and implementing a Docker-based deployment environment to ensure cross-platform reproducibility and reliable data distribution.
3. Hardware-Level Sensor Synchronisation: To engineer a deterministic hardware time bridge utilising Pulse-Per-Second (PPS) signals to achieve microsecond-level alignment between the LiDAR and IMU, thereby eliminating motion distortion.
4. Rigorous Performance Validation: To quantitatively evaluate the system using the NCLT benchmark dataset (Carlevaris-Bianco, Ushani and Eustice, 2016) and physical field trials in a GPS-denied subterranean environment.
5. Evolution from Odometry to LIO-SLAM: To extend the FAST-LIO2 odometry framework by integrating a global loop closure module based on Scan Context (Kim and Kim, 2018) and factor graph optimisation via the Open3D library (Zhou, Park and Koltun, 2018) to effectively bound accumulated trajectory drift.

2. Background

2.1 Sensor Fundamentals

In order to overcome the limitations of traditional surveying this project will make use of Light Detection and Ranging (LiDAR) and an Inertial Measurement Unit (IMU). Thus understanding the inner workings of these sensors is essential for implementing this project.

2.1.1 Operating Principles and Applications of LiDAR

LiDAR is an active remote sensing technology that measures the distance to an object by emitting laser pulses towards the said object and detecting the reflected laser. Unlike passive visual sensors (cameras), which depend on an external light source to operate, LiDAR generates its own light source meaning it has consistent performance in absolute darkness which is often the case in subterranean environments.

LiDAR's main fundamental principle of operation is the Time of Flight calculation which is used to measure the distance to an object. The principle is that laser pulses are emitted towards a target and the time taken for the reflection to return is measured. This is shown in equation (1.1), where the distance *(d)* to an object is obtained from the travel time *(t)* of the laser pulse, where *(c)* is the speed of light.

$$d = \frac{c \times t}{2} \tag{1.1}$$

The precise nature of lasers and the time of flight measurement allows modern LiDAR sensors to achieve extremely accurate measurements with accuracies ranging between 0.5mm and 10mm. This level of precision is crucial for mapping subterranean environments such as caves as it's capable of capturing the complex nature of caves with extreme accuracy.

Due to its versatility LiDAR technology has been used for a number of different applications that include:

- Autonomous Vehicles: LiDAR sensors are crucial for self-driving cars by providing real time obstacle detection. LiDAR technology is currently in use by Waymo for their fleet of autonomous cars.
- Environmental Management: In forestry and land management, LiDAR is used to assess canopy density and height (Wulder et al., 2012).

- Space Robotics: LiDAR is a critical aspect of autonomous rendezvous and docking for spacecraft. For example both the Space Shuttle and SpaceX Dragon vehicle utilise LiDAR to dock with the International Space Station (Brazzel, Clark and Milenkovic, 2015).

2.1.2 The Motion Distortion Problem

In order to capture the complex geometry of subterranean environments this project will use a 3D mechanical LiDAR. Unlike a solid state sensor such as a camera which captures a whole scene instantly, mechanical LiDARs construct a 360 degree field of view by rotating a vertical array of laser emitters. This mechanical nature of scanning introduces an error source known as “Motion Distortion”.

A standard mechanical LiDAR operates at a scan frequency of 10 Hz, meaning that it requires 100 milliseconds to complete a full rotation. In static surveying this is not an issue, however a UAV is a highly mobile platform with dynamic motion. Thus during the 100ms scan the drone may relocate or experience angular rotation. Since the laser points are collected sequentially the UAVs motion causes the generated point cloud to appear skewed (Wu et al., 2022). Without correction, this geometric distortion prevents the SLAM algorithm from working correctly, leading to map divergence and drift.

The solution to the motion distortion problem is to use Inertial Measurement Unit (IMU) based de-skewing. This involves integrating a high frequency IMU (>100 Hz) into the system. By recording the UAVs angular velocity and acceleration during a LiDAR sweep it is possible to mathematically de-skew the scan by projecting every laser point back to the position where the drone would have been at the time of the start of the scan.

2.1.3 Inertial Measurement Unit (IMU)

As mentioned in section 2.1.2 in order to correct motion distortion a Micro-Electrical-Mechanical System (MEMS) Inertial Measurement Unit (IMU) is used. An IMU is a sensor package that combines a 3 axis accelerometer and a 3 axis gyroscope to measure the vehicles linear acceleration and angular velocity in real time.

However low cost IMU’s are prone to significant noise, as noted in (Wu et al., 2022) using an IMU alone for navigation is fundamentally unstable. This is due to the fact that in order to obtain position from acceleration the signal must be integrated twice. This means that even a small sensor error grows

quadratically with time. Without external correction the accuracy decays rapidly. This is why the IMU must be tightly coupled with the LiDAR, this way the IMU handles short movement and fuses its data with the LiDAR that can provide a stable long term position estimate.

2.2 Simultaneous Localisation and Mapping (SLAM) & Odometry

Simultaneous Localisation and Mapping (SLAM) is an algorithm used by autonomous vehicles in order to navigate. As described in (Durrant-Whyte and Bailey, 2006), SLAM is the process by which a mobile robot placed in an unknown environment can build a consistent map of its surroundings while simultaneously determining its own location within it.

The SLAM problem is framed fundamentally as a probabilistic estimation problem. The goal is to compute the posterior probability distribution of the robots/vehicles position and the map given the history of inputs and observations. Durrant-Whyte and Bailey define this distribution as:

$$P(x_k, m | Z_{0:k}, U_{0:k}, x_0) \quad 2.1$$

Where:

x_k: Is the state vector describing the location and orientation of the vehicle

m: Is the set of all landmarks on the map

$Z_{0:k}$: The history of all landmark observations

$U_{0:k}$: The history of control inputs

In order to solve this problem in real time SLAM employs an estimation method consisting of two steps:

1. Pose prediction: A motion model $P(x_k|x_{k-1}, u_k)$ is used to predict the new position of the robot based on its past location and inputted control. This way we can estimate where the robot is going within the map. However this step is also affected by the systems noise meaning that with each step the uncertainty/drift of the vehicles position increases.
2. Pose Correction: The robot perceives its environment via sensors and the observation model $P(z_k|x_k, m)$. It compares these new observations with the previously generated map and corrects both the map and the vehicles position within the map.

A key takeaway is the method of error correction. Because observations are made from the vehicle any error in the estimated position of the vehicle will also introduce a common error to the whole map and

landmarks. However the landmarks are stationary and are highly correlated, as the robot moves and takes more observations using its sensors the maps correlation between landmarks will increase. Thus the map will converge and take on a more rigid and defined form. Figure 2.1 below shows this process graphically.

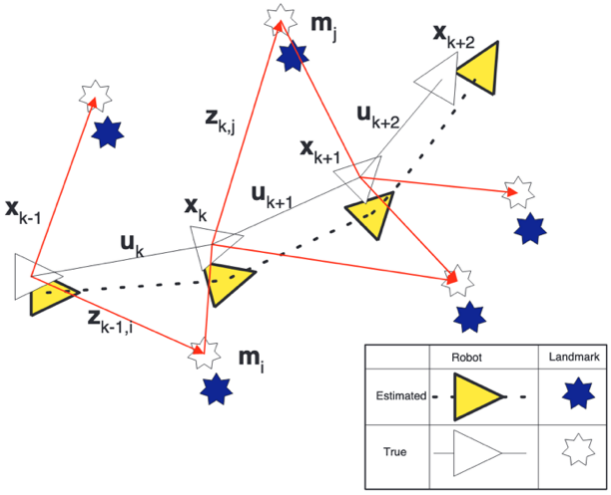

Figure 2.1: An overview of the essential SLAM problem. Source (Durrant-Whyte and Bailey, 2006)

The main computational approach used in order to implement SLAM is EKF-SLAM. This method uses an Extended Kalman Filter in order to linearise the motion and observation models. This method maintains a massive state vector representing both the robots and every landmarks position in the map. Accompanied also by a covariance matrix that tracks the uncertainty of the entire system. The advantage of this method is that it is very accurate and has great convergence. However its drawback is that the entire matrix needs to be updated after each observation and the computation cost increases quadratically with the number of landmarks (N^2), this means with a very large map the method can be slow.

2.3 Visual vs Lidar SLAM

While the SLAM algorithm itself is sensor agnostic, practical implementations of the system are categorised by their primary perception sensor. Currently there are two main systems in use, Visual SLAM (vSLAM) and LiDAR SLAM.

A Visual SLAM system uses passive cameras or RGB-D to perceive its environment and estimate the robots pose. As detailed in the survey by (Abaspur Kazerouni et al., 2022), vSLAM algorithms operate by extracting distinct features or key points such as corners or edges from a video stream. The algorithm

tracks these features across the different frames that the camera records to minimise the reprojection error. The reprojection error shows how the camera/vehicle must have moved in order for the features to shift between frames. The advantage of this approach is that cameras are cheap and energy efficient and vSLAM is currently widely in use today, as is seen in Tesla's robotaxi.

However vSLAM is fundamentally flawed for subterranean environments due to the harsh environment. As shown by (Zhang et al., 2017) in their work on mapping caves using UAVs there are two catastrophic failure points for vSLAM in subterranean environments.

1. Illumination Dependence: Cameras require ambient light in order to operate and perceive the environment correctly. There is often little to no light in subterranean environments. (Zhang et al., 2017) suggests to mitigate this by using supporting drones with light sources to illuminate the environment, however this appears too impractical due to power constraints and the complication of swarm robotics.

2. Featureless landscape: In order for vSLAM to work the system has to detect and extract features, however this becomes difficult as in poor lighting cave walls often lack high contrast colour variations required for feature extraction. If a camera observes a smooth uniform rock surface such as a tunnel it will not detect motion and lose its position estimate.

In contrast LiDAR SLAM uses active laser ranging to perceive the environment around it. Instead of inferring geometry from 2D pixels LiDAR can directly measure the 3D structure of its surroundings. Thus the use of LiDAR SLAM would eliminate the issue of illumination dependence.

However there is still the risk of degeneracy and motion distortion as mentioned in section 2.1.2. Consequently, the chosen architecture is a tightly-coupled LiDAR-Inertial SLAM (LIO-SLAM) system. This approach directly integrates high-frequency inertial measurements with raw LiDAR scans within a unified estimator, effectively mitigating motion distortion at the front-end while providing robust navigation in degenerate environments where pure LiDAR methods would otherwise fail.

3. Literature Review

3.1 The Evolution of LiDAR-Inertial SLAM

As established previously in the background (section 2.3), LiDAR-based SLAM has become the standard for subterranean mapping due to its independence from external lighting sources. However the specific methodology for fusing the LiDAR and inertial data has evolved significantly in order to meet the computational and real life constraints of mobile robotics. This evolution can be categorised into three distinct architectural generations, loose coupling, tight coupling and direct registration.

3.1.1 Feature Based Architectures

The foundational architecture for modern LIO-SLAM is LiDAR Odometry and Mapping (LOAM) (Zhang and Singh, 2014). LOAM proposed the concept of splitting the SLAM problem into two parallel threads in order to achieve real-time performance. The first thread was the odometry thread running at 10 Hz that would perform scan to scan matching to estimate the motion that has occurred between scans. The second thread would perform scan to map registration to refine the global map.

In order to achieve the SLAM mapping LOAM utilised feature extraction. This process involved calculating the curvature of local points in order to classify them as either Edge Points with sharp geometric features or Planar Points with smooth surfaces.

LeGO-LOAM (Shan and Englot, 2018) optimised this approach further by introducing a segmentation step to filter out ground points, which would significantly reduce the computational load for ground vehicles.

While somewhat efficient, feature based methods suffer from a critical limitation in certain environments such as caves or tunnels which lack distinct straight edges or flat planes required by LOAM. This leads to unstable feature tracking which in turn leads to significant drift making this approach unusable on a UAV platform.

3.1.2 Tightly Coupled Factor Graphs

To address the drift present in the loosely coupled LOAM architecture, LIO-SAM (Shan et al., 2020) formulated the problem using a factor graph. This approach integrates both IMU pre-integration and LiDAR odometry factors into a unified optimisation graph. This tight coupling ensures that if the

LiDAR loses tracking in a featureless corridor, the IMU factors can prevent immediate failure. Additionally, LIO-SAM maintains a keyframe-based global map, allowing for loop closure via a geometric radius search. However, while LIO-SAM offers robust tracking, it is computationally expensive due to the overhead of feature extraction and the high RAM usage required to maintain the global map. Furthermore, the geometric radius search is prone to failure if the accumulated odometry drift exceeds the search radius.

3.1.3 Direct Methods

The current state of the art in high-efficiency motion estimation is represented by FAST-LIO2 (Xu et al., 2021). This framework marks a significant shift from traditional feature-extraction methods toward Direct Methods. Instead of discarding most LiDAR data, FAST-LIO2 instead utilised the raw dense point cloud which increases the quality of the mapping and mostly eliminates the degeneracy issue. Even though FAST-LIO2 utilises the raw point cloud it is still computationally more efficient than previous approaches such as LIO-SAM due to two core innovations.

The first innovation is the use of the Iterated Error State Kalman Filter (IESKF), Instead of estimating the robots global position directly the IESKF estimates the error from the predicted path. The IESKF re-linearises multiple times per step until the calculation converges. This iterative process handles the rapid, non-linear motions of a UAV much more accurately than a standard filter, achieving high precision without the heavy computational cost of a full factor graph optimisation.

The second innovation is the use of a new Incremental KD-Tree (ikd-tree). Traditional KD-Trees are static meaning that they need to be rebuilt every time new points are added to the map. As the map grows to millions of points the rebuilding process becomes very slow making the system infeasible in real time. However the ikd-Tree is a dynamic structure that allows for individual points to be inserted or deleted instantly without rebuilding the entire tree. This incremental update capability allows the system to maintain a high-density map in RAM while processing new LiDAR scans at high speed.

While FAST-LIO2 provides exceptional local tracking accuracy and superb efficiency suitable for embedded systems, it remains a pure odometry framework not a full SLAM system as it lacks a native loop closure module, meaning that inevitable sensor noise and small errors in the IMU bias estimation

accumulate over time. Table 3.1 below shows a comparative synthesis of the different state of the art LIO-SLAM frameworks.

Framework	Architecture	Mapping Method	Loop Closure	Limitations
LOAM	Feature Based	Edge/Plane matching	No	Fails in organic, unstructured tunnel geometry.
LeGO-LOAM	Feature Based	Feature Map	Geometric Radius Search	Relies on flat ground; fails during 3D flight.
LIO-SAM	Factor Graph	Keyframe Map	Geometric Radius Search	High compute cost; Radius search fails if drift is large.
FAST-LIO2	Direct Method	ikd-Tree	No	Accumulates unbounded drift over long trajectories.
This Work	Direct+Global	ikd-Tree	Global Scan Context	Requires PPS hardware synchronisation between sensors.

Table 3.1: Comparison of LIO-SLAM frameworks

3.2 Global Place Recognition

As identified in Section 3.1.3, pure odometry systems accumulate drift over long trajectories. While frameworks like LIO-SAM attempt to correct this via Euclidean Radius Search, this method relies on the robot's estimated position being relatively accurate. If the accumulated drift exceeds the search radius the system will look in the wrong location and fail to detect the loop closure. To resolve this, research has shifted toward Global Place Recognition, which identifies a location based on its geometric structure rather than its coordinates.

Current approaches diverge into learning-based and hand-crafted methods. Deep learning frameworks, such as PointNetVLAD (Uy and Lee, 2018) which utilises neural networks to extract global feature vectors. While highly accurate, the process requires significant GPU acceleration, rendering them unsuitable for the embedded architecture suitable for the drones targeted in this research.

Consequently, this project utilises Scan Context (Kim & Kim, 2018), a lightweight hand-crafted descriptor. Scan Context converts the visible 3D point cloud into a 2D egocentric descriptor by encoding the maximum height of points within azimuthal and radial bins. This process creates a rotation-invariant fingerprint of the location. By matching these 2D images, the system can recognise a previously visited

location regardless of the UAVs orientation or the magnitude of accumulated odometry drift, providing robust loop closure with minimal computational overhead.

3.3 Comparative Analysis of Subterranean Exploration Systems

The autonomous exploration of subterranean environments has historically posed severe navigational challenges due to complex terrain, confined spaces, and the complete absence of GNSS signals and ambient illumination. To address these hazards, research has increasingly shifted from teleoperated rovers to fully autonomous, multi-modal robotic deployments.

Early aerial systems attempted to overcome the lack of light using active visual sensing. For instance, Zhang et al. (2017) developed *SmartCaveDrone*, which relied on RGB-D sensors paired with active lighting. However, active illumination drastically increases power consumption and payload weight, severely limiting the flight endurance of micro-UAVs. Similarly, Papachristos et al. (2017) evaluated Near-Infrared (NIR) stereo cameras combined with Time-of-Flight (ToF) sensors. While reducing the reliance on visible light, these visual methods still demonstrated poor performance and rapid tracking failure in feature-sparse, homogenous rock environments.

Consequently, 3D mechanical LiDAR has emerged as the primary perceptual sensor for subterranean environments, providing high-fidelity spatial data without requiring external illumination. While early LiDAR applications by Weishampel et al. (2011) were restricted to airborne platforms identifying cave entrances through forest canopies, recent advancements have focused on internal volumetric mapping. For example, Grasso et al. (2023) mapped the Bossea Cave using LiDAR-based SLAM technology. However, while the system provided raw odometry, achieving a globally consistent map required over three hours of offline post-processing to execute loop closures and trajectory optimisation. This heavy reliance on post-flight computation renders the approach unsuitable for active UAV deployment, which strictly requires simultaneous, real-time mapping and state estimation to facilitate autonomous navigation.

The push for real-time, autonomous aerial mapping has been heavily accelerated by the DARPA Subterranean (SubT) Challenge. Winning deployments, such as the *CERBERUS* system presented by Tranzatto et al. (2022), successfully demonstrated UAVs executing LiDAR-inertial SLAM in complex

underground networks. However, to handle the computational load of SLAM and target detection algorithms, these state-of-the-art UAVs required massive high-wattage payloads, specifically, a 16-thread AMD 4800U processor, 64 GB of RAM, and three Intel Neural Compute Sticks. This extreme Size, Weight, and Power (SWaP) penalty severely restricted their flight endurance. A comparison of these varied cave exploration modalities and their limitations is summarised in Table 3.2.

Study	Technology	SLAM	Real-time	Limitations
Zhang et al. (2017)	RGB-D, Active Lighting	Yes	Yes	High power consumption; lighting increases payload weight.
Papachristos et al. (2017)	NIR, ToF	No	No	Tracking failure in feature-sparse rock environments.
Weishampel et al. (2011)	Airborne LiDAR	No	No	Limited to exterior mapping; cannot enter cave structures.
Grasso et al. (2023)	Handheld LiDAR / UAV	Yes	No	Requires hours of offline post-processing for map consistency.
Tranzatto et al. (2022)	Multi-modal (LiDAR/Vis)	Yes	Yes	Extreme SWaP requirements; limits flight time to 10-22 mins.
This Work	LiDAR, IMU	Yes	Yes	Requires intensive algorithmic and hardware optimisation.

Table 3.2: Comparison of existing subterranean/cave mapping solutions

4. Methodology & System Design

This chapter presents the complete system architecture designed to address the engineering challenges identified in Chapters 1 and 2. The design process was governed by three main constraints that had to be satisfied simultaneously. These main constraints are achieving real time SLAM performance, remaining within the strict Size, Weight, and Power (SWaP) budget of a lightweight UAV payload, and maintaining full navigational autonomy in a GPS denied, subterranean environment. Each architectural decision presented in this chapter is directly motivated by these constraints.

4.1 System Scope & Overview

Before presenting the system architecture it is necessary to define the precise engineering scope of this thesis. This project focuses exclusively on the perception and computing payload required for autonomous subterranean navigation.

The following components are considered in scope**:**

- SLAM algorithm adaptation, optimisation, and loop closure integration
- Sensor integration and driver development for the LiDAR and IMU
- Hardware-level sensor synchronisation
- Embedded computing platform selection and optimisation
- Real-time telemetry pipeline to a remote base station
- Rigorous system performance analysis

The following are explicitly out of scope**:**

- The mechanical design, fabrication, and assembly of the quadcopter airframe
- The UAV flight controller
- Autonomous path planning and obstacle avoidance

The UAV platform is therefore treated as an independent carrier that provides power and mobility to the payload. This scoping decision allows the research to focus entirely on the perception and computing stack, which represents the primary technical challenge in subterranean autonomous mapping.

A high level overview of the complete system is illustrated in Figure 4.1 below.

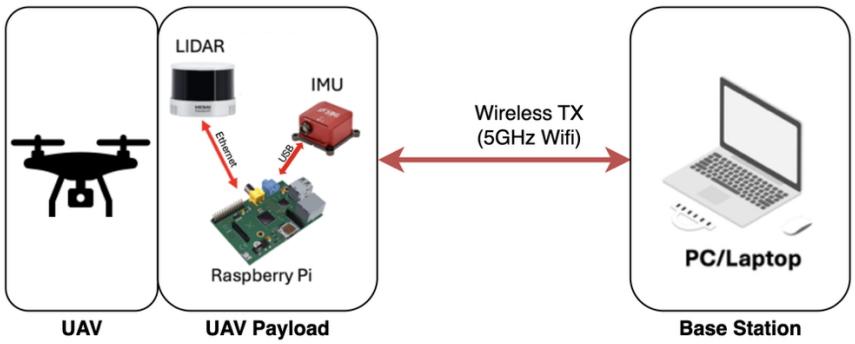

Figure 4.1: High level system overview

As illustrated in Figure 4.1, the system architecture is organised into two distinct sections, the UAV platform and the remote base station. On the UAV platform, the Hesai Pandar XT-16 LiDAR and the SBG Ellipse-D IMU form the sensor payload. Raw point cloud data from the LiDAR and high-frequency inertial measurements from the IMU are transmitted directly to the Raspberry Pi 5 embedded computer via Ethernet and USB respectively. A hardware Pulse-Per-Second (PPS) synchronisation signal is routed from the IMU to the LiDAR to ensure microsecond-level time alignment between both sensors prior to any algorithmic processing. The Raspberry Pi 5 acts as the centralised edge compute unit, executing the FAST-LIO2 state estimator locally to produce real-time odometry and constructing Scan Context descriptors for loop closure detection. The resulting odometry trajectory, down sampled point clouds, and loop closure constraints are saved locally on the device as well as additionally streamed over a 5GHz wireless telemetry link to the base station. At the base station, a laptop receives the transmitted data which is rendered live using Foxglove Studio (Foxglove Studio, 2026) to give live information to the operator. Critically, the architecture is designed to be fully decentralised. The edge SLAM pipeline operates entirely independently of the base station connection, meaning that in the event of a communication failure, which is a frequent occurrence in deep subterranean environments, the UAV continues to maintain accurate state estimation and can complete its mapping mission without any ground control intervention.

4.2 Hardware Architecture

As mentioned the hardware architecture of the system was designed under three competing constraints that had to be satisfied simultaneously. First, the embedded computing unit must deliver sufficient processing throughput to execute the full LIO-SLAM pipeline in real time, maintaining a per-frame

latency below the 100ms threshold which is imposed by the 10Hz LiDAR scan rate. Second, the entire payload must minimise Size, Weight, and Power (SWaP) consumption, as every gram of additional payload and every watt of additional power draw directly reduces UAV flight endurance and limits operational range. Third, the sensors must provide accurate measurements for subterranean mapping, specifically the ability to operate in complete darkness with sub-centimetre range precision, and capture a sufficiently wide field of view to resolve the full geometry of cave passages in a single scan. The hardware components selected for this system and the justification for each selection are presented in the following subsections.

4.2.1 Lidar Sensor

The primary perception sensor selected for this system is the Hesai Pandar XT-16 mechanical rotating LiDAR. The sensor was selected on the basis of four key specifications that directly address the requirements of subterranean mapping. First, the XT-16 provides a 360° horizontal by 30° vertical field of view. The full 360° horizontal coverage ensures that the complete cross sectional geometry of a cave passage can be captured in a single rotation without requiring any reorientation of the sensor. The 30° vertical field of view is equally important as it allows both the floor and ceiling of a subterranean passage to be captured simultaneously, which is essential for constructing an accurate 3D map. Second, the sensor provides a maximum ranging distance of 120 metres, which comfortably exceeds the expected dimensions of the subterranean passages and corridors targeted in this research, ensuring that distant walls and geological features are detected before the UAV approaches them. Third, the XT-16 achieves a range precision of 0.5cm, which is sufficient to resolve fine geological surface features and produce high-fidelity point cloud representations of complex cave geometry. Fourth, and most critically for the target deployment environment, the XT-16 is an active sensing technology that generates its own laser illumination. Unlike passive visual sensors which depend entirely on ambient light, the LiDAR operates with identical performance in complete darkness, making it the only viable primary perception sensor for subterranean environments where no natural or artificial lighting can be assumed. It should be noted that there are multiple other sensors with the same parameters that could be selected and would

perform in the same manner, however the Hesai Pandar was selected specifically due to supplier constraints. The Hesai Pandar XT-16 is shown below in Figure 4.2.

Figure 4.2: Hesai Pandar XT-16

It should be noted that the FAST-LIO2 framework was originally developed and validated for Livox and Velodyne LiDAR sensors and does not natively support the Hesai Pandar XT-16. A custom driver interface and sensor configuration were therefore required to integrate this sensor into the pipeline. The implementation details of this integration are presented in Section 5.

4.2.2 IMU Sensor

The inertial sensor selected for this system is the SBG Ellipse-D, an industrial-grade Inertial Navigation System (INS). While the Ellipse-D is capable of providing a full onboard navigation solution via GPS, in this system it is utilised exclusively for its high-frequency raw inertial data output and its hardware timing capabilities. For clarity, it will be referred to throughout this thesis as an IMU, reflecting the functional role it plays within the LIO-SLAM pipeline.

Figure 4.3: SBG Ellipse-D

The most important specification driving the IMU selection is its output data rate of 200Hz. To understand why this frequency is important we consider the motion de-skewing/distortion problem

introduced in Section 2.1.2. A mechanical LiDAR requires 100 milliseconds to complete a single full rotation, during which the UAV is continuously moving. The de-skewing algorithm must mathematically project every individual laser point back to the position the drone occupied at the precise moment that point was acquired. At 200Hz, the IMU produces one measurement every 5 milliseconds, meaning that across a single 100ms LiDAR sweep the algorithm has access to 20 inertial measurements with which to reconstruct the continuous motion trajectory. This dense sampling allows accurate interpolation of the UAV pose at the timestamp of each individual laser point, producing a geometrically consistent de-skewed scan. A lower frequency IMU would produce sparser measurements, forcing the algorithm to interpolate over longer time intervals and introducing distortion error into every scan.

Beyond its output rate, the SBG Ellipse-D was selected for its industrial grade noise characteristics. As discussed in Section 2.1.3, consumer-grade MEMS IMUs are prone to significant noise and bias instability. A high-noise IMU introduces larger bias estimation errors that accumulate more rapidly between LiDAR corrections, degrading the quality of the motion de-skewing and ultimately increasing odometry drift. The Ellipse-D's superior noise performance therefore directly contributes to the long-term accuracy of the state estimator over extended subterranean missions.

4.2.3 Embedded Computing Unit

The embedded computing unit is the most critical hardware selection in the system, as it must deliver sufficient processing throughput to execute the full LIO-SLAM pipeline in real time while simultaneously satisfying the strict SWaP constraints of a lightweight UAV payload. The Raspberry Pi 5 was available as an existing resource and was therefore selected as the primary development and deployment platform. However, to rigorously validate this selection and confirm it represents the optimal choice among viable alternatives, a systematic cross-platform feasibility analysis was conducted.

A direct performance comparison across platforms was not possible through native execution, as the development workstation used for baseline profiling employs an Intel Ultra 5 processor based on the x86 CISC architecture, while all embedded candidates employ ARM RISC architectures. Executing ARM-compiled binaries on x86 hardware via emulation introduces significant timing inaccuracies that

do not reflect true hardware performance. Instead, a two-stage calibrated projection model was developed.

In the first stage, a naive prediction was derived using Geekbench 6's publicly available single core benchmark scores (www.geekbench.com). Geekbench 6 was selected because its single-core workloads, including matrix operations, image processing, and tree traversal closely mirror the computational characteristics of the FAST-LIO2 algorithm. Assuming processing latency scales inversely with single-core benchmark score, the naive projected latency for any target platform is given by:

$$T_{target} = T_{base} \times \frac{S_{base}}{S_{target}} \tag{4.1}$$

Where T_{base} is the measured baseline FASTLIO2 latency on the Intel Ultra 5 with a voxel filter of 0.4 and point filter of 3, S_{base} is the Intel Ultra 5 Geekbench 6 score, and S_{target} is the score of the candidate platform. However, this naive model carries a fundamental limitation, it assumes that the computational workload scales identically across different processor architectures, which is not the case. The transition from x86 CISC to ARM RISC introduces systematic performance differences arising from different instruction set efficiency.

To correct for these architectural effects, an empirical architecture correction factor k_{arch} was derived using the two available real hardware measurements, the Intel Ultra 5 measured latency of 7.90ms and the Raspberry Pi 5 measured latency of 55.52ms. The naive model predicted the Pi 5 latency as 21.22ms. The ratio between the actual measured value and this prediction gives the correction factor of:

$$k_{arch} = \frac{T_{Pi5,measured}}{T_{Pi5,predicted}} = \frac{55.52\ ms}{21.22\ ms} = 2.62 \tag{4.2}$$

This factor empirically captures the performance penalty of the x86 to ARM architectural transition for this specific workload. The corrected projected latency for any ARM candidate platform is then:

$$T_{target,corrected} = T_{base} \times \frac{S_{base}}{S_{target}} \times k_{arch} \tag{4.3}$$

This corrected model is grounded in real hardware measurements rather than theoretical assumptions, making it significantly more reliable than the Geekbench scaling alone. It should be noted that applying a uniform k_{arch} across all ARM platforms remains a simplification, as architectural differences

between the Cortex-A76 of the Pi 5 and the Cortex-A78 of the Jetson Orin Nano introduce additional variation. The model should therefore be interpreted as a calibrated first-order estimate rather than an exact predictor. The corrected latency projections are visualised in Figure 4.4 and summarised alongside key SWaP and cost metrics in Table 4.1.

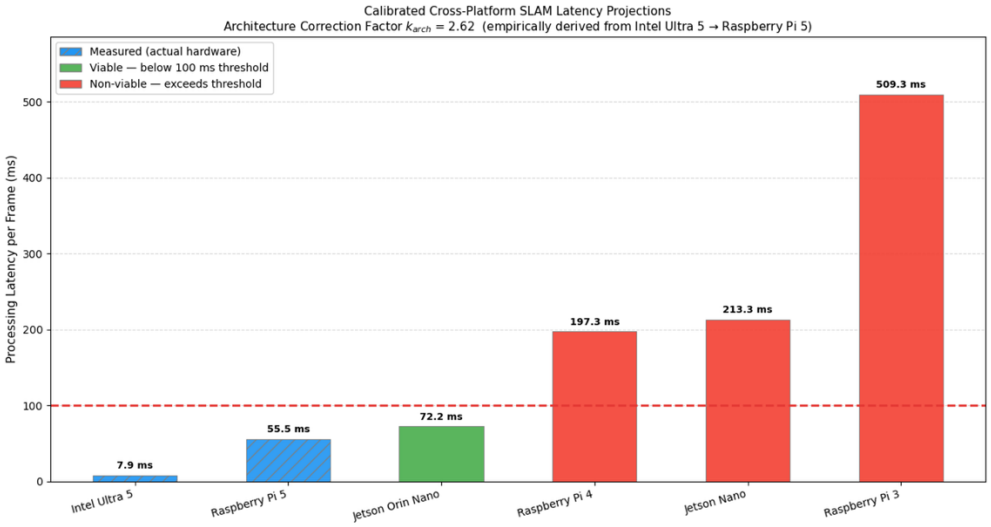

Figure 4.4: Projected latency of FASTLIO2 on different architectures

Platform	GB Score	Projected Latency	Power Draw	Approx. Cost	Viable
Intel Ultra 5	2415	7.9ms (measured)	~28W	—	Yes
Raspberry Pi 5	899	55.5ms (measured)	~12W	€150	Yes
Jetson Orin Nano	691	72.2ms	~15W	€350	Marginal
Raspberry Pi 4	253	197.3ms	~7W	€100	No
Jetson Nano	234	213.3ms	~10W	€250	No
Raspberry Pi 3	98	509.3ms	~5W	€50	No

Table 4.1: Cross-platform FASTLIO2 latency projections with SWaP and cost comparison

The analysis identifies a clear three-tier structure among the candidate platforms. The Raspberry Pi 3, Raspberry Pi 4, and Jetson Nano are all non-viable. Their projected latencies of 509.3ms, 197.3ms, and 213.3ms respectively far exceed the 100ms real-time threshold, and would result in continuous frame drops and immediate state estimator divergence during live UAV operation.

The Jetson Orin Nano presents a more nuanced case. Its corrected latency of 72.2ms falls below the 100ms threshold, leaving a safety margin of approximately 28ms. However, this margin is still smaller than that of the PI5 meaning there is less safety overhead for additional computational requirements. Furthermore, the Jetson platforms include a CUDA-capable GPU which provides significant acceleration for deep learning workloads but confers no benefit for this system. The FAST-LIO2 IESKF and ikd-Tree operations are entirely CPU-bound, the IESKF relies on sequential Eigen linear algebra

updates and the ikd-Tree relies on sequential tree traversal, both of these are more suited for the CPU. The GPU therefore adds substantial cost, weight, and power consumption to the payload, at approximately €350 and 15W versus the Pi 5's €150 and 12W, while providing zero algorithmic benefit for this specific workload. Carrying unnecessary hardware on a weight-constrained UAV directly reduces flight endurance and is an unjustifiable design decision.

The Raspberry Pi 5 is therefore confirmed as the optimal platform selection. Its measured latency provides a safety factor relative to the real-time threshold, leaving meaningful headroom for the overhead processes and transient load spikes. Its power consumption of approximately 12W is the lowest among viable candidates, and its cost of approximately €150 is less than half that of the Jetson Orin Nano.

4.2.4 Hardware-Level Sensor Synchronisation

Accurate state estimation in LiDAR-Inertial systems is fundamentally dependent on the precise time alignment of data from two physically separate sensors operating on independent internal clocks, without an explicit synchronisation mechanism, these clocks will drift relative to one another over time. This subsection describes the synchronisation problem in detail and presents the hardware-level solution implemented in this system.

As established in Section 2.1.2, a mechanical LiDAR does not capture a scene instantaneously. Instead it fires laser pulses sequentially, meaning the drone has continued to move by the time the scan is complete. Each laser point is therefore measured from a slightly different position and orientation of the drone, producing a geometrically smeared point cloud. FAST-LIO corrects this through a two-step process formalised by Xu and Zhang (2021).

As illustrated in Figure 4.5 below (Livox Technology, 2021), during a single LiDAR scan spanning from $t_{k-1}$to t_k, the IMU is simultaneously recording measurements at a much higher frequency. Each individual laser point lands at a different moment ρ_j along the IMU's time axis. The first correction step, described by Xu and Zhang (2021, Eq. 9), performs a backward IMU propagation, it integrates the recorded IMU measurements in reverse, starting from the scan-end time t_k and working backwards to each point's individual sample time ρ_j. This recovers the relative body pose ${}^{I_k}\check{T}_{I_j}$, which describes

precisely how much the drone translated and rotated between the moment point j was fired and the end of the scan.

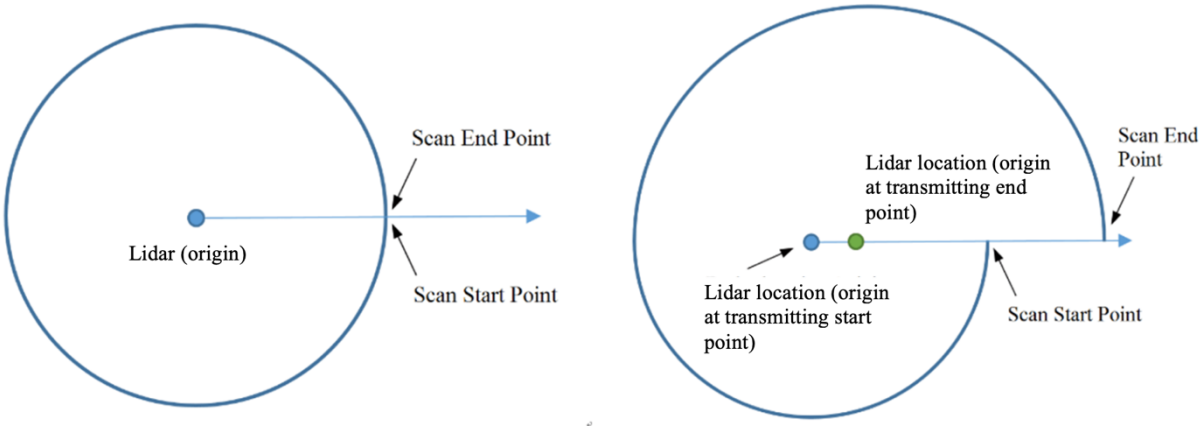

Figure 4.5: 360-degree mechanical LiDAR motion distortion schematic. Source (Livox Technology, 2021)

This relative pose is then applied in the de-skewing projection (Xu and Zhang, 2021, Eq. 10), reproduced here as Equation 4.4 below.

$$^{L_k}\boldsymbol{p}_{f_i} = {}^{I}T_{L}^{-1} \cdot {}^{I_k}\check{T}_{I_j} \cdot {}^{I}T_{L} \cdot {}^{L_j}\boldsymbol{p}_{f_i} \tag{4.4}$$

To understand what this equation is doing, it helps to unpack the notation first. The bold **p** denotes a 3D position vector. The subscript f_j identifies the feature point j. The superscript on the left, written above and to the left of **p**, identifies the coordinate frame in which that position is expressed. Thus $^{L_j}\boldsymbol{p}_{f_i}$ represents the position of feature point j, expressed in the LiDAR's own coordinate frame at the moment it was fired, this is simply the raw reading that comes directly out of the sensor. The output $^{L_k}\boldsymbol{p}_{f_i}$ represents the same point, now re-expressed in the LiDAR frame at scan-end time t_k, this is the corrected, de-skewed coordinate.

Reading Equation 4.4 from right to left, four operations are applied in sequence. First, the raw point $^{L_j}\boldsymbol{p}_{f_i}$ is multiplied by $^{I}T_{L}$, which transforms it from the LiDAR's coordinate frame into the IMU's body frame. This accounts for the fixed physical offset and rotation between the two sensors as they are bolted together on the drone, and is a known constant determined during sensor calibration. Second, $^{I_k}\check{T}_{I_j}$ is applied, this is the relative motion recovered by the backward propagation of Xu and Zhang (2021, Eq. 9), and it repositions the point as if the drone had been stationary at its scan-end pose when the laser was fired. This is the step that actually removes the motion distortion related smearing. Third,

${}^{I}T_{L}^{-1}$ converts the result back from the IMU body frame into the LiDAR frame, which is simply the inverse of the first step. After applying this to every point in the scan, all points share a single geometrically consistent reference frame and the distortion is eliminated.

The correctness of this entire pipeline rests on one critical assumption: that the timestamp ρ_j assigned to each laser point accurately reflects the true physical moment that point was measured, expressed on the same time axis as the IMU. The backward propagation of Xu and Zhang (2021, Eq. 9) uses ρ_j to determine exactly which IMU measurements to integrate over. If the LiDAR and IMU clocks are not synchronised, the timestamp ρ_j will be offset from reality by some unknown error. The algorithm then integrates the wrong segment of IMU data, producing an incorrect relative transform ${}^{I_k}\check{T}_{I_j}$, and each affected point is projected to the wrong location in the de-skewed cloud. This is not random noise, it is a systematic geometric distortion that propagates directly into the state estimator, degrading both odometry accuracy and map quality. Even a clock offset of just a few milliseconds, which is well within the range of unsynchronised USB or serial interfaces, is sufficient to introduce meaningful positional errors when the drone is undergoing rapid rotation or acceleration.

To eliminate this source of error, hardware-level synchronisation was implemented between the LiDAR and IMU. To understand why this is necessary, it is first important to consider why software based synchronisation/timestamping is fundamentally inadequate for this application.

When a sensor transmits data over a standard interface such as USB, the operating system assigns a timestamp to the data packet at the moment it is processed by the software driver. However, this timestamp does not reflect the true physical moment the measurement was taken. USB is a polled protocol, meaning the host controller queries device's on a scheduled basis rather than receiving data the instant it is produced. The time between a measurement being made at the sensor and the software receiving and timestamping it is therefore variable and unpredictable, introducing latency on the order of milliseconds that fluctuates depending on CPU load and bus contention.

Ethernet-based interfaces offer substantially lower and more deterministic latency than USB and are in principle better suited to time-sensitive sensor applications. The Hesai Pandar XT-16 LiDAR used in

this system communicates over Ethernet, which allows it to transmit point cloud data with lower jitter than a USB connection would permit. However, low-latency transmission alone does not solve the synchronisation problem. Even if data arrives quickly, the LiDAR and IMU are still operating on entirely independent internal oscillators with no shared notion of time. A packet arriving quickly over Ethernet is still timestamped by a clock that has no guaranteed relationship to the IMU's clock. The two sensors may be drifting apart at different rates, and without a common reference signal this drift is unobservable and uncorrectable in software.

The only reliable solution is to establish a shared hardware time reference that both sensors are physically locked to. This is achieved using the Pulse Per Second (PPS) signal generated by the IMU. The IMU contains a high-precision internal oscillator that produces a PPS signal which is a clean electrical pulse emitted exactly once per second. This pulse represents an accurate hardware level pulse of the IMU's clock and is entirely independent of any software layer or communication protocol. The PPS output pin of the IMU is connected directly to the synchronisation input pin of the LiDAR via a physical wire. Upon receiving each pulse, the LiDAR resets and aligns its internal clock to match the IMU's clock, locking both sensors to the same hardware time reference.

With this synchronisation in place, both the LiDAR and the IMU embed accurate hardware timestamps directly into their outgoing data packets at the moment each measurement is taken instead of at the moment the data is received by the host computer. This way even if a data packet is delayed in transit due to network congestion or USB polling latency the timestamp contained within the packet still accurately records the true physical moment the measurement occurred. This satisfies the fundamental assumption of FASTLIO and guarantees that the backward propagation integrates exactly the correct segment of IMU data for each laser point, ensuring that the de-skewing projection in Equation (4.4) is geometrically accurate.

4.3 Proposed Edge-Cloud Split-Compute Architecture

The fundamental challenge motivating this architecture is a hardware constraint established earlier in this thesis. State-of-the-art LiDAR-inertial SLAM algorithms, utilised in their full global mapping

configuration on a resource constrained embedded platform such as the Raspberry Pi 5, inevitably saturate available memory and resources and cause catastrophic system failure. As demonstrated empirically in Section 5, running FAST-LIO2 in global mapping mode caused RAM consumption to grow linearly, reaching 8.5 GB and triggering an Out-of-Memory crash on the Raspberry Pi 5. Running the same algorithm in odometry-only mode, by contrast, maintained a stable peak memory footprint. This result strictly necessitates an architectural solution that preserves real-time navigational autonomy on the UAV while offloading the computationally expensive global optimisation to a more powerful computer.

The proposed solution is an edge-cloud split compute architecture, illustrated in Figure 4.6 below. The system is divided into two distinct processing domains, an edge domain operating onboard the UAV in real time, and a cloud domain operating on a base station PC. The complete data flow between these two domains is described below.

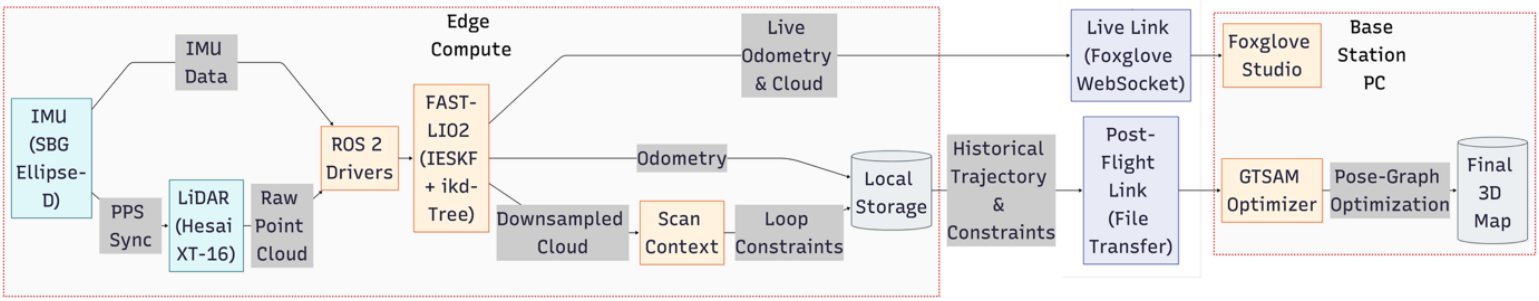

Figure 4.6: Proposed edge-cloud split compute architecture

Edge Compute Domain

The edge compute domain runs exclusively on the Raspberry Pi 5 mounted onboard the UAV. It is responsible for all time critical and mission operations that must complete within the 100ms real time limit imposed by the 10 Hz LiDAR scan rate.

The first stage of the pipeline is sensor ingestion. The SBG Ellipse-D IMU delivers high-frequency inertial measurements at over 200 Hz via its ROS 2 driver, while the Hesai XT-16 LiDAR delivers raw point clouds over an Ethernet connection. As established in Section 4.2.4, the PPS synchronisation signal is routed directly from the IMU to the LiDAR ensuring that both data streams carry accurate timestamped measurements before they enter the software pipeline.

Both data streams are consumed by FAST-LIO2, which runs the Iterated Error-State Kalman Filter (IESKF) with an incremental kd-Tree (ikd-Tree) map structure. FAST-LIO2 outputs two data streams, a real time high-frequency odometry estimate representing the UAV's current pose, and a down sampled point cloud.

The down sampled point cloud is passed to the Scan Context loop closure module. Scan Context encodes each 3D scan into a compact 2D global descriptor by dividing the point cloud into azimuthal and radial bins and recording the maximum point height in each bin. This lightweight representation allows the Raspberry Pi 5 to continuously generate loop closure candidates during flight without incurring the heavy compute cost of full pose-graph optimisation. The resulting loop closure constraints are stored in local storage alongside the odometry trajectory, ready for post-flight retrieval.

Both the live odometry and the down sampled point cloud are simultaneously transmitted over a 5 GHz wireless telemetry link via a Foxglove WebSocket connection to the base station, enabling live visualisation of the UAV's trajectory and map during flight.

Cloud Compute Domain

The cloud compute domain operates on a high-performance base station PC and assumes responsibility for all computationally expensive operations that cannot be executed within the Raspberry Pi 5's memory constraints.

During flight, the base station receives the live odometry and point cloud stream via the Foxglove WebSocket link and renders them in Foxglove Studio for real-time operator monitoring. This decouples all graphical rendering from the edge device entirely, preventing the severe memory overhead of 3D visualisation from competing with FASTLIO2 for compute.

Following the conclusion of a flight, the stored trajectory, point clouds, and Scan Context loop closure constraints are transferred from the UAV's local storage to the base station. The base station then runs pose-graph optimisation, taking the odometry poses as constraints and the Scan Context detections as loop closure constraints. Pose Graph optimisation then solves the resulting Maximum A Posteriori estimation problem, retrospectively distributing and correcting the drift accumulated throughout the

flight trajectory. In addition the base station can transform the odometry and point clouds into a unified global map.

This strict division of labour is the central engineering contribution of the system architecture. By confining the Raspberry Pi 5 exclusively to survival odometry and descriptor generation and not maintaining a global keyframe map nor executing factor graph optimisation the edge device never accumulates unbounded memory. The UAV therefore maintains reliable real-time navigational autonomy regardless of mission duration or trajectory length. Making this system suitable for real deployment.

4.4 Software Architecture

The software stack that binds the hardware components described in Section 4.2 into a functioning real-time SLAM system is built on Ubuntu 22.04 LTS running the ARM64 architecture on the Raspberry Pi 5 along with the Robot Operating System 2 (ROS 2) Humble middleware (Macenski et al., 2022). This section describes each layer of the software architecture in turn, beginning with the middleware foundation and deployment environment, and progressing through the sensor drivers, state estimator, loop closure module, global optimiser, and telemetry pipeline in subsequent subsections.

4.4.1 ROS 2 Humble & Docker Environment

ROS 2 Humble was selected as the middleware framework for this system for two primary reasons. First, it is the current supported release of ROS 2 which is the standard for modern robotics platforms and it also provides native support for the ARM64 architecture of the Raspberry Pi 5, meaning all core libraries compile and execute natively on the embedded platform. Second, ROS 2 is built on a Data Distribution Service (DDS) communication layer, which enables fully asynchronous, decentralised communication. Unlike its predecessor ROS 1, which relied on a single centralised master node that represented a single point of failure, ROS 2's DDS architecture allows the sensor drivers, state estimator, and loop closure module to operate as independent processes communicating over message topics. This decentralised model is critical for this system because it prevents the high-frequency LiDAR ingestion process from blocking or being blocked by the computationally intensive FASTLIO2 update cycle, ensuring neither pipeline starves the other of CPU time. A significant portion of the development effort

involved porting the FAST-LIO2 algorithm from its original ROS 1 implementation to the modern ROS 2.

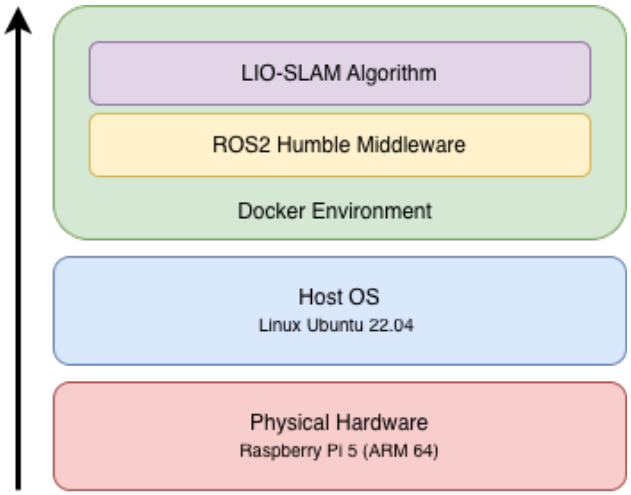

Figure 4.7: Overview of software and hardware stack

To ensure cross-platform reproducibility and eliminate the risk of dependency conflicts when deploying on the Raspberry Pi 5, the entire software environment was containerised using Docker. The Docker container encapsulates the exact library versions, build flags, and ARM64 compilation required to build and run the full software stack. This means the identical environment that was developed and validated on a Linux desktop workstation (Ubuntu 22.04) can be deployed to the embedded platform without modification. This provides the user with a simple plug and play solution for any platform. An overview of the software and hardware stack is shown in Figure 4.7 above.

4.4.2 Sensor Drivers and Quality of Service Configuration

The two physical sensors, the SBG Ellipse-D IMU and the Hesai XT-16 LiDAR each communicate with the Raspberry Pi 5 through dedicated ROS 2 driver nodes. The IMU driver receives inertial measurements from the SBG Ellipse-D at approximately 200 Hz via a serial interface and publishes them as standard `sensor_msgs/Imu` messages on a ROS 2 topic. The Hesai LiDAR driver receives raw point cloud packets from the XT-16 over Gigabit Ethernet and publishes them as `sensor_msgs/PointCloud2` messages at 10 Hz. Both drivers also handle the ingestion of the hardware timestamps embedded in each sensor's data packet and pass these through to the downstream FAST-LIO2 node so that every measurement entering the state estimator carries an accurate time reference.

A critical and deliberate aspect of the driver configuration is the Quality of Service (QoS) policy assigned to each published topic. In ROS 2, QoS policies govern how messages are transmitted and handled when the system is under load, and choosing the wrong policy for a given data stream can cause either stale data to accumulate in a queue or critical messages to be silently dropped. For this system, two distinct policies are applied depending on the nature of the data being transmitted.

The LiDAR point cloud and the IMU Data topics are configured with a Best Effort QoS policy. This means that if a UDP packet is dropped during transmission from the sensor to the driver, or if the subscriber is momentarily busy, the message is discarded rather than queued for retransmission. This is an intentional design choice. The FAST-LIO2 IESKF operates on a continuous stream of scans and is designed to process the most recent available data at each update cycle. If a stale scan were to sit in a queue and be delivered late, the state estimator would process outdated measurements, introducing timing errors that violate the fundamental assumption of the backward propagation described in Section 4.2.4. Low and consistent latency is therefore prioritised over guaranteed delivery of every individual scan.

The odometry output topic published by FAST-LIO2 is configured with the opposite policy of Reliable QoS. Unlike raw sensor data, which is continuously refreshed at a high frequency, odometry poses are sequential estimates that feed directly into the Scan Context loop closure module and are written to local storage for post-flight pose graph optimisation. A dropped odometry message would create a gap in the trajectory record, breaking the continuity of the pose graph and corrupting the loop closure constraints. Reliable QoS guarantees that every odometry message is delivered to all subscribers, accepting the marginal additional latency this entails in exchange for complete trajectory integrity. An overview of the ROS2 system including the nodes, data streams and Quality of Service configurations is shown below in Figure 4.8.

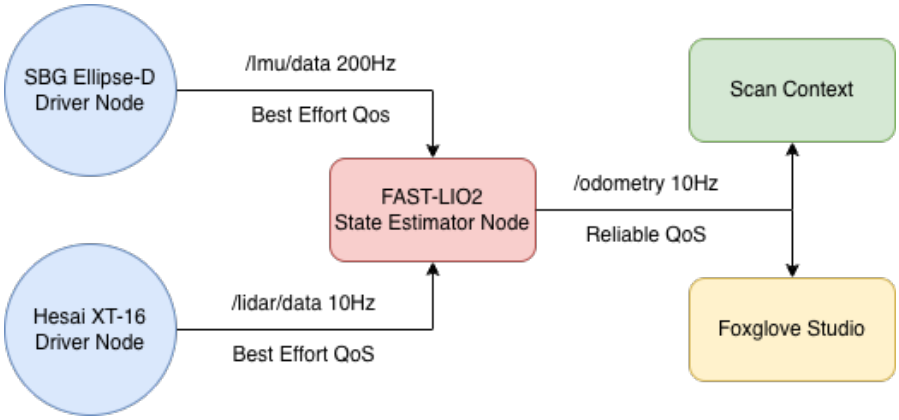

Figure 4.8: Overview of ROS2 system

4.5 State Estimation - FAST-LIO2

The core state estimation engine of this system is FAST-LIO2 (Xu et al., 2022), which is a direct LiDAR-inertial odometry framework selected specifically for its computational efficiency. This section describes the mathematical formulation of the filter, its two key innovations that make embedded deployment feasible, and the direct point registration method that replaces traditional feature extraction.

4.5.1 State Vector

First we will take a look at how FAST-LIO2 models the state vector which represents the UAV's full kinematic state. The full state vector is modelled as a 24-dimensional vector defined as (Xu et al., 2022, Eq. 2):

$$\boldsymbol{x} \triangleq [{}^{G}\boldsymbol{R}_I^T \; {}^{G}\boldsymbol{p}_I^T \; {}^{G}\boldsymbol{v}_I^T \; \boldsymbol{b}_\omega^T \; \boldsymbol{b}_\alpha^T \; {}^{G}\boldsymbol{g}^T \; {}^{I}\boldsymbol{R}_L^T \; {}^{I}\boldsymbol{p}_L^T]^T \tag{4.5}$$

This state vector is the filter's complete internal model of the drone at any given moment, it encapsulates everything needed to predict future motion and register incoming LiDAR measurements. The components are as follows, the IMU attitude ${}^{G}\boldsymbol{R}_I$ and position ${}^{G}\boldsymbol{p}_I$ in the global frame (the primary odometry output), velocity ${}^{G}\boldsymbol{v}_I$ for predicting motion between scans, gyroscope bias $\mathbf{b}_\omega$ and accelerometer bias $\mathbf{b}_\alpha$ which are estimated and corrected continuously to prevent drift, the gravity vector ${}^{G}\boldsymbol{g}$, and the LiDAR-IMU extrinsic calibration ${}^{I}\boldsymbol{R}_L$, ${}^{I}\boldsymbol{p}_L$ representing the physical rotation and offset between the two sensors on the drone body.

Most of these quantities are ordinary numbers that live in flat space $\mathbb{R}^{15}$, meaning normal arithmetic applies to them. However, the two rotation quantities being the drone's attitude ${}^{G}\boldsymbol{R}_I$, and the LiDAR-IMU rotational extrinsic ${}^{I}\boldsymbol{R}_L$ cannot be treated as ordinary numbers. Unlike position or velocity, 3D

rotations behave differently, for example rotating right then tilting forward produces a different result than tilting first then rotating. This means rotations live on a curved mathematical space called $SO(3)$, where standard addition and subtraction are not valid and special geometric operations must be used instead. The full state therefore lives on the compound manifold $\mathcal{M} = SO(3) \times \mathbb{R}^{15} \times SO(3) \times \mathbb{R}^3$, and FAST-LIO2 operates the Kalman filter directly on this manifold to ensure orientation estimates remain geometrically valid at all times, even during the aggressive high-rotation manoeuvres of UAV flight.

4.5.2 The Iterated Error-State Kalman Filter

Rather than estimating the UAV's absolute global pose directly, the Iterated Error-State Kalman Filter (IESKF) estimates the error between the predicted state and the true state. This error-state formulation offers two key advantages. First, it keeps the estimated quantities small which reduces the linearisation error introduced when approximating the non-linear dynamics. Second, it allows the filter to iterate the linearisation multiple times per LiDAR scan until the estimate converges, handling the non-linear rapid rotations and accelerations characteristic of UAV flight far more accurately than a single-step Extended Kalman Filter.

The filter operates in two alternating steps. The forward propagation step runs upon the arrival of each IMU measurement at over 200 Hz. It integrates the IMU kinematic model to propagate the state and its covariance estimate forward in time, producing a predicted state $\hat{\mathbf{x}}_k$ at scan-end time t_k. This predicted state also drives the backward propagation motion compensation described in Section 4.2.4, ensuring each laser point is correctly de-skewed before entering the update step.

The update step runs at the LiDAR scan rate of 10 Hz. For each incoming scan, each raw LiDAR point ${}^{L}\boldsymbol{p}_J$ is projected into the global frame using the current state estimate and its five nearest neighbours in the ikd-Tree map are retrieved. These neighbours are used to fit a local plane patch, and the perpendicular distance from the projected point to that plane defines the residual $\mathbf{z}_j^{\kappa}$ (Xu et al., 2022, Eq. 9):

$$\mathbf{z}_j^{\kappa} = \boldsymbol{u}_j^T \left({}^{G}\hat{T}_{L_k}^{\kappa} \cdot {}^{I}\hat{T}_{L_k}^{\kappa} \cdot {}^{L}\boldsymbol{p}_j - {}^{G}\boldsymbol{p}_j \right) \tag{4.6}$$

where $\mathbf{u}_j$ is the plane normal vector and ${}^{G}\boldsymbol{p}_j$ is a point on the plane. The filter then solves a Maximum A Posteriori (MAP) estimation problem that minimises the sum of the prior state uncertainty and the measurement residuals across all m points in the scan (Xu et al., 2022, Eq. 13):

$$\min_{\tilde{x}_k^\kappa} \left(||x_k \boxminus \hat{x}_k||^2_{\hat{P}_k} + \sum_{j=1}^{m} \left|\left| \boldsymbol{z}_j^\kappa + \boldsymbol{H}_j^\kappa \tilde{\boldsymbol{x}}_k^\kappa \right|\right|^2_{R_j} \right) \tag{4.7}$$

This process repeats iteratively, re-linearising around the updated estimate at each iteration, until the change between successive estimates falls below a convergence threshold ϵ.

4.5.3 Reformulated Kalman Gain

At every LiDAR scan update, the filter must make a decision given that both the IMU prediction and the LiDAR measurement are imperfect, how much should each one influence the final state estimate. The Kalman gain **K** is the mathematical weight that resolves this. It scales the residual **z** between what the LiDAR scan observed and what the filter predicted, and applies this correction to the current state estimate:

$$\hat{\boldsymbol{x}}_{k+1} = \hat{\boldsymbol{x}}_k + \boldsymbol{K}\boldsymbol{z} \tag{4.8}$$

A large **K** pulls the estimate strongly towards the LiDAR measurement; a small **K** trusts the IMU prediction more. The Kalman gain essentially fuses the IMU and LiDAR data together at every scan, without it there is no sensor fusion, only IMU dead reckoning which drifts rapidly, or raw LiDAR registration which fails during fast flight.

In the conventional formulation the gain is computed as $\mathbf{K} = \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R})^{-1}$, which requires inverting an $m \times m$ matrix where m is the number of LiDAR points per scan. For the Hesai XT-16, m is approximately 32,000, making this an $\mathcal{O}(m^3)$ operation that is too computationally expensive on an embedded ARM processor at 10 Hz. FAST-LIO2 resolves this by applying the Matrix Inversion Lemma to produce a mathematically equivalent form (Xu et al., 2022, Eq. 14):

$$\boldsymbol{K} = \left(\boldsymbol{H}^T\boldsymbol{R}^{-1}\boldsymbol{H} + \boldsymbol{P}^{-1}\right)^{-1}\boldsymbol{H}^T\boldsymbol{R}^{-1} \tag{4.9}$$

The fusion result is identical, but the matrix now inverted has size $n \times n$ where $n = 24$ is the fixed state dimension. This reduces complexity from $\mathcal{O}(m^3)$ to $\mathcal{O}(n^3)$, completely decoupling the computation cost from the point cloud density and making real-time odometry on the Raspberry Pi 5 possible .

4.5.4 Direct Point Registration and the ikd-Tree

Unlike predecessor systems such as LOAM and LIO-SAM which extract edge and plane feature points, FAST-LIO2 registers every raw LiDAR point directly to the map. This direct method offers two advantages relevant to subterranean mapping. First, it eliminates the need for an engineered feature extraction module that is tuned to a specific LiDAR scanning pattern. Second, in geometrically degenerate environments such as long featureless cave feature extraction frequently fails and produces too few points to make the pose estimate reliable, whereas the direct method retains and exploits every available geometric constraint from the raw scan.

The increased number of points registered per scan would be computationally infeasible if the map was maintained as a static k-d tree, since a full rebuild of the tree after every scan insertion has $\mathcal{O}(M \log M)$ complexity where M is the total number of map points. As the map grows to millions of points over a long subterranean mission this rebuild time eventually exceeds the 100ms real-time budget and causes the system to drop frames. FAST-LIO2 addresses this with the incremental k-d tree (ikd-Tree) data structure (Xu et al., 2022), which supports point-wise insertion, on-tree downsampling, and box-wise deletion in $\mathcal{O}(\log M)$ time without requiring a full rebuild. This in turn keeps RAM consumption stable allowing prolonged flight without RAM saturation.

4.6 Global Loop Closure Module

As established in Section 3.1.3, FAST-LIO2 is a pure odometry framework. While its direct point registration method provides exceptional local tracking accuracy and computational efficiency, the system accumulates unbounded drift over extended trajectories as small sensor noise and IMU bias estimation errors compound with every scan. For short missions this drift is negligible, as demonstrated by the NCLT APE results in Section 6.2, but for extended subterranean missions, the accumulated error will eventually degrade map quality and compromise the UAV's ability to return to its starting position safely.

The global loop closure module addresses this by detecting when the UAV revisits a previously mapped location and generating a geometric constraint that anchors the drifting trajectory back to the earlier

visit. When a loop closure is detected, the resulting constraint is stored locally and passed to the pose-graph optimiser at the base station for retrospective trajectory correction. The module is composed of three sequential stages: keyframe selection, Scan Context descriptor generation, and pose-graph optimisation, each described in the following subsections.

4.6.1 Keyframe Selection

Generating a Scan Context descriptor for every LiDAR scan at 10 Hz would be computationally wasteful and produce a redundant scans where consecutive entries describe nearly identical scans. Instead, a distance-based keyframe selection strategy ensures descriptors are only generated when the UAV has moved sufficiently far from its last recorded position to provide genuinely new geometric information.

A new keyframe is registered when the Euclidean distance between the current estimated position and the position of the most recent keyframe exceeds a fixed threshold d_{kf}:

$$\| \mathbf{L}_c - \mathbf{L}_l \| > \mathrm{d} \tag{4.10}$$

where $\mathbf{L}_t$ is the UAV's current 3D position in the global frame at time t, $\mathbf{L}_c$is the 3D position at which the most recent keyframe was saved, and d is the minimum travel distance required to trigger a new keyframe. When this condition is satisfied, the current downsampled point cloud is passed to the Scan Context descriptor generator and $\mathbf{L}_l$ is updated to $\mathbf{L}_c$.

4.6.2 Scan Context Descriptor

Once a keyframe is selected, the corresponding down sampled point cloud is encoded into a compact 2D global descriptor using the Scan Context algorithm (Kim and Kim, 2018). The scan context algorithm effectively unrolls the 360° LiDAR scan into a compact 2D image where rows (i), represent radial distance rings from the centre of the scan. Columns (j), represent azimuthal angle sectors (viewing direction). Pixel value, represents the maximum height of points in that specific bin. To determine if the current location matches a historical location, the system compares the current scan query (I^Q) to a historical candidate (I^C). The similarity is calculated using the Cosine Distance (d) between their respective column vectors which is normalised by the number of sectors N_S.

$$d(I^Q, I^C) = \frac{1}{N_s}\sum_{j=1}^{N}\left(1 - \frac{c_j^q \cdot c_j^c}{||\, c_j^q||\, ||c_j^c||}\right) \tag{4.11}$$

Where:

- c_j^q and c_j^c are The j-th column vectors of the Query (Current) Scan and Candidate (Historical) Scan.
- The term $\frac{c_j^q \cdot c_j^c}{||\, c_j^q||\, ||c_j^c||}$ represents the cosine similarity, which measures the alignment of the geometric profile in that specific direction.

A critical challenge in cave mapping is that the drone may return to an area from a different direction. In the Scan Context matrix, a physical rotation of the drone results in a column shift of the image. To achieve rotation invariance, the system calculates the distance for all possible column shifts n and identifies the minimum distance defined as:

$$D(I^Q, I^C) = \min_{n \in N_s} d\left(I^Q, I_N^C\right) \tag{4.12}$$

Where I_N^C represents the candidate image with its columns shifted by n indices. This minimisation guarantees that the system can recognise the same cave geometry regardless of the drone's orientation.

4.6.3 ICP Geometric Verification

The Scan Context comparison described in Section 4.6.2 identifies candidate loop closures based on geometric similarity, but it does not produce a six degree-of-freedom relative transformation that is required by the pose graph optimiser. The cosine distance in Equation 4.12 yields only a similarity score and an approximate yaw offset. Furthermore, because the descriptor operates on a compressed 2D representation, false positive detections can occur. A geometric verification step is therefore required to both validate the candidate and compute the precise relative pose constraint.

This verification is performed using the Iterative Closest Point (ICP) algorithm (Besl and McKay, 1992). ICP computes the rigid transformation which is a rotation **R** and translation **t** that best aligns a source point cloud to a target point cloud. The algorithm operates iteratively, for each point in the source cloud, the nearest point in the target cloud is identified and the rotation and translation that

minimise the sum of squared distances between all matched pairs are computed. The source cloud is then transformed and the process repeats until convergence. Besl and McKay (1992, Eq. 22) define the point-to-point objective as:

$$f(\vec{q}) = \frac{1}{N_p} \sum_{i=1}^{N_p} \| \vec{x}_i - \mathbf{R}(\vec{q}_R)\vec{p}_i - \vec{q}_T \|^2 \tag{4.13}$$

where $\vec{p}_i$ is a source point, $\vec{x}_i$ is its nearest neighbour match in the target cloud, **R** is the rotation matrix parameterised by the unit quaternion $\vec{q}_R$, and $\vec{q}_T$ is the translation vector. The algorithm converges to the nearest local minimum of this metric.

ICP requires a reasonable initial alignment to avoid converging to an incorrect local minimum. This initial guess is provided by the Scan Context module. The column shift index that minimises the descriptor distance in Equation 4.12 directly encodes the approximate yaw rotation between the two scans, which is used as the initial rotational guess for ICP. Following convergence, alignment quality is evaluated using a threshold. If the fitness score exceeds the acceptance threshold, the resulting transformation is accepted as a verified loop closure constraint and passed to the pose graph optimiser

4.6.4 Pose Graph Optimisation

When the ICP verification described in Section 4.6.3 confirms a loop closure, the accumulated odometry trajectory and loop closure constraints are passed to a pose graph optimiser for retrospective trajectory correction. The optimisation is implemented using the Open3D library (Zhou, Park and Koltun, 2018).

The trajectory is modelled as a pose graph in which each node represents the pose of the UAV at a keyframe, and edges encode geometric constraints between nodes. Two types of edges are used. Odometry edges connect consecutive keyframes using the relative transformation extracted from the FAST-LIO2 trajectory, forming a chain that connects the drifted path. Loop closure edges connect non-consecutive keyframes identified by the Scan Context and ICP pipeline, providing the corrective constraints that anchor the drifting trajectory back to previously visited locations.

The key challenge is that the odometry chain and the loop closure edges will disagree, the chain says the robot drifted 2 metres from the start, while the loop edge says it returned to the start. The optimiser resolves this contradiction by adjusting all keyframe poses simultaneously to minimise the total disagreement across all edges, distributing the accumulated drift smoothly along the trajectory rather than concentrating the correction at a single point.

4.7 Validation Strategy & Key Performance Indicators

The validation of this system is structured across two phases. Phase 1 evaluates computational performance and odometry accuracy using the NCLT dataset (Carlevaris-Bianco, Ushani and Eustice, 2016), executed directly on the Raspberry Pi 5 to ensure reported figures reflect the true constraints of the embedded platform. Phase 2 deploys the fully assembled payload across three real-world environments, validating the complete end-to-end system under mission-representative conditions. Where no external ground truth exists, trajectory accuracy is quantified using the closed-loop closing error metric.

Five Key Performance Indicators (KPIs) are defined to evaluate system performance consistently across both phases and are summarised in Table 4.2.

Absolute Pose Error (APE): quantifies global trajectory accuracy as the RMSE of the Euclidean distance between estimated and ground truth poses after rigid body alignment.

$$\text{APE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \| \mathbf{p}_i - \hat{\mathbf{p}}_i \|^2} \tag{4.14}$$

Closing Error: quantifies trajectory drift for Phase 2 deployments where no ground truth is available, by measuring the distance between the estimated final pose and the known starting position upon completing a closed loop:

$$e_{close} = \| \mathbf{p}_{end} - \mathbf{p}_{start} \| \tag{4.15}$$

Processing Latency: measures the delay between LiDAR scan acquisition and odometry publication. This must remain below 100ms at all times to match the 10 Hz scan rate and prevent frame drops.

CPU Load: is monitored as a stability margin. Sustained sub 100% utilisation across all cores prevents the OS from scheduling processes within their required time windows, causing frame drops and system freezing.

Peak RAM Usage: monitors maximum memory allocation. Exceeding the Raspberry Pi 5's physical limit causes either an Out-of-Memory process kill or SD card swapping, both of which are fatal to the real-time pipeline.

KPI	Target
Absolute Pose Error	Minimise
Closing Error	Minimise
Processing Latency	< 100 ms
CPU Load	<100 %
Peak RAM Usage	< 8 GB

Table 4.2: Key Performance Indicators for system evaluation.

5. Implementation

This chapter documents the end-to-end engineering process of transforming the system architecture described in Chapter 4 from design into a physically deployed, real-time SLAM payload. The work was structured across three development sprints, 1st software integration, 2nd hardware integration and finally algorithmic extension. The complete source code, including the Docker environment, all configuration files, benchmarking scripts, and loop closure implementation, is openly available at https://github.com/Stepanletsko/LIO_SLAM.

5.1 Development Environment & Containerisation

5.1.1 Host Workstation Configuration

Initial development was conducted on a Dell workstation running Ubuntu 24.04.3 LTS on an x86-64 architecture. This immediately introduced a cross-platform challenge since the target deployment hardware is the Raspberry Pi 5 running an ARM64 architecture. This means any software built natively on the x86 workstation cannot run directly on the embedded platform. Furthermore, Ubuntu 24.04 does not have a Tier 1 supported ROS 2 release at the time of development, meaning native ROS 2 installation on the host workstation would introduce compatibility risks with sensor drivers and third-party packages that target the Ubuntu 22.04 and ROS 2 Humble combination. Rather than attempting

to resolve these cross-platform and OS compatibility issues manually the decision was made to address both problems simultaneously through containerisation.

5.1.2 Docker Container Design

To resolve the cross-platform incompatibility between the x86 Dell workstation and the ARM64 Raspberry Pi 5, and to avoid the dependency conflicts that arise from developing on Ubuntu 24.04 against a ROS 2 Humble stack that officially targets Ubuntu 22.04, the entire software environment was containerised using Docker. The container is built from a ROS 2 Humble base image on Ubuntu 22.04 and installs all required dependencies at image build time, including the Point Cloud Library (PCL) (Rusu and Cousins, 2011), rosbags, numpy, pandas, matplotlib, psutil, Open3D (Zhou et al., 2018), and the evo trajectory evaluation package (Grupp, 2017). This produces a fully self-contained, reproducible environment that runs identically on the development workstation and the Raspberry Pi 5 without any manual configuration on either platform. The relevant docker file is shown in appendix A.3.

The container lifecycle is managed by a `docker-compose.yml` file which mounts four directories from the host into the container at runtime. The `src/` directory is mounted so that source code edits on the host are immediately reflected inside the container without requiring an image rebuild. The `build/` and `install/` directories are mounted so that compiled artifacts persist between container restarts, avoiding redundant full recompilation during iterative development. Additionally the `.git/` directory is mounted to allow Git operations. The complete configuration is available in the project repository.

5.2 FAST-LIO2 ROS 1 to ROS 2 Migration

The upstream FAST-LIO2 repository is implemented against ROS 1, which uses a fundamentally different communication and build architecture to ROS 2. Porting it to ROS 2 Humble required changes across such as the build system, the node communication model, and the sensor driver interfaces.

System Build:

ROS 1 uses the `catkin` build system whereas ROS 2 uses `ament_cmake`. All `CMakeLists.txt` and `package.xml` files were changed to comply with the `ament_cmake` build format, including updating

dependency declarations, target linking, and the package export format. The Livox ROS driver, which is a dependency of the original FAST-LIO2 for Livox solid-state LiDARs, required its own `package.xml` to be created from scratch as the upstream repository did not include a ROS 2 compatible package manifest. This was added as `package.xml` with the correct `ament_cmake` format declaring all required dependencies including `rclcpp`, `sensor_msgs`, `pcl_conversions`, and `rosidl` interface packages.

QoS Policy Configuration:

ROS 2 introduces Quality of Service policies that have no direct equivalent in ROS 1. During initial testing on the Hesai XT-16 LiDAR driver, point cloud messages were being silently dropped due to a QoS incompatibility between the driver publisher and the FAST-LIO2 subscriber, a mismatch that does not exist in ROS 1 and produced no error messages, making it a difficult bug to diagnose. The LiDAR point cloud subscriber was re-configured with a Best Effort QoS profile to match the driver's publisher policy, eliminating the silent drops. The odometry output topic was separately configured with a Reliable QoS policy to guarantee delivery to downstream nodes.

5.3 Sensor Driver Integration

With the core FAST-LIO2 framework ported to ROS 2, the next stage was integrating the two physical sensors into the ROS2 pipeline. This involved configuring both drivers, resolving a significant point cloud data type incompatibility between the Hesai XT-16 and FAST-LIO2, and tuning both drivers for real-time performance on the Raspberry Pi 5.

5.3.1 Hesai XT-16 LiDAR Driver Configuration

The Hesai XT-16 communicates with the Raspberry Pi 5 over Gigabit Ethernet, publishing raw point cloud data as UDP packets which are assembled by the Hesai ROS 2 driver into `/lidar_points` messages on the Hesai topic. Initial configuration set the sensor's static IP address to `192.168.1.100`. Driver performance tuning was required to achieve stable 10 Hz operation on the embedded platform. The LiDAR spin speed was set to 600 RPM to produce a 10 Hz scan rate matching the FAST-LIO2 update frequency. The packet loss diagnostic tool was also disabled as it introduced unnecessary processing overhead.

5.3.2 Hesai Point Cloud Data Type Adaptation

The most significant integration challenge was a fundamental incompatibility between the Hesai XT-16's point cloud message format and the data structures expected by FAST-LIO2. FAST-LIO2 was originally developed and benchmarked against Velodyne LiDARs from the NCLT dataset. Inspecting the raw messages format from both sensors reveals that they are structurally incompatible, as shown in Table 5.1 below:

Field	Velodyne	Hesai
x,y,z	float32, offsets 0–8	float32, offsets 0–8
intensity	absent	float32, offset 12
time	float32, offset 16 (relative, seconds within scan)	absent
timestamp	absent	float64, offset 18 (absolute UNIX time)
ring	uint16, offset 20	uint16, offset 16
point_step	22 bytes	26 bytes
points per scan	~17,400	~63,900

Table 5.1: Comparison of Velodyne and Hesai Lidar message format

There are four critical incompatibilities. First, the Velodyne provides a relative `time` field in float32, the exact format FAST-LIO2 expects for per-point motion de-skewing. The Hesai provides no such field. Second, the Hesai instead provides an absolute UNIX `timestamp` in float64, from which relative times must be derived. Third, all field byte offsets differ between the two sensors, meaning the Velodyne parser cannot be reused at all. Fourth, the Hesai produces approximately 3.7 times more points per scan, increasing the computational load on the preprocessing pipeline significantly.

To resolve this, a custom `HesaiPointXYZIRT` struct was defined in `common_lib.h` describing the Hesai data format and registered with PCL's point cloud framework (Appendix A.1). A dedicated case 5 handler was written in `preprocess.cpp` which iterates directly over the raw message memory. The per-point relative timestamp required by FAST-LIO2 for backward IMU propagation is derived as:

$$\rho_j^{rel} = clamp\left(\left(\rho_j^{abs} - t_{base}\right) \times 1000, 0, 150\right) ms \tag{5.1}$$

where t_{base} is the absolute UNIX timestamp of the first valid point in the scan. The clamp to $[0, 150]$ms prevents a corrupted timestamp from causing the backward propagation to integrate over an absurdly large time window, which would immediately destabilise the IESKF.

Three additional safety measures were added that are absent from the original FAST-LIO2 handlers. NaN and infinite coordinate values are explicitly filtered before any point enters the ikd-Tree, since a single invalid coordinate causes undefined behaviour in the nearest-neighbour search. Out-of-range points inside the blind zone are discarded. Finally, a memory leak was discovered and then fixed, the `pl_surf`, `pl_corn`, and `pl_full` buffers were not being cleared at the start of each scan in the Hesai case, causing points to accumulate across scans and producing $O(N^2)$ memory growth that crashed the system during extended runs. Adding explicit `.clear()` calls at the entry of the handler resolved this (Appendix A.1).

5.3.3 SBG Ellipse-D IMU Driver Configuration

The SBG Ellipse-D IMU driver was configured to output inertial measurements at 200 Hz, which required increasing the serial baud rate to 921600 bps with automatic fallback to 115200 bps for initial negotiation. The ENU (East-North-Up) coordinate convention was enabled to ensure the IMU's output frame is consistent with FAST-LIO2's expected global frame convention. Only the required ROS output topics were enabled such as `log_imu_data` at 200 Hz with all unused log channels such as `imu_short` and `ekf_euler` disabled to prevent serial bandwidth saturation that would otherwise reduce the effective output rate below the required 200 Hz. The Hesai FAST-LIO2 configuration was updated to subscribe to the `/imu/data` topic published by the SBG driver. The newly made `hesai.yaml` configuration is also available in Appendix A.2.

5.4 Physical Sensor Payload Design and Assembly

5.4.1 Design Requirements and CAD Modelling

We will now move onto the design and assembly of the physical sensor payload. The physical sensor payload was designed with one primary mechanical constraint in mind. That being that all three components, the Hesai XT-16 LiDAR, the SBG Ellipse-D IMU, and the Raspberry Pi 5 must be rigidly fixed relative to one. This is a requirement of the FAST-LIO2 algorithm, which assumes that the extrinsic transform between the LiDAR and IMU coordinate frames is static throughout operation. Any relative movement between the sensors during a mission would invalidate this assumption and introduce systematic errors into the de-skewing and state estimation pipeline.

The mounting platform was designed from scratch in Autodesk Fusion 360, using the official 3D CAD models of all three components to design the structure around their exact geometry. This ensured precise fit and correct spatial relationships between components without requiring physical trial and error. As illustrated in Figures 5.1 and 5.2, the design places the Hesai XT-16 elevated above the main platform deck, with the Raspberry Pi 5 and SBG Ellipse-D mounted flush on the deck surface below and to the rear of the LiDAR. The elevated LiDAR position was a deliberate design decision to ensure the rotating laser has an unobstructed 360° field of view. Mounting the LiDAR at the same height as the compute and IMU hardware would cause the platform body to obstruct a significant angular sector of the scan, creating a permanent blind spot in the point cloud.

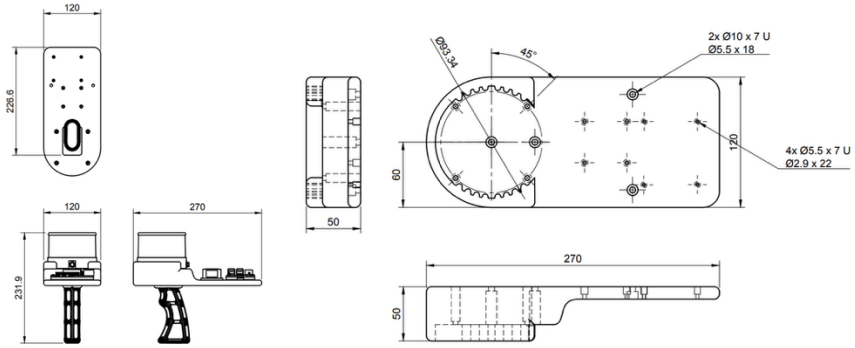

Figure 5.1: Technical assembly drawing showcasing overall dimensions

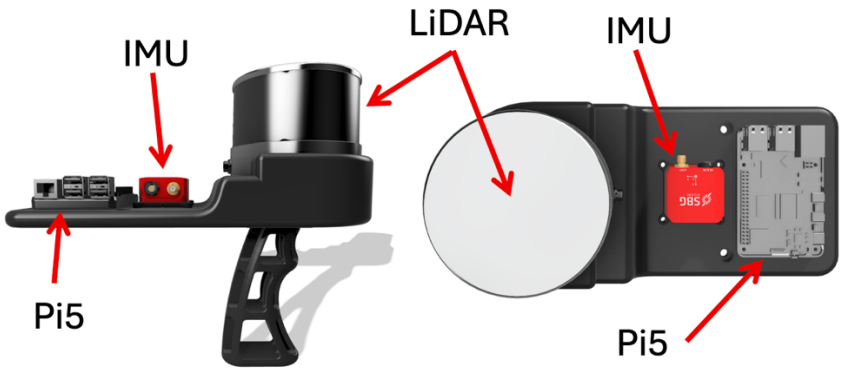

Figure 5.2: CAD render of the sensor payload showing side and top views. The Hesai XT-16 LiDAR is elevated above the platform deck to provide an unobstructed 360° field of view.

A key design consideration during CAD modelling was the deliberate alignment of the IMU and LiDAR coordinate axes. The SBG Ellipse-D was oriented on the platform such that its X, Y, and Z axes are parallel to those of the Hesai XT-16's body frame. This means the rotational extrinsic between the two sensors is the identity matrix:

$$ {}^{I}\boldsymbol{R}_{L} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{5.2} $$

This simplifies the initial configuration of FAST-LIO2 and the extrinsic calibration, as the filter only needs to refine a small translational offset rather than a combined rotation and translation. The translational extrinsic ${}^{I}\boldsymbol{p}_{L}$ was measured directly from the CAD model as the vector from the centre of the LiDAR body frame to the centre of the IMU body frame:

$$ {}^{I}\boldsymbol{p}_{L} = [0.0, -0.106368, 0.052867]\ m \tag{5.3} $$

These values were entered directly into the `extrinsic_T` and `extrinsic_R` fields of the Hesai configuration file `hesai.yaml` which can be seen in Appendix A.2.

5.4.2 Mechanical Assembly

The platform structure was 3D printed in PLA. All three components were secured to the platform using bolts and nuts, allowing individual components to be detached and reattached for maintenance, testing, or replacement without discarding the entire assembly. This modular fastening approach also allows the mounting orientation of any component to be adjusted if future recalibration is needed.

As shown in the technical drawings in Figure 5.1 the overall sensor platform is 270 mm × 120 mm × 50mm. The LiDAR mounting platform uses a toothed ring pattern to provide support.

The platform includes a grip handle attached to the underside of the deck, allowing the system to be carried and operated by hand for mobile ground testing. The handle is detachable, meaning the same platform can be bolted directly to a UAV frame for aerial deployment by removing the handle and using the deck's mounting holes as attachment points to the airframe. However due to no availability of a UAV for this thesis the grip handle will be used for manual testing. The final physical assembled prototype is shown below in Figure 5.3.

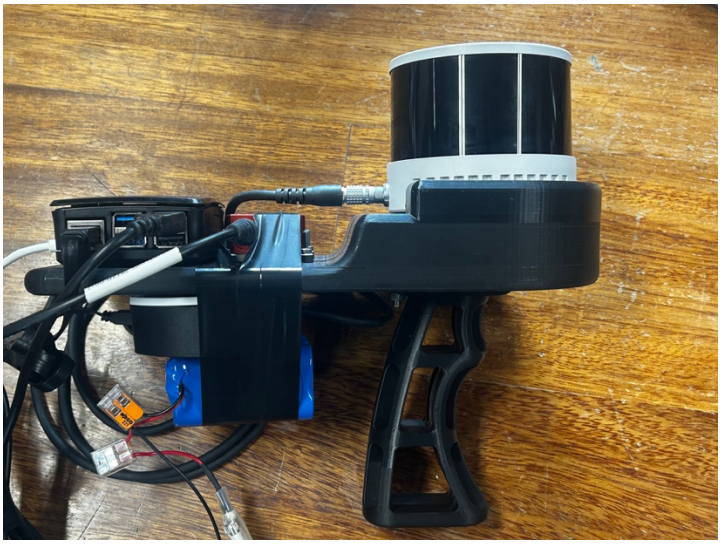

Figure 5.3: Final physical assembled prototype

5.5 PPS Hardware Synchronisation & Electrical Wiring

As established in Section 4.2.4, accurate LiDAR point cloud de-skewing requires that the per-point timestamps embedded in each scan are expressed on the same clock reference as the IMU measurements. Prior to hardware synchronisation both sensors operated on independent clocks, the SBG Ellipse-D on its internal oscillator and the Hesai XT-16 using the receive timestamp from the Raspberry Pi 5's system clock with no guaranteed alignment between them. The hardware synchronisation implementation described in this section locks both sensors to a shared 1 Hz pulse derived from the SBG's internal IMU clock, eliminating this timing uncertainty entirely allowing for accurate motion de-skewing. We will also take a look at the general electrical wiring setup required to operate the hardware setup discussed in section 5.4. The diagram in Figure 5.4 below shows the general electrical wiring setup.

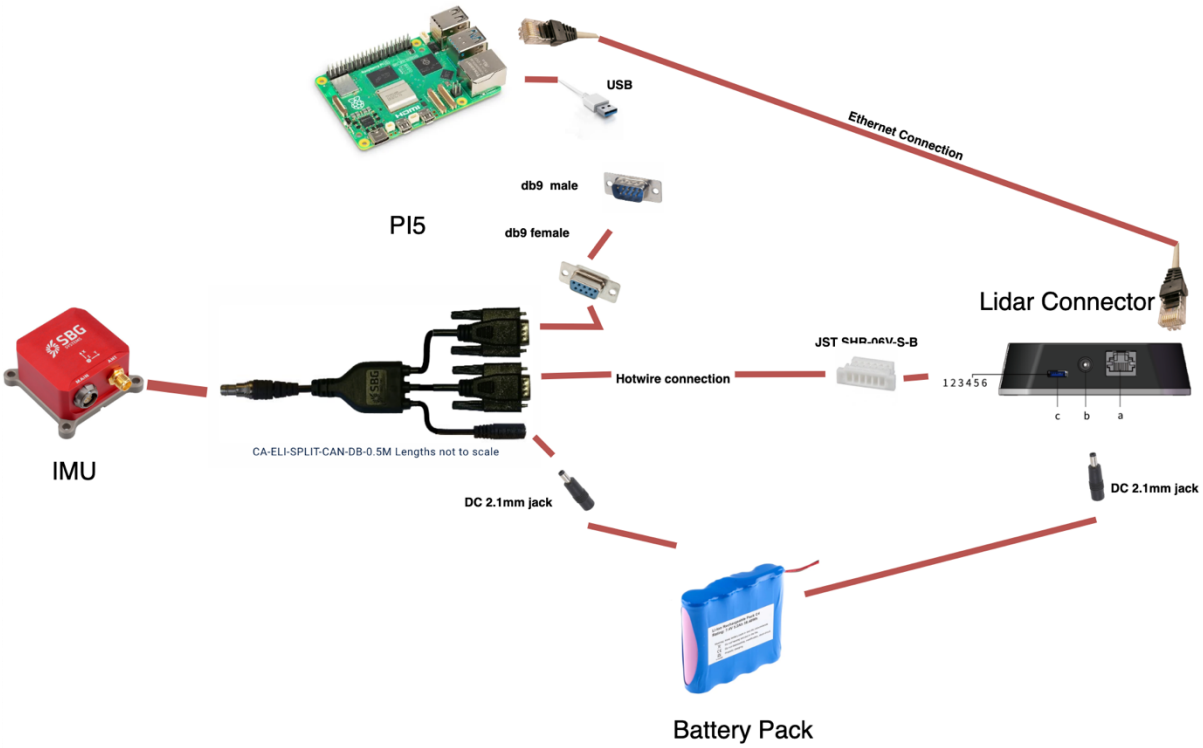

Figure 5.4: Hardware system electrical wiring

The sensor payload requires three power sources to satisfy the differing voltage and current requirements of each component. The Hesai XT-16 LiDAR operates from a 12 V DC supply connected through the sensor's external connection box, drawing approximately 8 W during normal operation at 10 Hz (Hesai, 2026). The SBG Ellipse-D IMU accepts a wide input voltage range of 5–36 V and draws 750 mW at 12 V (SBG Systems, 2024). Both the LiDAR and IMU are therefore powered from a shared 12 V lithium battery pack via the DC 2.1 mm barrel jack on the CA-ELI-SPLIT-RS232-DB-0.5M splitter cable, with the centre pin carrying VCC and the outer ring carrying GND. The Raspberry Pi 5 is powered independently from a dedicated USB-C power bank, as it operates from a 5 V USB-C input and draws approximately 12 W under full computational load (Raspberry Pi Ltd., 2023).

5.5.1 Communication & Synchronisation

As we can see from Figure 5.4 the Hesai Lidar communicates to the PI5 via an ethernet cable, this allows for the high throughput of data that the lidar required to transfer point cloud data. While for the SBG IMU the CA-ELI-SPLIT-RS232-DB-0.5M splitter cable exposes two physically separate DB9 connectors from the SBG Ellipse-D's main connector, each serving a distinct purpose. The first connector, labelled PORT A, is the primary bidirectional RS232 serial communication interface through

which all configuration commands and IMU data are exchanged with the Raspberry Pi 5. A custom cable connects PORT A to a USB-to-RS232 adapter, with the TX and RX lines intentionally crossed, SBG Pin 3 (TX) routed to Adapter Pin 2 (RX), and SBG Pin 2 (RX) routed to Adapter Pin 3 (TX) this implements the required crossover that allows the two device's to communicate. This connection carries the 200 Hz IMU data stream and all configuration traffic between the SBG and the Raspberry Pi 5 at 460800 bps.

The second connector, labelled SYNC/PORT E, serves a completely different purpose and is used exclusively for hardware synchronisation. It exposes the SBG's dedicated SYNC OUT A signal on Pin 4, an auxiliary serial port on Pins 2 and 3, and a ground reference on Pin 5. It is through this connector that the PPS timing signal is routed to the Hesai XT-16 LiDAR.

The SYNC OUT A pin is configurable via sbgCenter, the SBG Systems proprietary Windows configuration GUI. Since the system operates underground without GPS coverage, the standard GPS-derived PPS mode cannot be used, it requires a valid GPS fix to generate a time-locked pulse and would remain permanently inactive in a subterranean environment. Instead, the sync output is configured to use the SBG's internal IMU oscillator as its time base, producing a free-running pulse regardless of GPS availability:

- Mode: PPS (1 Hz, driven by internal IMU clock)
- Polarity: Rising edge
- Duration: 50 ms

This produces a 1 Hz square wave on SYNC OUT A with a 50 ms pulse width and 3.39 V peak amplitude, as verified using an oscilloscope shown in Figure 5.5 below.

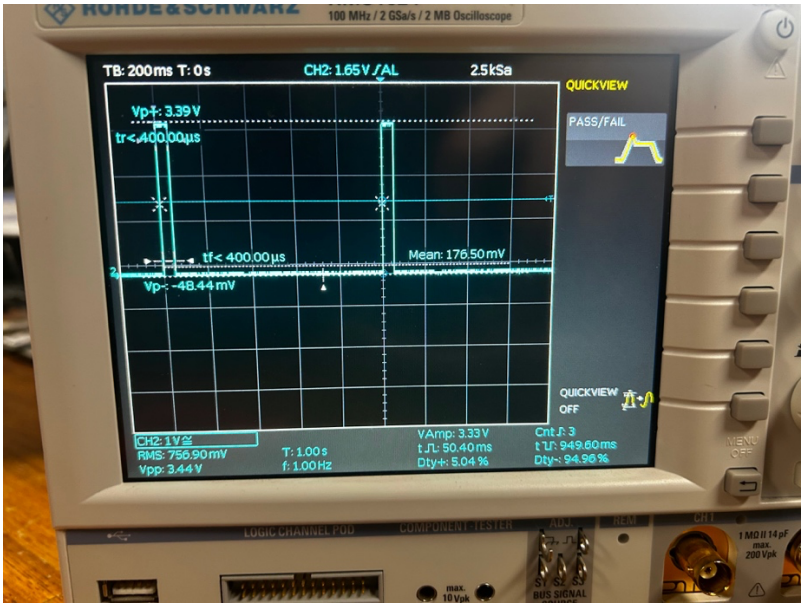

Figure 5.5: Oscilloscope measurement showing PPS signal

The measured signal parameters are compared against the Hesai XT-16 time synchronisation port requirements in Table 5.2 below:

Parameter	Measured	Hesai Requirement
Frequency	1 Hz	1 Hz $\pm$ 50 μs
Period	1 s	1 s $\pm$ 50 μs
Pulse Width	50.4 ms	$\geq$ 1 ms (rec. 10 – 100 ms)
Peak Voltage	3.39 V	3.3V – 12V

Table 5.2: Measured vs required time synchronisation parameters

All measured parameters are within the Hesai's specified requirements. The 50 ms pulse width falls within the recommended range of 10–100 ms, and the 3.39 V peak voltage is within the Hesai's accepted input range of 3.3 V–12 V, confirming that no level converter is required between the SBG output and the Hesai input.

5.5.2 PPS Physical Wiring

The synchronisation PPS signal connection routes from the SYNC/PORT E DB9 connector on the SBG splitter cable to the Hesai XT-16's JST SHR-06V-S-B GPS port on the connection box using three wires that are mapped out in Table 5.3 below.

SBG SYNC/PORT E DB9	Hesai GPS Port (JST SHR-06V-S-B)
Pin 4 (SYNC OUT A)	Pin 1 (PPS input)
Pin 3 (PORT E TX - NMEA)	Pin 4 (serial data input)
Pin 5 (GND)	Pin 3 (GND)

Table 5.3: SBG to Hesai pinout map

The PPS wire on Pin 4 → Pin 1 delivers the 1 Hz rising edge pulse that the Hesai uses to reset and align its internal scan clock. The NMEA wire on Pin 3 → Pin 4 carries NMEA timing sentences from the SBG's auxiliary PORT E serial output to the Hesai's GPS serial data input, providing a coarse time reference that the Hesai uses alongside the PPS pulse. The ground wire on Pin 5 → Pin 3 provides the common voltage reference required for correct signal interpretation. The zoomed wiring detail is shown in Figure 5.6 below:

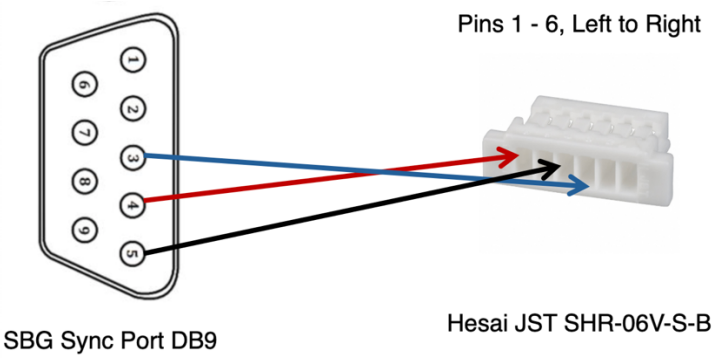

Figure 5.6: SBG to Hesai pinout diagram

5.5.3 Driver Reconfiguration

With hardware synchronisation active, both the Hesai and SBG drivers were reconfigured to use their internal hardware point cloud timestamps rather than the Pi 5 system clock receive timestamps.

However a software issue was discovered during integration testing. The SBG ROS 2 driver conditionally suppresses NMEA output when the device does not have a valid GPS time fix, it checks for GPS synchronisation validity before publishing NMEA sentences and silently drops them if no GPS connection is present. Since the system operates underground without GPS coverage this condition is never met, meaning the Hesai would receive no NMEA timing reference on its GPS serial input and could not complete its synchronisation procedure.

The fix involved modifying the SBG ROS 2 driver to remove this GPS validity condition, allowing NMEA sentences to be published unconditionally from the SBG's internal clock regardless of GPS fix status. The absolute time values in the NMEA sentences are incorrect without GPS since the internal clock has no UTC reference, but this does not affect synchronisation correctness. The Hesai only requires that the NMEA time increments consistently and aligns with the PPS rising edge. Since both

the NMEA sentences and the PPS pulses originate from the same SBG internal oscillator they are by definition synchronised, and the Hesai can use them to lock its scan timing correctly regardless of whether the absolute time is accurate.

5.6 Benchmarking & Analysis Toolchain

Before any algorithmic optimisation or physical hardware testing could be conducted, a dedicated suite of Python scripts was developed to provide rigorous, repeatable measurement of the system's computational performance. Without this instrumentation infrastructure, claims about real-time performance, RAM stability, and parameter optimality would rest on qualitative observation rather than quantitative evidence. All scripts in the toolchain operate inside the Docker container and are available in the project repository under the `scripts/` directory.

The two foundational measurement tools are `profile_fastlio.py` and `benchmark_fastlio.py`, which address complementary aspects of system performance.

`profile_fastlio.py` attaches to the running FAST-LIO2 process using the `psutil` library and samples CPU utilisation and RAM consumption at a fixed interval while a ROS 2 bag is replayed through the pipeline. The sampled data is written to a CSV file and plotted as time-series graphs. This script was the primary instrument used to discover the Out-of-Memory crash behaviour described in Section 5.9, as it made the linear RAM growth under global mapping mode directly observable. It was also used to generate the stable memory profiles that validated the odometry-only configuration.

`benchmark_fastlio.py` addresses processing latency specifically. Rather than inferring latency from odometry publish timestamps, it parses FAST-LIO2's own internal C++ timing log output to extract per-frame breakdown times including ICP registration, map update, filter solve, and total frame duration. This approach captures the true algorithmic processing time independent of any ROS 2 communication or scheduling overhead. The extracted timing data is saved to a CSV file and can be

plotted independently using `plot_latency.py`, which renders per-frame latency over the duration of a run.

`run_full_analysis.py` is the master orchestration script that chains the entire evaluation pipeline into a single automated run. Given a bag file as input, it sequentially launches FAST-LIO2, attaches the CPU and RAM sampler, extracts the odometry trajectory, saves the point cloud map to a `.pcd` file, and executes APE analysis against the ground truth. This end-to-end automation was essential for the parameter sweep experiments, where the same complete evaluation had to be repeated across dozens of configurations without manual intervention. The `run_full_analysis.py` file is also provided in the appendix A.4.

In addition two scripts support the data capture and map reconstruction workflow that underpins the broader edge-cloud architecture.

`record_fastlio.py` records the live sensor inputs or mapping output topics to a new ROS 2 bag file. This script is the primary data capture tool during live physical deployments. Rather than attempting to construct a full global map in RAM during flight, which Section 5.9 demonstrates is infeasible on the Raspberry Pi 5, the system instead records all SLAM outputs to local storage as the mission progresses. This recorded bag forms the post-flight data archive that is later transferred to the base station for global map construction and pose graph optimisation, and is a direct realisation of the edge-cloud split described in Section 4.3.

`bag_to_pcd.py` handles the post-flight map reconstruction step. Once `run_full_analysis.py` has finished replaying a bag through FAST-LIO2, it automatically invokes `bag_to_pcd.py` as the final stage of the pipeline. The script reads all `/lidar_points` frames from the bag sequentially, accumulates them into a single point cloud using Open3D (Zhou et al., 2018), and writes the result to a `.pcd` file. This file can then be opened in Open3D, CloudCompare, or any compatible point cloud viewer for visual inspection and geometric analysis.

Quantitative trajectory accuracy evaluation requires converting the ROS 2 odometry output and the NCLT ground truth into a common format compatible with the `evo` Python evaluation package (Grupp, 2017). Two conversion scripts handle this.

`bag_to_tum.py` reads the `/Odometry` topic from a recorded ROS 2 bag and writes the full trajectory to the TUM format (`timestamp x y z qx qy qz qw`), which `evo_ape` accepts directly for APE computation. `nclt_to_tum.py` performs the complementary conversion for the ground truth side, reading the NCLT dataset's CSV ground truth file, which provides timestamps in microseconds alongside position and Euler angles, and converting it to TUM format with identity quaternions.

5.7 Telemetry & Visualisation Pipeline

During initial simulation testing, ROS2's RViz2 was used as the primary visualisation tool for monitoring the SLAM output. However, two significant limitations became apparent as the system moved towards physical deployment. First, RViz2 struggled to render large, dense point clouds at an acceptable frame rate, causing severe graphical lag that made live monitoring impractical. This was due to RViz2's older rendering engine, which is not optimised for the scale of point clouds produced by a long continuous mapping session. Second, and more critically for the operational architecture of this system, RViz2 requires the receiving machine to be running Linux with a compatible ROS 2 installation. This would severely restrict the usability of the base station, requiring a dedicated Linux laptop in the field rather than allowing any available device to serve as the operator terminal. For a system intended for real-world subterranean deployment, this was an unacceptable constraint.

The solution to this issue was to adopt Foxglove Studio (Foxglove Studio, 2026), a cross-platform robotics visualisation and observability platform. Foxglove connects to the Raspberry Pi 5 over a WebSocket connection, which the Pi 5 can access using the `foxglove_bridge` ROS 2 package. This means that any device, Windows, macOS, or Linux with Foxglove Studio installed can connect to the live SLAM pipeline simply by entering the Pi 5's IP address, providing a genuinely plug-and-play operator interface with no ROS 2 installation required on the receiving machine. During field

deployment the Pi 5 acts as a 5 GHz Wi-Fi hotspot, and the operator laptop connects directly to it, giving the base station a live telemetry feed without requiring any external network infrastructure.

Beyond eliminating the RViz2 compatibility constraints, Foxglove's modern rendering engine handles the dense point cloud streams produced by the Hesai XT-16 without the frame drop issues observed previously, allowing smooth real-time 3D visualisation even on a standard consumer laptop. The Raspberry Pi 5 itself operates without any visualisation with no display or graphical process running on the edge device. All rendering is handled exclusively on the base station, which is critical for preserving the Raspberry Pi 5's CPU and RAM budget entirely for the SLAM pipeline processing.

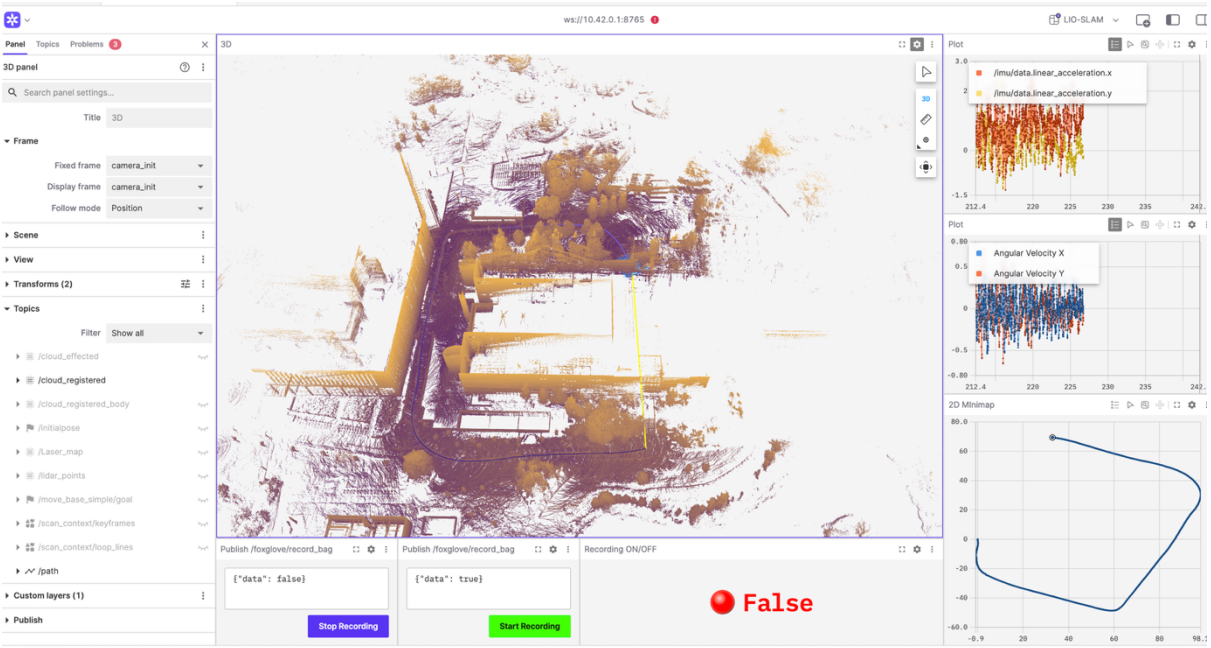

Figure 5.7: Command Centre view from Foxglove

A custom Foxglove command centre layout was developed to present all operationally relevant data simultaneously, as shown in Figure 5.7 above. The central panel renders the live 3D point cloud map as it is built in real time, with the accumulated scan coloured by height . The blue line overlaid on the map is the odometry trajectory, giving the operator a live trace of the path taken through the environment. A 2D minimap panel in the bottom right renders the same trajectory from a top-down perspective, providing a cleaner spatial overview of the mission progress. Two time-series plot panels on the top right display the live IMU data stream, showing linear acceleration and angular velocity on their respective axes at the full 200 Hz output rate, which allows the operator to immediately detect sensor anomalies or unexpected platform motion during a run. Finally, a dedicated recording control

panel at the bottom of the layout exposes start and stop recording buttons that publish to the `foxglove/record_bag` topic, allowing the operator to trigger and halt bag recording on the Raspberry Pi 5 remotely from the base station without requiring a separate SSH terminal session.

5.8 Automated Launch

With the full hardware stack assembled and the telemetry pipeline in place, a final practical challenge remained for field deployment. Operating the system in a subterranean environment means there is no monitor, keyboard, or display attached to the Raspberry Pi 5. At most, the operator has a single SSH connection from the base station laptop. Requiring the operator to manually access the Docker container, source the ROS 2 workspace, and launch each component individually via the command line before every mission would be error-prone and impractical in the field. To address this, an automated launch system was implemented that reduces the entire startup procedure to a single physical action: powering on the Pi 5.

The automation is implemented as a system service at `/etc/systemd/system/slam_autostart.service`, which is registered to execute automatically on every boot of the Raspberry Pi 5. Systemd is the Linux init system responsible for managing services and their dependencies, making it the appropriate mechanism for guaranteed startup behaviour on an embedded Linux platform.

The service is configured with two startup dependencies, it waits for the Docker system to be fully initialised and for the network interfaces to be ready before executing. This prevents a race condition where the service would attempt to start the container before Docker was operational. Once both conditions are satisfied, the service first starts the `thesis_container` Docker container and then executes `mission.launch.py` inside the container, which brings up the full SLAM pipeline stack. From power-on to a fully operational SLAM pipeline takes approximately 15 seconds.

The Pi 5 was additionally configured to automatically start a 5 GHz Wi-Fi hotspot on boot, ensuring that the Foxglove WebSocket telemetry link is immediately available to the base station laptop without requiring any network configuration by the operator.

The `mission.launch.py` launch file brings up five components that are necessary to the SLAM pipeline. The SBG IMU driver and Hesai LiDAR driver are started immediately and simultaneously as the first stage, since both sensors need time to initialise and begin publishing before the SLAM algorithm can consume their data. The Foxglove bridge node is started at the same time, opening a WebSocket server on port 8765 accessible from any address on the network. A topic whitelist is applied to the bridge that excludes the raw `/lidar_points` topic from being streamed over the WebSocket, since transmitting full-resolution raw point clouds over Wi-Fi would saturate the link bandwidth. The down sampled registered cloud published by FAST-LIO2 is streamed instead, which is sufficient for live operator monitoring. The bag recorder listener script `bag_trigger.py` is also launched at this stage, which listens for the start and stop recording messages published by the Foxglove recording control panel.

FAST-LIO2 itself is started with a three-second delay, ensuring that both sensor drivers are fully initialised and publishing data before the state estimator begins consuming them. It is launched with the `hesai.yaml` configuration and with RViz2 explicitly disabled, preserving the Pi 5's CPU and RAM budget entirely for the SLAM pipeline.

5.9 RAM Optimisation & Map Management

During initial benchmarking of FAST-LIO2 on the Raspberry Pi 5, a critical memory scalability problem was identified. When running in its default global mapping configuration where the complete accumulated point cloud map is maintained in RAM throughout the mission memory consumption grew linearly and without bound, eventually triggering an Out-of-Memory (OOM) kill and crashing the system entirely. The full empirical profiling of this behaviour is presented in Section 6, but the finding confirmed that native global mapping is fundamentally unsustainable on the Raspberry Pi 5 for missions of any significant duration and directly motivated the edge-cloud split architecture described in Section 4.3. Fundamentally FASTLIO2 is excellent at using low RAM, however this is only for odometry as it simply discards old points in the map. In order to have a fully operational system we must have an ability to save a full map of the environment we wish to map.

The solution to the aggressive linear RAM growth was to disable global map saving entirely during flight and configure FAST-LIO2 to run in odometry-only mode. This is controlled by setting pcd_save_en: false in hesai.yaml, which prevents the system from accumulating the global point cloud in RAM. In this configuration FAST-LIO2 still maintains a local sliding map window around the drone's current position which is necessary for the IESKF residual computation described in Section 4.5.2 but does not retain historical map points beyond the local window. As demonstrated empirically in Section 6, this configuration reduces peak RAM consumption throughout the full test sequence to well within the Raspberry Pi 5's hardware limit.

Disabling in-flight map saving does not mean the global map is lost. Rather than accumulating the full point cloud in RAM during flight, the new system records the complete raw sensor data and output odometry to the disk as a ROS 2 bag file during the mission using `record_fastlio.py`. This bag file contains the full LiDAR point cloud and IMU data stream for the entire flight. After the flight lands and the live odometry is no longer needed the recorded output can be stitched together to produce a full-resolution point cloud that is saved as a `.pcd` file. The `bag_to_pcd.py` script in the analysis toolchain automates this conversion from recorded bag file to final point cloud. Rather than loading all point cloud frames into RAM simultaneously which would recreate the OOM problem on the base station for long missions it uses a chunked parallel processing strategy. Frames are accumulated into a buffer until either a configurable chunk size threshold is reached or RAM usage exceeds 80%.

This approach completely decouples global map construction from the real-time flight constraints. The drone maintains reliable real-time navigation throughout the mission using the memory-stable odometry-only configuration, while the computationally expensive task of assembling the global map is deferred to post-flight processing on hardware that can support it without risk of failure.

5.10 FAST-LIO2 Parameter Tuning

With the hardware stack integrated and the benchmarking toolchain in place, attention turned to tuning the FAST-LIO2 algorithm for stable real-time operation on the Raspberry Pi 5. The motivation for this work arose from a practical observation during initial testing, the generated map and APE was

inconsistent with the quality expected from the algorithm. Specifically there appeared to be unbounded drift while mapping was occurring, this phenomenon is shown graphically in Figure 5.8 below.

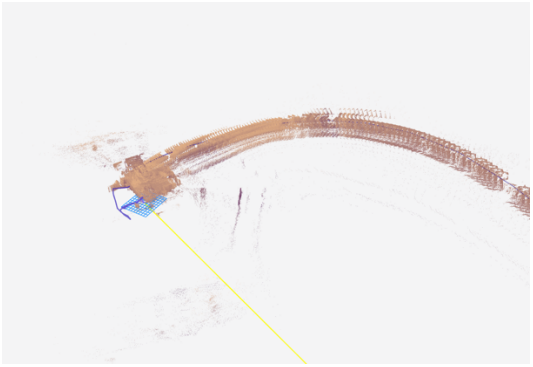

Figure 5.8: Graphical showcase of unbounded drift during mapping

Investigation using the benchmarking toolchain revealed that the root cause was not an algorithmic or sensor integration error, but a computational one. The system was dropping LiDAR frames because each scan was taking longer to process than the 100ms real-time budget allowed. When FAST-LIO2 cannot process a scan before the next one arrives, the missed scan breaks the continuity of the IESKF state propagation, introducing sudden positional jumps that manifest as the observed map distortion.

The two parameters identified as having the dominant impact on per-frame computational cost are `point_filter_num` (referred to throughout as the decimation stride k) and `filter_size_surf` (referred to as the voxel grid size v). These two parameters jointly control the density of the point cloud that enters the IESKF update step at each scan. `point_filter_num` subsamples the raw scan by retaining only every k-th point in the order they are received from the sensor, reducing the input point count by a factor of k with minimal preprocessing cost. `filter_size_surf` applies a voxel grid filter that partitions the scan into cubic cells of side length v metres and replaces all points within each cell with a single representative point, reducing density while preserving spatial distribution more uniformly than stride-based subsampling.

Rather than tuning k and v manually by trial and error, a systematic automated sweep was designed and executed using `run_parameter_sweep.py`. The script iterates over a grid of $k \in \{1, 2, 3, 4, 5\}$ and $v \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ m, yielding 25 configurations in total. During each run, `psutil` monitors the FAST-LIO2 process at 0.5-second intervals to record CPU utilisation and RAM consumption, while

FAST-LIO2's internal timing log is parsed after each run to extract per-frame latency figures. On completion of each configuration, the results of the average latency, maximum latency, peak RAM, average CPU, peak CPU, total frames processed, and the frame drop rate derived by comparison against the maximum possible frame count are appended to a master CSV. Once all 25 configurations have been evaluated, `plot_sweep_heatmaps.py` is invoked automatically to render the results as 2D heatmaps of latency and RAM usage as a function of k and v. The full results of this sweep and the identification of the optimal operating configurations are presented and discussed in Section 6.1.2.

5.11 Loop Closure Module Implementation

This section documents the implementation of the offline loop closure pipeline described methodologically in Section 4.6. As established by the RAM saturation analysis in Section 6.1.1, executing pose graph optimisation on the Raspberry Pi 5 during flight is infeasible global mapping alone saturated the platform of physical memory. The loop closure module is therefore implemented entirely as an offline post-processing pipeline that runs on the base station PC after the flight data has been transferred from the UAV, directly realising the cloud compute domain of the edge-cloud split architecture described in Section 4.3.

The offline pipeline is structured as three sequential stages that communicate with each other, as illustrated in Figure 5.9. This decoupled design allows any individual stage to be re-executed independently, for example, the pose graph optimisation can be re-run with different parameters without re-extracting keyframes from the bag.

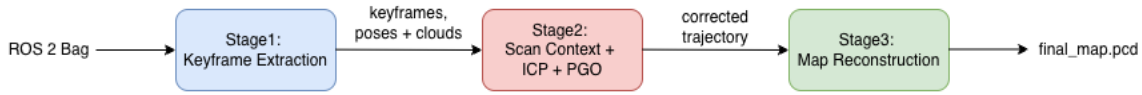

Figure 5.9: Loop closure module stages

Stage 1 Keyframe Extraction: The recorded ROS 2 bag containing the /Odometry and /cloud_registered topics from the flight and is read sequentially. Keyframes are sampled based on the distance-based selection criterion defined in Equation 4.10, and saved to disk as paired pose and point cloud files.

Stage 2 Pose Graph Optimisation: The saved keyframes are loaded, Scan Context descriptors are computed for each keyframe, loop closure candidates are identified and verified via ICP, and the resulting pose graph is optimised. The output is a corrected TUM-format trajectory file.

Stage 3 Map Reconstruction: Each keyframe's point cloud is transformed using its corrected pose and merged into a single globally consistent point cloud, saved as a .pcd file.

The entire pipeline is orchestrated by a single Python script that can be invoked with one command, taking the recorded bag as input and producing the corrected trajectory and map as output.

5.11.1 Bag-to-Keyframe Extraction

Keyframes are extracted directly from the recorded ROS 2 bag using the rosbags Python library. The script iterates through the bag reading two topics in parallel, /Odometry for the FAST-LIO2 pose estimates and /cloud_registered for the de-skewed and down sampled point clouds.

Not every frame is retained. Following the keyframe selection criterion defined in Equation 4.10, a new keyframe is saved only when the Euclidean distance between the current pose and the most recently saved keyframe exceeds a configurable threshold. This reduces a typical 3,000-frame bag to approximately 200–400 keyframes. This process can be visualised in Foxglove studio as shown below in Figure 5.10 below, where each red dot is a saved keyframe.

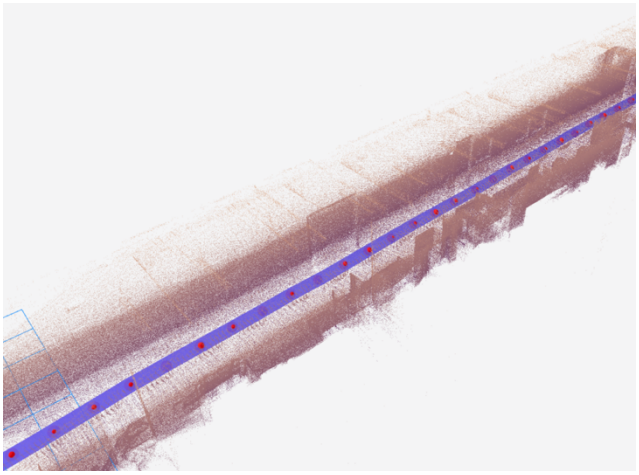

Figure 5.10: Keyframe selection (red spheres) visualised in Foxglove Studio

For each selected keyframe, two files are written to a numbered output directory. The pose is saved as a single-line text file in TUM format (timestamp tx ty tz qx qy qz qw), which allows direct use with the evo evaluation toolchain.

5.11.2 Scan Context Implementation

The Scan Context descriptor described in Section 4.6.2 is implemented as a standalone Python module following the port of the original formulation of Kim and Kim (2018). Each keyframe's point cloud is turned into a 20-ring × 60-sector polar grid covering a maximum radius of 30 m, with each cell storing the maximum height value of all points within it. An example collected from the UCD Lake Circuit dataset is shown below in Figure 5.11.

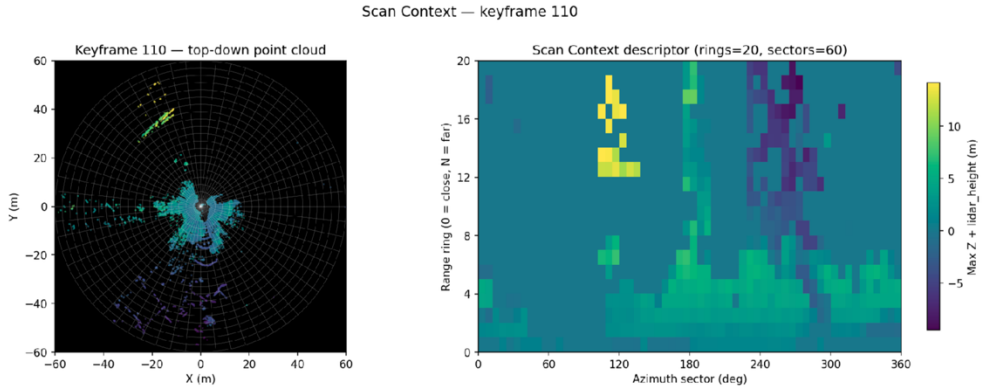

Figure 5.11: Scan Context keyframe 110 from UCD Lake Circuit Dataset

For retrieval, each descriptor's row-wise mean is stored as a key in a KD-tree. At query time, the 10 nearest keys are retrieved, and the full cosine distance with column-shift yaw alignment (Equation 4.12) is computed only for these candidates. Candidates are forwarded to ICP verification only if the cosine distance is below the threshold and the candidate is separated by at least 30 keyframes and 20 m of path length from the query.

5.11.3 ICP Verification & Pose Graph Optimisation

Each Scan Context candidate is verified using Open3D's Generalised ICP implementation (Zhou, Park and Koltun, 2018). Both clouds are voxel-down sampled to 0.3 m prior to registration. The initial guess is a pure Z-axis rotation derived from the Scan Context column shift, with zero translation. Scan Context identifies the approximate yaw offset and ICP refines the residual translation and rotation. A candidate is accepted if the inlier fitness score exceeds the selection threshold. Candidates failing the selection criterion are rejected as false positives.

The pose graph is constructed using Open3D's `PoseGraph` class. Each keyframe becomes a node initialised with its raw FAST-LIO2 world pose, and edges between nodes encode geometric constraints representing how far apart two keyframes should be relative to each other. Odometry edges connect consecutive keyframes using the relative transformation from FAST-LIO2. These edges are assigned a tight uncertainty, reflecting the high local accuracy of the IESKF estimator. Together they form the sequential chain that represents the drift-affected trajectory.

Loop closure edges connect non-consecutive keyframes that were identified as revisiting the same physical location by the Scan Context and ICP pipeline. These edges are assigned a looser uncertainty, as a single ICP alignment is inherently less trustworthy than FAST-LIO2's frame-to-frame tracking. These edges are flagged as `uncertain=True`, which enables Open3D's process to automatically reduce the influence of any loop edge that disagrees too strongly with its neighbours, preventing a single incorrect loop from corrupting the trajectory.

The optimiser then adjusts all keyframe poses simultaneously to satisfy both the odometry and loop closure constraints as well as possible. Where the two types of constraint disagree, for example the odometry chain says the trajectory ended 12.85 m from the start while a loop edge says it returned to the start, the solver distributes the correction smoothly along the entire trajectory rather than applying an abrupt jump at a single point. The first keyframe is held fixed as a spatial anchor so the graph has a stable reference frame. The graph is solved using the Levenberg-Marquardt algorithm and converges in approximately 1 second on the Lake Circuit dataset. The corrected poses are extracted and written as a TUM-format trajectory file. All of the parameters for loop closure throughout the whole process are controlled via the `pgo_params.yaml` configuration file that is shown in appendix A.5.

6. Experimental Setup and Results Evaluation

This chapter presents the experimental validation of that system across a structured sequence of tests, each targeting a distinct engineering aspect of the system.

6.1 Computational Performance on Embedded Hardware

This section empirically characterises the computational boundaries of the Raspberry Pi 5 as a real-time SLAM platform. Three experiments are presented in sequence. The first directly validates the necessity of the edge-cloud split architecture by demonstrating the RAM saturation behaviour of native global mapping. The second identifies the optimal algorithmic parameters through a systematic sweep of the voxel grid configuration. The third uses those optimal parameters to establish a stable computational baseline for the physical deployment experiments that follow.

6.1.1 Odometry-Only vs Global Mapping - RAM Saturation Analysis

The fundamental architectural decision described in Section 4.3, being to restrict the Raspberry Pi 5 to odometry-only operation and offload global map construction to the base station was motivated by an empirically observed memory saturation. To validate this decision, FAST-LIO2 was run under two configurations over the full 1,025-second NCLT benchmark sequence (Carlevaris-Bianco, Ushani and Eustice, 2016), first in its default global mapping mode, which accumulates the complete registered point cloud in RAM throughout the mission, and second in odometry-only mode, which maintains only a local sliding map window around the current position.

The memory allocation profiles recorded by `profile_fastlio.py` over the duration of the sequence are shown in Figure 6.1 below. In the global mapping configuration, RAM consumption grew linearly and without bound throughout the simulation, reaching a peak of 8.5 GB before triggering an Out-of-Memory kill by the Linux kernel, crashing the system entirely. This behaviour is a direct consequence of the ikd-Tree accumulating every registered point cloud frame into a persistent global map as the trajectory grows, so does the map, and on a platform with a limited amount of physical RAM this growth is unsustainable for missions of any significant duration. The failure of the odometry system will trigger the crash and loss of the UAV, thus this outcome must be avoided at all costs.

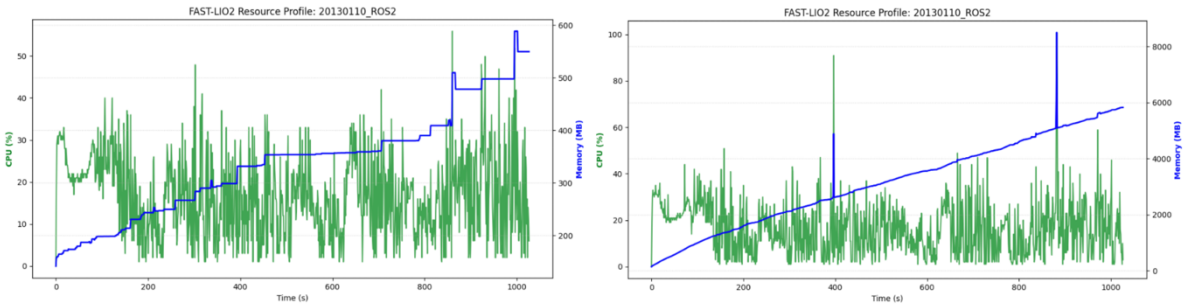

Figure 6.1: RAM and CPU usage for 2012-01-08 NCLT dataset. (left) Odometry-only mode (right) Mapping mode

In the odometry-only configuration, RAM consumption was significantly lower and climbed steadily but remained low for the entire 1,025-second sequence, with a peak allocation of 588 MB, a reduction of over 93% compared to the global mapping peak. CPU utilisation followed the same pattern, remaining consistently below the saturation threshold throughout the odometry-only run. These results empirically confirm that native global mapping is fundamentally incompatible with the Raspberry Pi 5 for extended missions, and that the edge-cloud split architecture described in Section 4.3 is a strict architectural requirement rather than an optional optimisation.

6.1.2 Spatial Resolution vs Compute Trade-off - Parameter Sweep

To characterise the computational boundaries of FAST-LIO2 on the Raspberry Pi 5 as a function of filtering parameters, a systematic parameter sweep was executed over a grid of decimation stride $k \in \{1, 2, 3, 4, 5\}$ and voxel grid size $v \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ m, yielding 25 configurations in total. The sweep was conducted on the UCD lake circuit dataset as it represents the most computationally demanding deployment scenario. The open outdoor environment produces long-range LiDAR returns that populate a much larger spatial volume in the ikd-Tree map than a confined indoor space, making it the worst-case test for both memory consumption and processing latency. The three resulting heatmaps: peak RAM usage, average processing latency, and frame drop rate are presented together in Figure 6.2.

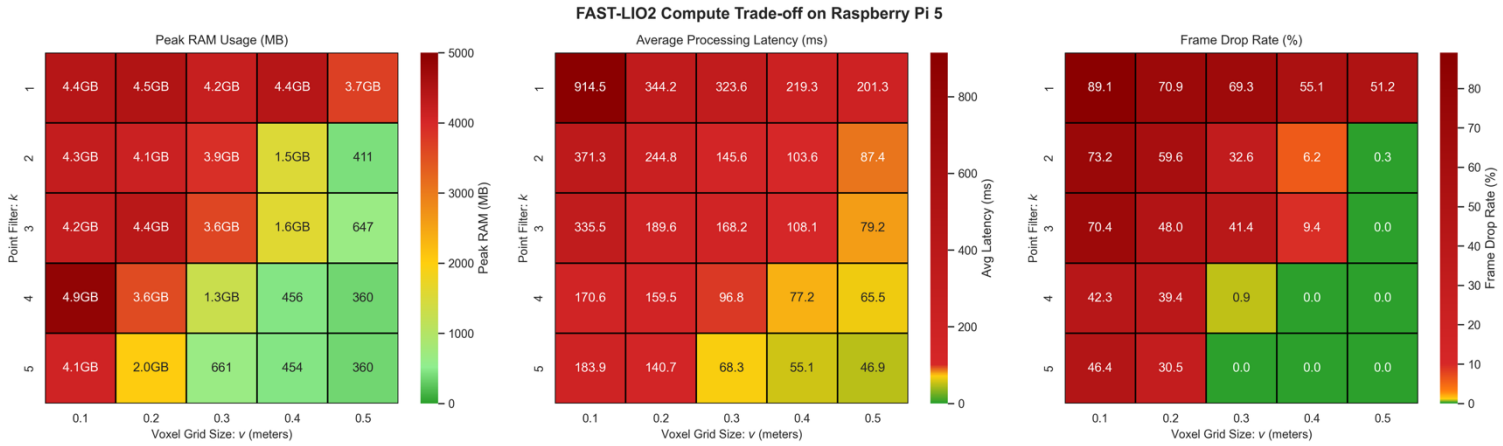

Figure 6.2: Heatmap plots of peak RAM usage, Average Processing Latency and Frame Drop Rate

The RAM heatmap reveals a strong and consistent relationship between voxel grid size and memory consumption. At small voxel sizes (v = 0.1 m), peak RAM usage reaches between 4.1 GB and 4.9 GB across all k values. This behaviour is a direct consequence of the ikd-Tree map structure. A finer voxel grid means more unique spatial cells are populated as long-range outdoor returns are inserted into the map. In an open environment where beams travel up to 120 m before returning, the map grows rapidly to cover a large spatial volume, and each new cell adds a node to the ikd-Tree that must be retained in RAM throughout the mission. As v increases the number of unique cells decreases because more points are merged into each voxel, keeping the tree compact and memory consumption low. High RAM consumption directly threatens system stability on the Raspberry Pi 5, when the ikd-Tree approaches the physical memory limit the Linux kernel begins swapping memory pages to the SD card. SD card access is orders of magnitude slower than RAM, introducing multi-millisecond delays into nearest-neighbour queries that corrupt the timing assumptions of the IESKF backward propagation and can cause catastrophic trajectory divergence.

The latency heatmap shows the expected general trend, latency decreases as both k and v increase, since more aggressive filtering reduces the number of points entering the IESKF residual computation and the number of nodes in the ikd-Tree that must be traversed per nearest-neighbour query. The worst-case configuration (k = 1, v = 0.1 m) reaches 914.5 ms over nine times the 100ms real-time threshold while the most aggressive configuration (k = 5, v = 0.5 m) achieves 46.9 ms, providing a comfortable 53ms safety margin. However the latency heatmap also contains several non-monotonic anomalies where increasing k does not reduce latency as expected, for example k = 3, v = 0.1 m shows higher latency than k = 2, v = 0.1 m in certain cells. These inversions are attributed to two

compounding effects being CPU thermal throttling and Linux OS scheduling behaviour under CPU overload when the system is heavily saturated the kernel scheduler may pre-empt the FAST-LIO2 process to service background tasks, introducing variable delays that distort the latency measurement.

The frame drop rate heatmap defines the boundary between configurations that maintain real-time continuity and those that do not. The heatmap shows that near-zero frame drop rates are achieved consistently at $v \geq 0.3$ m for $k \geq 4$. Configurations with $v \leq 0.2$ m produce severe frame drop rates of 30–89% regardless of k, rendering them unsuitable for real-time deployment on the embedded platform.

Considering all three constraints jointly the viable operating region on the Raspberry Pi 5 for outdoor deployment narrows to configurations satisfying the near zero frame drop and sub 100ms processing time. Within the viable region, multiple configurations achieve these constraints with stable memory consumption. However, computational metrics alone cannot identify which of these configurations produces the most accurate localisation. A configuration may be computationally viable yet produce poor trajectory accuracy if the remaining point density or map resolution is insufficient for reliable IESKF plane fitting. Determining the optimal configuration therefore requires empirical accuracy validation, which is conducted in Section 6.4 through closing error experiments on the physical UCD dataset.

6.2 Baseline Odometry Accuracy – NCLT Dataset

Having established a stable computational operating point in Section 6.1, the trajectory accuracy of the FAST-LIO2 odometry was evaluated against an external ground truth reference. The University of Michigan North Campus Long-Term (NCLT) Vision and LiDAR dataset (Carlevaris-Bianco, Ushani and Eustice, 2016) was selected for this evaluation because it provides high-precision RTK-GPS ground truth alongside synchronised Velodyne HDL-32E LiDAR data, making it one of the few publicly available datasets that enables rigorous quantitative APE analysis without requiring a physical motion capture system. The 2013-01-10 sequence was used, covering a 1,146.31 m trajectory over 1,025 seconds recorded on the University of Michigan campus.

The odometry trajectory was extracted from the FAST-LIO2 output using `bag_to_tum.py` and the ground truth was converted to TUM format using `nclt_to_tum.py` as described in Section 5.6. APE was then computed using the `evo_ape` tool from the evo evaluation package (Grupp, 2017), with SE(3) Umeyama alignment applied to remove the initial coordinate frame offset between the two trajectories before error computation. The results are shown in Table 6.1 and Figure 6.3 below.

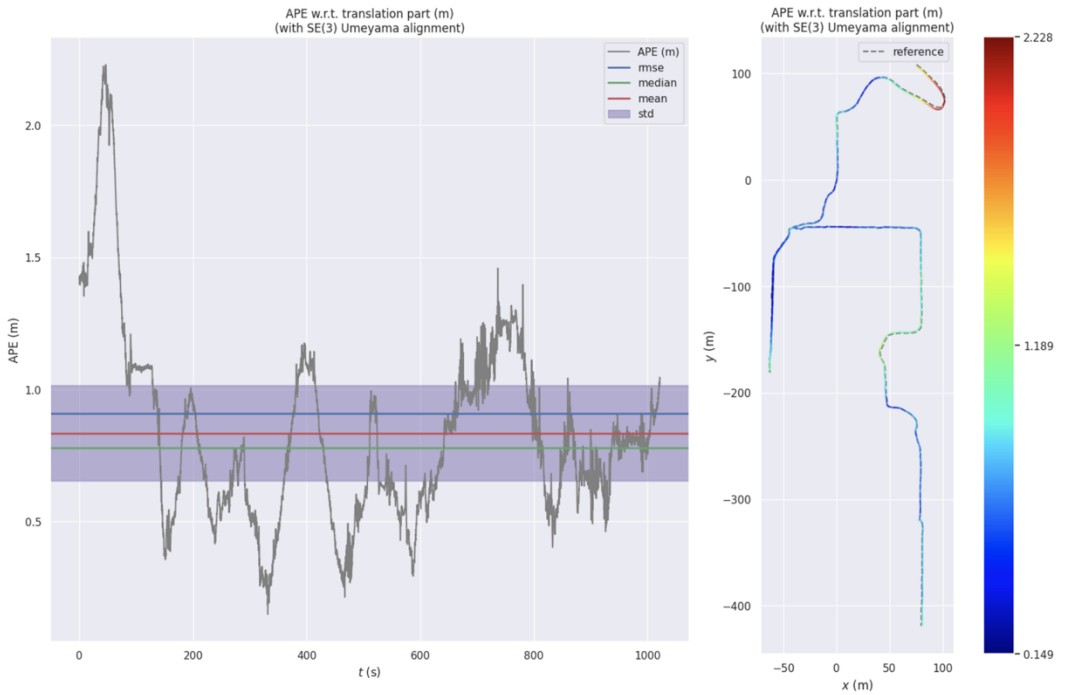

Figure 6.3: APE analysis of NCLT sequence 2013-01-10

Metric	Value (m)
RMSE	0.908
Mean	0.834
Median	0.778
Maximum	2.228
Minimum	0.149
Std. Deviation	0.361

Table 6.1: APE Analysis of NCLT sequence 2013-01-10 results summary

The results demonstrate a high level of odometry accuracy for a resource-constrained embedded platform operating in real time. The RMSE of 0.908 m over a 1,146.31 m trajectory corresponds to a relative drift of approximately 0.079% of the total distance travelled, meaning the system accumulates less than one metre of error per kilometre of traversal. The mean APE of 0.834 m and the low standard deviation of 0.361 m together indicate that the error is not characterised by sudden large jumps but rather by a gradual, consistent accumulation.

Critically, this level of accuracy is an excellent result for an odometry mapping system. However the residual drift of under 2.228 m at peak represents the bounded error that the pose-graph optimisation module, described in Section 4.6.3, is designed to correct retrospectively using the loop closure constraints generated by the Scan Context module during the mission. It is important to note that the APE is not critical for live obstacle avoidance and control of the drone, it is in fact a mapping quality metric. The live control and obstacle avoidance of the drone can still be implemented via live LiDAR DataStream.

6.3 Hardware Deployment & UCD Dataset Collection

In this section we will cover the process of the physical hardware deployment as well as the collection of the UCD LiDAR dataset which is another key contribution of this project.

All three physical deployments used the assembled sensor payload described in Section 5.4, carried by hand using the detachable grip handle mounted to the underside of the platform deck. The Raspberry Pi 5 was powered by its dedicated USB-C power bank and brought up the full SLAM stack automatically as described in Section 5.8. The mapping operation had two team members one carrying the mapping device and one monitoring the live mapping process from a laptop connected to the Pi 5's 5 GHz Wi-Fi hotspot, viewing the real-time 3D point cloud, odometry trajectory, and IMU data streams through the Foxglove Studio interface described in Section 5.7. Bag recording was initiated and terminated remotely using the Foxglove recording control panel. This workflow validated the plug-and-play operational model of the system under conditions representative of a real field mission, where direct access to the embedded computer is not available once the platform is deployed.

6.3.1 Dataset Descriptions

The three LiDAR-inertial datasets collected as part of this work are described in Table 6.2 below.

Property	Basement Corridor	Engineering Building Exterior	Lake Circuit
Date Recorded	11/04/2026	11/04/2026	11/04/2026
Duration (s)	175.68 s	343.57 s	371.51 s
Trajectory Length (m)	211.02 m	437.03 m	503.63 m
Bag File Size (GB)	2.7 GB	5.4 GB	5.8 GB
LiDAR Frames	1757	3435	3716

Lighting Conditions	Minimal/None	Daylight	Daylight
Closed Loop	Yes	Yes	Yes

Table 6.2: UCD LiDAR dataset summary

Basement Corridor: GPS-Denied, Low Light Environment

Recorded in the basement of the UCD Engineering building under near-darkness conditions with no GPS signal, directly replicating the target subterranean deployment environment. The corridor geometry is largely featureless, presenting the geometric degeneracy challenge discussed in Section 2.3. A closed loop was walked to enable closing error measurement.

Engineering Building Exterior: Structured Outdoor Environment

Recorded outdoors around the perimeter of the UCD Engineering building in daylight. Features a structured architectural geometry. A closed loop was walked around the building perimeter.

Lake Circuit: Open Outdoor

Recorded around the engineering lake on the UCD campus with an open environment surrounded by vegetation such as trees and bushes with building in the background.

An overview of the path taken for each dataset is shown below in Figure 6.4.

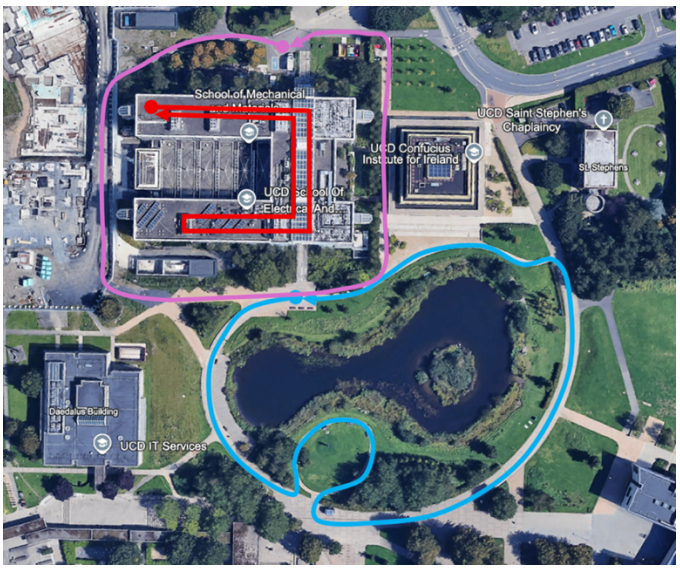

Figure 6.4: Overview of Dataset Path. Blue: Lake Circuit, Purple: Engineering Building Exterior, Red: Basement Corridor

The dataset is publicly available via google drive here.

6.4 UCD Dataset Analysis

This section presents the accuracy and mapping quality evaluation of the complete physical system across the three UCD datasets introduced in Section 6.3.

The parameter sweep conducted in Section 6.1.2 characterised the computational trade-off between voxel grid size and decimation stride across 25 configurations, but computational viability alone is insufficient to identify an optimal operating point as a configuration may sustain real-time throughput while producing insufficient point density for reliable IESKF plane fitting, resulting in poor localisation accuracy. To resolve this, the full parameter sweep was re-executed on the UCD Lake Circuit dataset with closing error as the primary evaluation metric. The Lake Circuit was selected for this accuracy sweep for the same reason it was used for the computational sweep in Section 6.1.2: as the most spatially demanding environment, it represents the worst-case test for both memory and localisation accuracy, making it the most viable basis for configuration selection. The resulting closing error heatmap is presented in Figure 6.5 below.

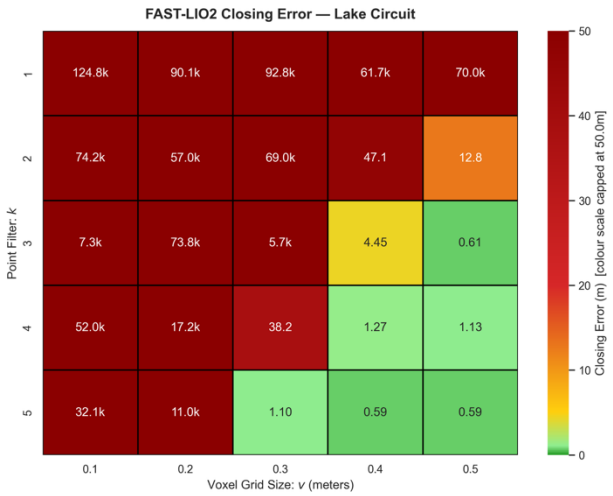

Figure 6.5: FAST-LIO2 Closing Error Heatmap for Lake Circuit

The heatmap reveals a clear and consistent pattern showing that closing error decreases sharply as both v and k increase toward the viable operating region, with the lowest errors concentrated in the bottom-right quadrant of the parameter space. Configurations with $v \leq 0.2$ m produce closing errors in the tens of kilometres regardless of k, indicating complete divergence as the point density under these settings exceeds the real-time processing budget established in Section 6.1.2, causing frame drops that break state propagation continuity and cause the estimator to accumulate unbounded drift. The

transition to viable performance is abrupt: at v = 0.3 m the closing error drops to single-digit metres for higher k values.

Within the viable region, k = 5, v = 0.4 m achieves the minimum closing error of 0.59 m, matching the result at v = 0.5 m but with a finer voxel resolution that preserves greater geometric detail in the reconstructed map. This configuration is therefore selected as the single operating configuration for all three UCD physical deployments. Its selection is further supported by the RAM and latency profiles from Section 6.1.2, which confirmed stable memory consumption and a per-frame latency.

6.4.1 Dataset Mapping Results

With a configuration of k = 5, v = 0.4 m being chosen each of the three UCD datasets was processed through the odometry pipeline using the selected configuration. The resulting point cloud maps are presented alongside satellite imagery in Figures 6.6–6.8 below.

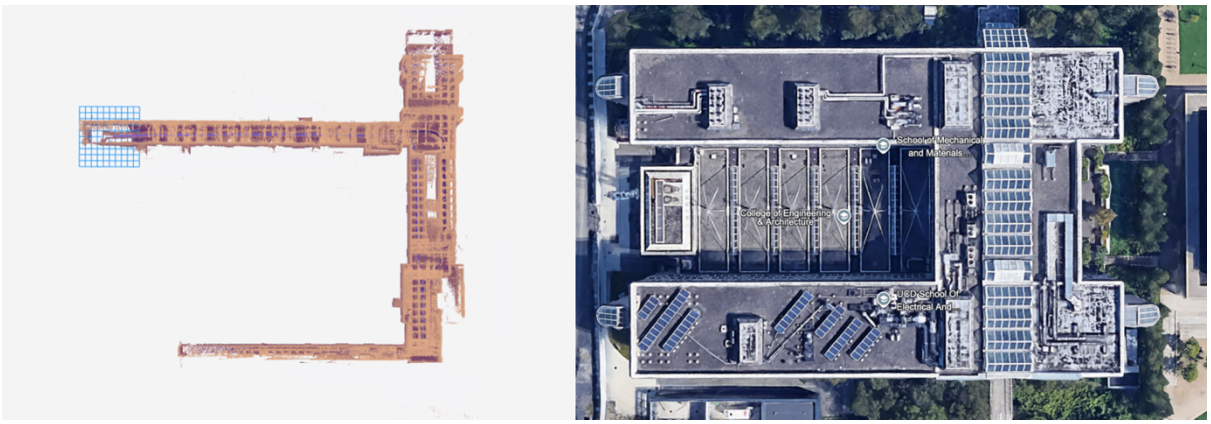

Figure 6.6: Basement - LiDAR map (left) vs satellite imagery (right)

The basement corridor map reconstructs the closed-loop trajectory under near-darkness and GPS-denied conditions successfully. Despite the featureless geometry of the corridor walls, the system maintained coherent state estimation throughout the run, with no visible map divergence or discontinuity.

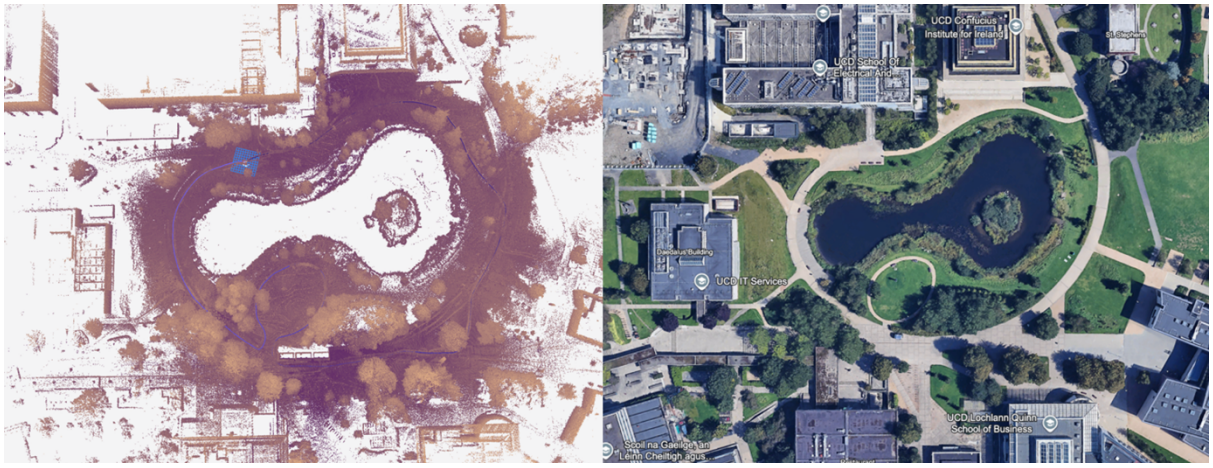

Figure 6.7: Lake Circuit - LiDAR map (left) vs satellite imagery (right)

The Lake Circuit map captures the full circuit path around the lake, with surrounding vegetation and building facades visible in the point cloud. The open outdoor environment produces the largest spatial coverage of the three datasets, consistent with the long-range returns characteristic of the Hesai XT-16 in unobstructed conditions.

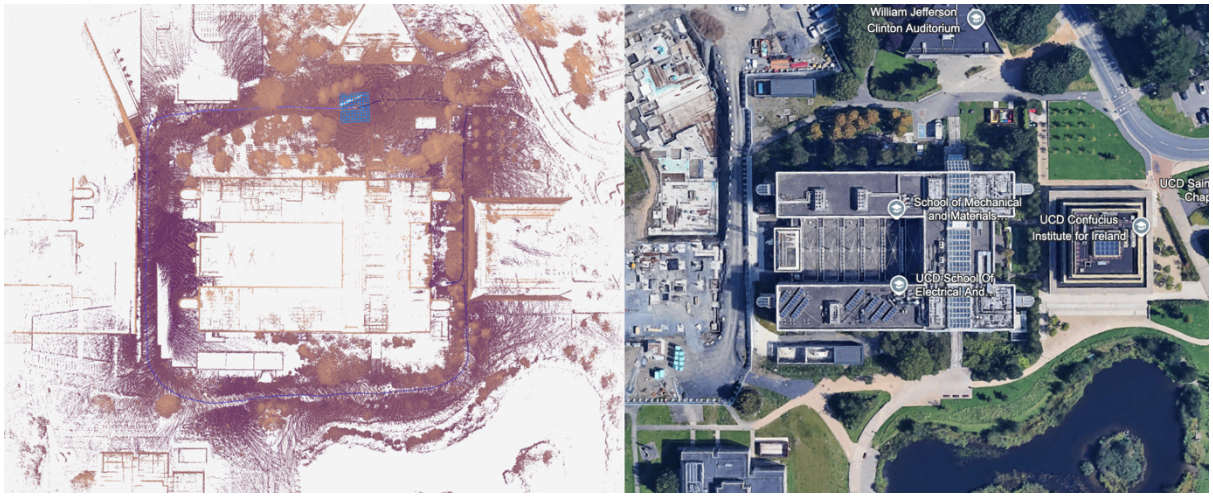

Figure 6.8: Engineering Building Exterior - LiDAR map (left) vs satellite imagery (right)

The Engineering Building Exterior map successfully reconstructs the full U-shaped perimeter of the building. While also partially capturing the lake and the surrounding buildings.

The quantitative accuracy and computational performance results for all three runs are summarised in Table 6.3 below.

Environment	Closing Error (m)	Drift (% trajectory)	Avg Latency (ms)	Peak RAM (MB)	Frame Drop Rate (%)
Basement Corridor	0.5287 m	0.2505 %	19.54 ms	177.91 MB	< 1%
Lake Circuit	0.5887 m	0.1169 %	77.42 ms	453.59 MB	< 1%
Engineering Exterior	0.0764 m	0.0175 %	70.73 ms	497.54 MB	<1%

Table 6.3: UCD dataset results summary

The results of per frame processing latency, RAM consumption and CPU utilisation are shown in relation to time in Figure 6.9 below.

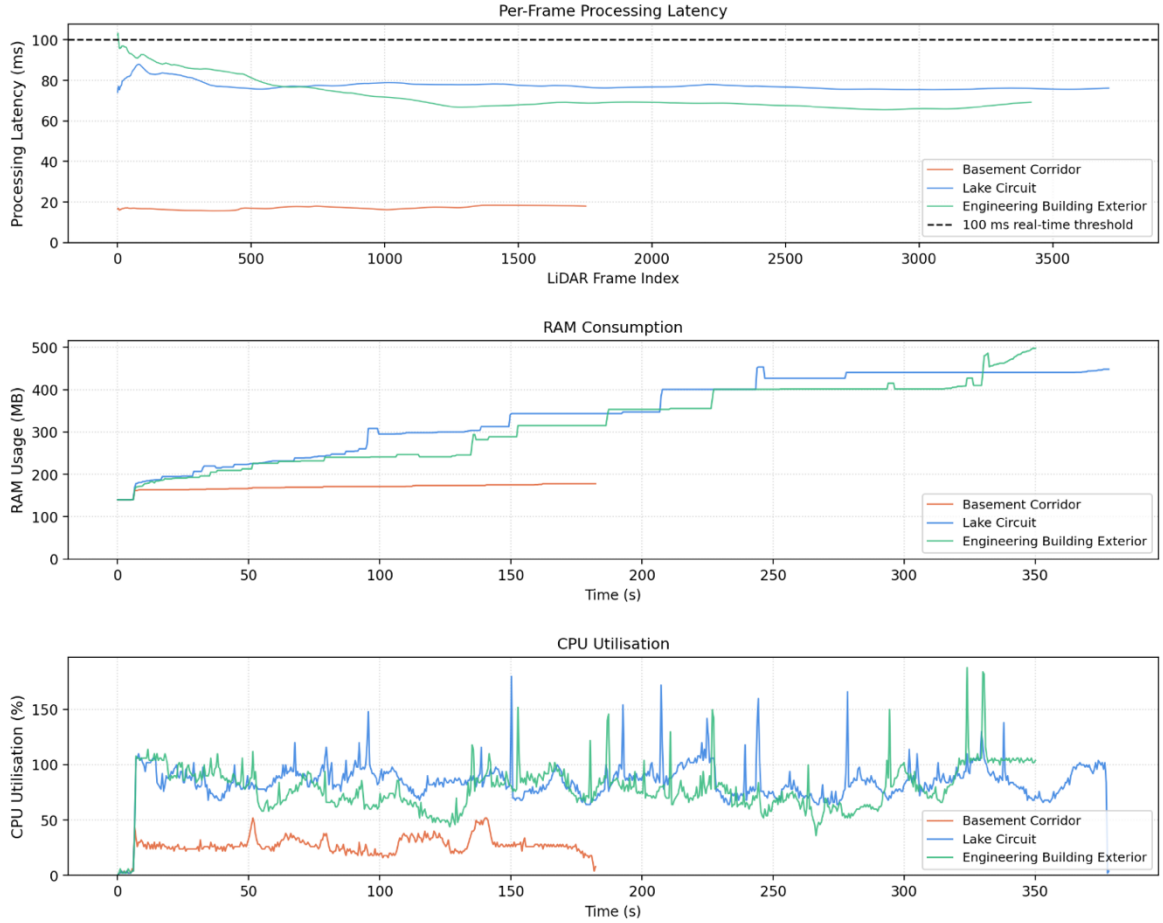

Figure 6.9: UCD dataset processing latency, RAM consumption and CPU utilisation.

Across all three deployment environments the system demonstrated consistent real-time performance and accurate trajectory estimation under the selected configuration. Every environment achieved a frame drop rate below 1%, confirming that the k = 5, v = 0.4 m configuration maintains the 100 ms real-time processing budget across both confined indoor and open outdoor conditions. Peak RAM consumption remained well below the Raspberry Pi 5's physical limit in all three cases, with the most

demanding outdoor datasets peaking at 454 MB and 498 MB respectively. This confirms that the odometry-only edge architecture described in Section 4.3 eliminates the RAM saturation behaviour observed in global mapping mode entirely, along with also validating that selecting k = 5, v = 0.4 m also reduced RAM consumption without sacrificing the odometry accuracy.

The closing error results validate the localisation accuracy of the system across geometrically diverse environments. The Engineering Building Exterior achieved the lowest closing error of 0.08 m over its full perimeter trajectory. The Lake Circuit and Basement Corridor both achieved sub-metre closing errors of 0.59 m and 0.53 m respectively, corresponding to drift percentages of 0.12% and 0.25% of total trajectory length. Notably, the Basement Corridor result is the most significant of the three, as it was obtained under near-darkness and GPS-denied conditions with a geometrically featureless corridor environment the primary mission representative scenario of this thesis. The sub-metre closing error in this environment directly validates the system's suitability for subterranean deployment.

6.5 Pose Graph Optimisation & Postprocessing

Following each physical deployment, the recorded ROS 2 bag files were transferred to the base station and processed through the offline loop closure pipeline described in Section 5.11. The pipeline extracts keyframes from the recorded trajectory, computes Scan Context descriptors, identifies loop closure candidates via ICP geometric verification, and applies pose graph optimisation to retrospectively distribute accumulated drift across the trajectory.

6.5.1 Pose Graph Optimisation

Results Under the Optimal Configuration

For all three datasets processed under the selected configuration of k = 5, v = 0.4 m, the raw FAST-LIO2 odometry already achieved sub-metre closing errors as reported in Section 6.4.1. In these cases the offline PGO stage produced no measurable improvement to trajectory accuracy. This outcome is expected and is in fact a validation of the parameter selection rather than a failure of the loop closure module. The practical implication is that for well-configured deployments the system produces accurate

globally consistent maps from odometry alone, with the loop closure module providing a safety net that is not invoked.

Results Under Degraded Configurations

To validate that the loop closure module functions correctly and produces meaningful corrections when odometry drift is significant, both datasets were additionally processed under deliberately degraded parameter configurations that produce higher raw closing errors. The results are summarised in Table 6.4 below.

Environment	Config	Raw Closing Error	After PGO	Reduction	Loop Closures
Lake Circuit	v = 0.5, k = 2	12.85 m	2.10 m	83.6%	5
Basement Corridor	v = 0.1, k = 4	81.95 m	7.60 m	90.7%	110

Table 6.4: Loop Closure Results for Degraded Configurations

A visual representation of the before and after of the Lake Circuit pose graph optimisation is shown in Figure 6.10 below.

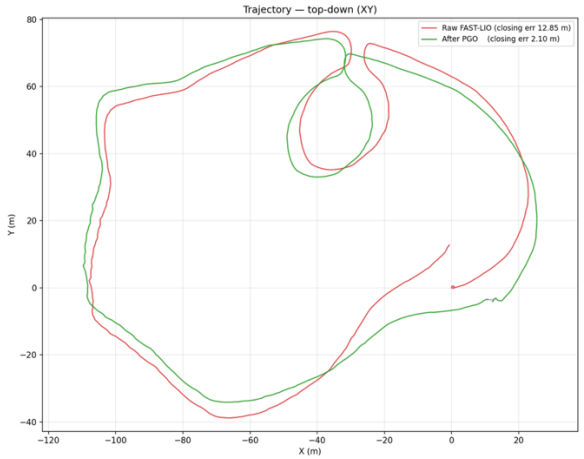

Figure 6.10: Top down view of the Lake Circuit odometry before and after pose graph optimisation

The Lake Circuit trajectory plot illustrates the effect of pose graph optimisation clearly, with the corrected green trajectory visibly tighter and more geometrically consistent than the drifted red odometry path. The 83.6% and 90.7% closing error reductions across the two degraded configurations confirm that the Scan Context and ICP pipeline is capable of detecting valid loop closures and that the pose graph optimiser distributes drift corrections across the trajectory as intended. However, the practical utility of the module is limited in the context of this system. As demonstrated in Section 6.4.1, the correctly configured odometry pipeline already achieves sub-metre closing errors across all three

deployment environments without any post-processing correction. The loop closure module only produces meaningful improvements when the underlying odometry has diverged significantly, which in this system is a consequence of poor parameter selection rather than a fundamental limitation of the estimator. Under the optimal configuration, pose graph optimisation therefore provides no measurable benefit, and the accuracy requirements of the system are fully met by the odometry pipeline alone.

6.5.2 Post-Flight High-Resolution Map Reconstruction

While the k = 5, v = 0.4 m configuration was selected to satisfy the real-time processing constraints of the Raspberry Pi 5, the coarse voxel size necessarily sacrifices geometric detail in the reconstructed map. The odometry trajectory produced under this configuration is accurate, as demonstrated by the sub-metre closing errors in Section 6.4.1, but the map itself retains only the spatial structure required for state estimation rather than the full resolution of the raw sensor data.

A key advantage of the edge-cloud architecture described in Section 4.3 is that the complete raw LiDAR data is preserved in the recorded ROS 2 bag file throughout the mission. At the base station, the bag can therefore be reprocessed offline with a significantly finer voxel configuration in this case v = 0.1 m using the accurate odometry trajectory from the flight as the pose reference. This decouples map resolution entirely from the real-time compute budget of the embedded platform. Figure 6.11 illustrates the resulting high-resolution point cloud reconstructions for all three UCD environments.

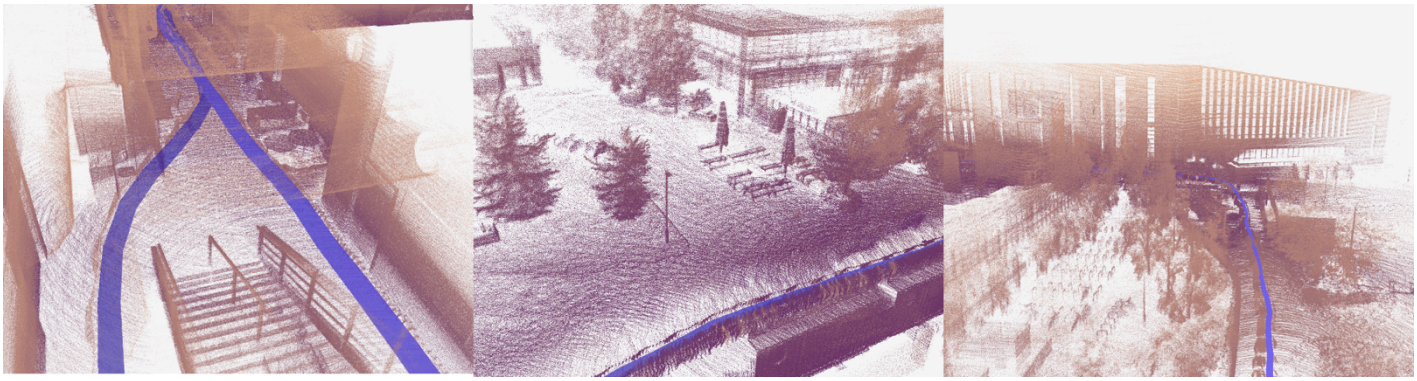

Figure 6.11: High Definition scan visuals of UCD dataset. Left: Basement Corridor, Middle: Lake Circuit, Right: Engineering Building Exterior

We can see from Figure 6.11 above that the resulting mapping quality is very high with fine details visible in every scan. In the basement corridor dataset we can clearly see fine indoor details such as stair railings, tables and couches. In the lake dataset we can clearly see outdoor vegetation such as trees

along with human structures such as benches and umbrellas. In the engineering building exterior we can clearly see in high detail building such as the UCD O'Connor Centre for Learning.

7. Discussion and Conclusion

This thesis set out to design, implement, and validate a real-time LiDAR-inertial SLAM system capable of operating entirely on a resource-constrained embedded platform in a GPS-denied subterranean environment. All five engineering objectives defined in Section 1.2 have been met.

The central architectural contribution of this work is the edge-cloud split-compute framework. Empirical profiling demonstrated conclusively that executing FAST-LIO2 in global mapping mode causes RAM consumption to grow linearly and without bound on the Raspberry Pi 5, reaching 8.5 GB for a short mission. Restricting the edge device to odometry-only operation reduced peak memory consumption by over 93% to 588 MB, establishing that the split architecture is a strict engineering requirement rather than an optional design choice.

The FAST-LIO2 framework was successfully ported from ROS 1 to ROS 2 Humble, containerised in Docker for cross-platform reproducibility, and extended with a custom Hesai XT-16 driver integration that required resolving fundamental point cloud format incompatibilities. Hardware-level PPS synchronisation was implemented and verified via oscilloscope, locking both sensors to a shared 1 Hz reference and satisfying the fundamental timestamp alignment assumption of the de-skewing pipeline.

A systematic 25-configuration parameter sweep identified k = 5, v = 0.4 m as the optimal operating point on the Raspberry Pi 5, balancing real-time throughput with localisation accuracy. Under this configuration, physical deployment across three geometrically diverse UCD environments produced consistent results with all three runs achieving frame drop rates below 1%, peak RAM usage below 500 MB, and sub-metre closing errors. The Basement Corridor result with a 0.53 m closing error under near-darkness and GPS-denied conditions in a featureless corridor is the most significant of the three results, directly validating the system's suitability for the target subterranean deployment scenario.

The offline loop closure pipeline, implementing Scan Context descriptor generation, ICP geometric verification, and Open3D pose graph optimisation, was validated under deliberately degraded parameter configurations, achieving closing error reductions of 83.6% and 90.7% on the Lake Circuit and

Basement Corridor datasets respectively. Under the optimal configuration, however, the odometry pipeline alone achieves sufficient accuracy that loop closure correction produces no measurable improvement, demonstrating that correct parameter selection is a more effective accuracy strategy than pose graph optimisation post flight.

The post-flight high-resolution map reconstruction pipeline, enabled by the edge-cloud architecture's preservation of the full raw sensor bag, produced detailed 3D reconstructions at v = 0.1 m that clearly resolved fine indoor features including stair railings and furniture, as well as outdoor structures and vegetation, entirely decoupled from the real-time compute budget of the embedded platform.

In summary, this work demonstrates that a €200 single-board computer is a viable platform for real time LiDAR-inertial odometry in GPS-denied environments, provided the system architecture carefully separates the memory-bounded survival odometry task from the computationally unbounded global mapping task. The resulting system achieved sub-metre drift across all tested environments, maintained a stable telemetry link to a cross-platform operator interface, and booted to a fully operational SLAM pipeline within 15 seconds of power-on meeting all performance indicators defined in Section 4.7.

8. Impact of Research

Societal and Humanitarian Impact:

The most immediate impact of this research is in search and rescue operations. As demonstrated by the 2018 Tham Luang cave rescue, where teams were forced to rely on incomplete decades-old 2D schematics (GIM International, 2018), the absence of accurate 3D maps of subterranean environments can directly cost human lives. The system developed in this thesis provides first responders with a lightweight, deployable tool capable of generating accurate maps of hazardous underground spaces such as collapsed mines, earthquake-damaged structures, and natural cave networks without requiring a human to enter the environment. By demonstrating that a constrained hardware platform such as a Raspberry Pi 5 can sustain real-time LiDAR-inertial odometry with sub-metre closing error in darkness and GPS-denied conditions, this work lowers the cost barrier to deploying autonomous mapping technology in emergency scenarios where it is most needed.

Beyond rescue operations, the system has direct applicability to geological and archaeological surveying of subterranean sites where manual access is impractical or destructive, supporting scientific discovery without physical risk to researchers.

Technological and Economic Impact:

A significant contribution of this work is the demonstration that state-of-the-art LiDAR-inertial SLAM previously dependent on high-wattage, high-cost computing platforms such as the 16-thread AMD 4800U, 64 GB RAM payloads used in the DARPA SubT CERBERUS system (Tranzatto et al., 2022) can be made viable on commodity embedded hardware through careful architectural partitioning. The openly published codebase, Docker environment, sensor drivers, and UCD LiDAR dataset provide a reproducible foundation that other researchers and small robotics companies can build upon without requiring access to expensive laboratory infrastructure. This lowers the barrier to entry for embedded SLAM development and may accelerate commercial deployment of autonomous mapping UAVs in industries including mining, construction inspection, and infrastructure surveying.

Ethical Considerations:

The ability of a UAV to navigate and map GPS-denied environments has potential dual-use implications, particularly in the context of military applications such as reconnaissance or drone navigation. However, the scope and design intent of this thesis are strictly humanitarian, targeting search and rescue and geological surveying.

Data privacy presents a secondary consideration. The system generates high-fidelity 3D maps of its operating environment using the Hesai XT-16's 0.5 cm range precision. In any deployment beyond the controlled UCD campus environments used in this thesis, operators would need to ensure that mapping activities comply with applicable privacy legislation and laws. However the nature of LiDAR is favourable for this as it is not capable of capturing facial details and thus provides anonymity for humans.

Sustainable Development Goals:

This research aligns with several UN Sustainable Development Goals. Most directly, it contributes to SDG 3 (Good Health and Well-Being) by enabling safer search and rescue missions that reduce the risk to human life in hazardous environments, addressing the operational gap highlighted by the Tham Luang rescue (GIM International, 2018). It also supports SDG 9 (Industry, Innovation and Infrastructure) through the development of a novel edge-cloud split-compute SLAM architecture that advances the state of the art in resource-constrained robotics.

From a green computing perspective, the core technical contribution of this thesis porting a high-performance SLAM pipeline built on FAST-LIO2 (Xu et al., 2022) to a 12 W Raspberry Pi 5 directly reduces the energy footprint of autonomous mapping relative to the multi-processor, high-wattage payloads used in competing state-of-the-art systems. Lower power consumption translates directly to longer UAV flight endurance, reducing the number of missions required to map a given area and therefore reducing the overall energy cost of a subterranean survey. This aligns with SDG 7 (Affordable and Clean Energy) by demonstrating that real-time SLAM performance and energy efficiency are not mutually exclusive in autonomous systems design.

9. Suggestions for Future Work

UAV Integration:

The most immediate and impactful extension of this work is the integration of the sensor payload with a physical UAV airframe. As noted in Section 4.1, the UAV platform was explicitly out of scope for this thesis due to hardware availability constraints, and all physical deployments were conducted by carrying the payload by hand. While this validated the SLAM pipeline under representative motion conditions, it does not replicate the full dynamics of aerial flight. Real UAV operation introduces additional challenges such as vibration transmitted through the airframe into the IMU, and the strict weight and balance constraints of a quadcopter platform. The sensor payload was designed with UAV mounting in mind via the detachable handle meaning the mechanical integration path is already established. Validating the complete system under live aerial flight conditions in a GPS-denied environment, represents the natural and necessary next step toward operational deployment.

Autonomous Path Planning and Navigation

The system developed in this thesis relies on a 5 GHz Wi-Fi telemetry link to stream live odometry and point cloud data to the base station operator. While this architecture is sufficient for controlled deployments, it presents a fundamental limitation in deep subterranean environments. Radio frequency signals attenuate rapidly in cave and tunnel geometries meaning the Wi-Fi link would become unreliable or completely unavailable beyond a relatively short distance from the cave entrance. This renders remote human teleoperation impractical for the target deployment scenario. A necessary future extension is therefore the integration of an onboard autonomous path planning and obstacle avoidance module that allows the UAV to execute a mapping mission entirely independently of any base station connection. The real-time odometry and point cloud produced by FAST-LIO2 provide exactly the state estimate and environmental representation required to drive such a planner. The system's decentralised architecture, which already operates the full navigation pipeline independently of the base station, provides a natural foundation for this extension.

True Subterranean Deployment

Although the Basement Corridor dataset was collected under near-darkness and GPS-denied conditions, replicating several characteristics of the target environment, it remains a university building corridor rather than a genuine cave or mining tunnel. Real subterranean environments present significantly more challenging geometry, including highly irregular surfaces, long featureless passages and airborne dust. Validating the system in an actual cave or mine would constitute the definitive test of the architecture's suitability for its intended application, and would expose failure modes that controlled campus testing cannot replicate.

Thermal Sensor Integration for Search and Rescue

The current system produces geometrically accurate 3D maps but carries no capability to detect the presence of survivors within a mapped environment. For search and rescue applications, the addition of a thermal infrared camera to the sensor payload would provide a complementary sensing modality that could identify human heat signatures within the generated point cloud map. Thermal cameras are passive sensors that operate without any illumination requirement, making them well suited to the absolute darkness of subterranean environments. Fusing thermal detections with the FAST-LIO2 odometry output would allow detected heat signatures to be localised within the 3D map and transmitted to the base station, providing rescue teams with not only a map of the environment but actionable information on the locations of potential survivors.

References

Abaspur Kazerouni, I. et al. (2022) 'A survey of state-of-the-art on visual SLAM', *Expert Systems with Applications*, 205, p. 117734.

Besl, P.J. and McKay, N.D. (1992) 'A method for registration of 3-D shapes', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2), pp. 239–256.

Brazzel, J., Clark, F. and Milenkovic, Z. (2015) 'LIDAR-based relative navigation with respect to non-cooperative objects', *Acta Astronautica*, 116, pp. 34–45.

Carlevaris-Bianco, N., Ushani, A.K. and Eustice, R.M. (2016) 'University of Michigan North Campus long-term vision and lidar dataset', *The International Journal of Robotics Research*, 35(9), pp. 1023–1035.

Dellaert, F. (2012) *Factor graphs and GTSAM: A hands-on introduction*. Technical Report GT-RIM-CP&R-2012-002. Georgia Institute of Technology.

Durrant-Whyte, H. and Bailey, T. (2006) 'Simultaneous localisation and mapping (SLAM): Part I', *IEEE Robotics & Automation Magazine*, 13(2), pp. 99–110.

Foxglove Studio (2026) *Foxglove Studio: robotics observability platform*. Available at: https://foxglove.dev (Accessed: 19 April 2026).

GIM International (2018) 'The Tham Luang Cave Rescue'. Available at: https://www.gim-international.com/ (Accessed: 19 April 2026).

Grasso, N. et al. (2023) 'Automated 3D cave mapping via LiDAR-based SLAM', *Remote Sensing*, 15(4), p. 912.

Grupp, M. (2017) *evo: Python package for the evaluation of odometry and SLAM*. Available at: https://github.com/MichaelGrupp/evo

Hesai (2026) *Pandar XT-16 user manual*. Hesai Technology Co., Ltd.

Kim, G. and Kim, A. (2018) 'Scan Context: Egocentric Spatial Descriptor for Place Recognition within 3D Point Cloud Map', in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Madrid, Spain: IEEE, pp. 4802–4809.

Liao, Z. et al. (2024) 'A Real-time Degeneracy Sensing and Compensation Method for Enhanced LiDAR SLAM', *arXiv preprint arXiv:2412.07513*.

Livox Technology (2021) *Livox LiDAR motion distortion correction*. Technical white paper. Livox Technology Co., Ltd.

Macenski, S. et al. (2022) 'Robot operating system 2: Design, architecture, and uses in the wild', *Science Robotics*, 7(66), p. eabm6074.

Papachristos, C., Khattak, S. and Alexis, K. (2017) 'Uncertainty-aware receding horizon exploration and mapping using aerial robots', in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. Singapore: IEEE, pp. 4568–4575.

Raspberry Pi Ltd. (2023) *Raspberry Pi 5 product brief*. Available at: https://www.raspberrypi.com (Accessed: 19 April 2026).

Rusu, R.B. and Cousins, S. (2011) '3D is here: Point Cloud Library (PCL)', in *2011 IEEE International Conference on Robotics and Automation (ICRA)*. Shanghai: IEEE, pp. 1–4.

SBG Systems (2024) *Ellipse series user manual*. SBG Systems SAS.

Shan, T. and Englot, B. (2018) 'LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain', in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Madrid, Spain: IEEE, pp. 4758–4765.

Shan, T. et al. (2020) 'LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping', in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Las Vegas, NV, USA: IEEE, pp. 5135–5142.

Tranzatto, M. et al. (2022) 'CERBERUS: Autonomous legged and aerial robotic exploration in the tunnel and urban circuits of the DARPA subterranean challenge', *Field Robotics*, 2(1), pp. 274–324.

Uy, M.A. and Lee, G.H. (2018) 'PointNetVLAD: Deep Point Cloud Retrieval for Large-Scale Place Recognition', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4470–4479.

Weishampel, J. et al. (2011) 'Detection of ancient Maya architectural remains using LiDAR', *Remote Sensing*, 3(7), pp. 1587–1600.

Wu, H. et al. (2022) 'Research on motion distortion optimization algorithm of laser SLAM', *Journal of Physics: Conference Series*, 2330, p. 012013.

Wulder, M.A. et al. (2012) 'Lidar sampling for large-area forest characterization: A review', *Remote Sensing of Environment*, 121, pp. 196–209.

Xu, W. and Zhang, F. (2021) 'FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated extended Kalman filter', *IEEE Robotics and Automation Letters*, 6(2), pp. 3317–3324.

Xu, W. et al. (2022) 'FAST-LIO2: Fast Direct LiDAR-Inertial Odometry', *IEEE Transactions on Robotics*, 38(4), pp. 2053–2073.

Zhang, J. and Singh, S. (2014) 'LOAM: Lidar Odometry and Mapping in Real-time', in *Robotics: Science and Systems*. Berkeley, CA.

Zhang, T. et al. (2017) 'Real-time autonomous exploration of underground mine environments using MAVs', in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. Singapore: IEEE, pp. 436–443.

Zhou, Q.-Y., Park, J. and Koltun, V. (2018) 'Open3D: A modern library for 3D data processing', *arXiv preprint arXiv:1801.09847*.

Appendix

Appendix A – Source Code Listings

Appendix A.1: Hesai Point Type Definition and Handler (excerpt)

`common_lib.h` : Custom HesaiPointXYZIRT struct and type registration:

```
struct HesaiPointXYZIRT
{
    PCL_ADD_POINT4D
    PCL_ADD_INTENSITY;
    uint16_t ring;
    double timestamp;
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW
} EIGEN_ALIGN16;

POINT_CLOUD_REGISTER_POINT_STRUCT (HesaiPointXYZIRT,
    (float, x, x) (float, y, y) (float, z, z) (float, intensity, intensity)
    (uint16_t, ring, ring) (double, timestamp, timestamp)
)
```

`preprocess.cpp` - Hesai case 5 handler (key excerpt):

```
case 5: // Hesai LiDAR (lidar_type: 5)
{
    pl_surf.clear();
    pl_corn.clear();
    pl_full.clear();

    int plsize = msg->width * msg->height;
    if (plsize == 0) break;
    pl_surf.reserve(plsize);

    sensor_msgs::PointCloud2ConstIterator<float>  iter_x(*msg, "x");
    sensor_msgs::PointCloud2ConstIterator<float>  iter_y(*msg, "y");
    sensor_msgs::PointCloud2ConstIterator<float>  iter_z(*msg, "z");
    sensor_msgs::PointCloud2ConstIterator<float>  iter_intensity(*msg, "intensity");
    sensor_msgs::PointCloud2ConstIterator<double> iter_time(*msg, "timestamp");

    // Find first valid base timestamp
    double time_base = 0.0;
    auto temp_iter_time = iter_time;
    for (int k = 0; k < plsize; ++k, ++temp_iter_time) {
        if (*temp_iter_time > 1000.0) {
            time_base = *temp_iter_time;
            break;
        }
    }

    for (int i = 0; i < plsize;
         ++i, ++iter_x, ++iter_y, ++iter_z, ++iter_intensity, ++iter_time)
    {
        if (i % point_filter_num != 0) continue;

        // Filter NaNs — critical for ikd-Tree stability
        if (!std::isfinite(*iter_x) || !std::isfinite(*iter_y) ||
            !std::isfinite(*iter_z) || !std::isfinite(*iter_time)) continue;

        // Discard points inside blind zone
        double range = (*iter_x * *iter_x) + (*iter_y * *iter_y)
                     + (*iter_z * *iter_z);
        if (range < (blind * blind)) continue;

        PointType added_pt;
        added_pt.x = *iter_x;
        added_pt.y = *iter_y;
        added_pt.z = *iter_z;
        added_pt.intensity = *iter_intensity;
        added_pt.normal_x = 0; added_pt.normal_y = 0; added_pt.normal_z = 0;

        // Compute relative time and clamp to [0, 150] ms
        double rel_time = (*iter_time - time_base) * 1000.0;
        if (rel_time < 0.0)   rel_time = 0.0;
        if (rel_time > 150.0) rel_time = 150.0;

        added_pt.curvature = rel_time;
        pl_surf.points.push_back(added_pt);
    }
}
```

```
break;
```

Appendix A.2: Hesai configuration file for Fastlio2

hesai.yaml

```
# Author: Stepan Letsko
# Date: January 12, 2026
# Purpose: Configuration file for FAST_LIO_ROS2 to support Hesai LiDARs (e.g., Pandar XT-16/XT-32).
#          It sets up topic names, extrinsic parameters, and mapping settings for use with standard PointCloud2
messages.

/**:
    ros__parameters:
        feature_extract_enable: false
        point_filter_num: 4          # INCREASED: Reduce CPU load (take 1 point every 4)
        max_iteration: 3
        filter_size_surf: 0.4
        filter_size_map: 0.4
        cube_side_length: 1000.0
        runtime_pos_log_enable: true
        map_file_path: "./RAM_TEST.pcd"
        loop_closure_enable: false  # Set to false to run standard FAST-LIO

        common:
            lid_topic:  "/lidar_points" # Updated to Hesai ROS 2 driver output
            imu_topic:  "/imu/data"     # Updated to SBG ROS 2 standard ENU output
            time_sync_en: false         # ONLY turn on when external time synchronization is really not possible
            time_offset_lidar_to_imu: 0.0 # Time offset between lidar and IMU calibrated by other algorithms, e.g.
LI-Init (can be found in README).
                                        # This param will take effect no matter what time_sync_en is. So if the
time offset is not known exactly, please set as 0.0

        preprocess:
            lidar_type: 5               # 1 for Livox serials LiDAR, 2 for Velodyne/Generic Mechanical LiDAR, 3
for ouster LiDAR, 4 for Hesai XT (ID 5)
            scan_line: 16               # Updated for Hesai XT16 (16 laser rings)
            scan_rate: 10               # only need to be set for velodyne, unit: Hz,
            timestamp_unit: 0           # the unit of time/t field in the PointCloud2 rostopic: 0-second, 1-
milisecond, 2-microsecond, 3-nanosecond. # Not used by lidar_type 5
            blind: 0.5                  # Adjusted to ignore UAV frame/propeller interference

        mapping:
            acc_cov: 0.1
            gyr_cov: 0.1
            b_acc_cov: 0.0001
            b_gyr_cov: 0.0001
            fov_degree:    360.0
            det_range:     120.0        # Updated to Hesai XT16 max detection range
            extrinsic_est_en:  false    # true: enable the online estimation of IMU-LiDAR extrinsic,
            extrinsic_T: [ 0.0, -0.106368, 0.052867 ]
            extrinsic_R: [ 1.0, 0.0, 0.0,
                           0.0, 1.0, 0.0,
                           0.0, 0.0, 1.0 ]

        publish:
            path_en:  true
            scan_publish_en:  true      # false: close all the point cloud output
            dense_publish_en: true      # false: low down the points number in a global-frame point clouds scan.
            scan_bodyframe_pub_en: true  # true: output the point cloud scans in IMU-body-frame

        pcd_save:
            pcd_save_en: false
            interval: -1                # how many LiDAR frames saved in each pcd file;
                                       # -1 : all frames will be saved in ONE pcd file, may lead to memory crash
when having too much frames.
```

Appendix A.3: Dockerfile

```
# Author: Stepan Letsko
# Date: January 10, 2026
# Purpose: This Dockerfile builds a reproducible environment for the Thesis project.
#          It sets up ROS 2 Humble, installs necessary dependencies for FAST_LIO and Livox drivers,
#          and prepares the workspace for building and running the SLAM system.

# Start from the official ROS 2 Humble base image
FROM ros:humble-ros-base

# 1. Install basic tools and dependencies for FAST_LIO
RUN apt-get update && \
    apt-get install -y --no-install-recommends ca-certificates gnupg && \
    rm -rf /var/lib/apt/lists/* && \
    apt-get update && \
    apt-get install -y --fix-missing \
    git \
```

```
    # git: Version control system to clone repositories
    cmake \
    # cmake: Cross-platform build system generator used by ROS packages
    build-essential
    libpcl-dev \
    ros-humble-pcl-ros \
    # ros-humble-pcl-ros: ROS 2 interface for PCL, allows PCL types in ROS nodes
    ros-humble-pcl-conversions \
    # ros-humble-pcl-conversions: Utilities to convert between ROS PointCloud2 messages and PCL objects
    ros-humble-libpointmatcher \
    # ros-humble-libpointmatcher:
    ros-humble-rviz2 \
    # ros-humble-rviz2: 3D visualization tool for ROS 2 (to see the map and lidar data)
    ros-humble-velodyne \
    ros-humble-gtsam \
    # Foxglove Bridge for visualization
    ros-humble-foxglove-bridge \
    # Topic tools for manipulating ROS 2 topics (like throttling)
    ros-humble-topic-tools \
    wget \
    libboost-all-dev \
    libyaml-cpp-dev \
    libpcap-dev \
    iputils-ping \
    net-tools \
    && rm -rf /var/lib/apt/lists/*

RUN wget https://raw.githubusercontent.com/jlblancoc/nanoflann/master/include/nanoflann.hpp -O
/usr/local/include/nanoflann.hpp && \
    wget https://raw.githubusercontent.com/gisbi-
kim/scancontext/main/cpp/module/Scancontext/KDTreeVectorOfVectorsAdaptor.h -O
/usr/local/include/KDTreeVectorOfVectorsAdaptor.h

# 2. Install Livox-SDK2 (Required by livox_ros_driver2)
RUN git clone https://github.com/Livox-SDK/Livox-SDK2.git /tmp/Livox-SDK2 && \
    # Clone the SDK source code to a temporary directory
    cd /tmp/Livox-SDK2 && \
    # Enter the directory
    mkdir build && cd build && \
    # Create a build directory and enter it (standard CMake practice)
    cmake .. && make -j$(nproc) && make install && \
    # Configure with CMake, compile using all available CPU cores, and install to system paths
    rm -rf /tmp/Livox-SDK2
    # Remove the source code to save space after installation

# 3. Setup the environment
# Set the default working directory inside the container to the ROS workspace root
WORKDIR /root/ros2_ws

# Add the ROS 2 setup script to .bashrc so it is sourced automatically
# whenever a new terminal is opened inside the container.
RUN echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc && \
    echo "if [ -f /root/ros2_ws/install/setup.bash ]; then source /root/ros2_ws/install/setup.bash; fi" >>
~/.bashrc && \
    echo 'export CPATH=$CPATH:$(python3 -c "import numpy; print(numpy.get_include())")' >> ~/.bashrc

# 4. Install Colcon
# Update apt again and install colcon
RUN apt-get update && apt-get install -y \
    python3-colcon-common-extensions \
    python3-pip \
    python3-tk \
    && pip3 install "rosbags" "numpy<2.0" lz4 pandas matplotlib psutil open3d

# 5. Install SBG ROS2 Driver
RUN mkdir -p src && \
    git clone --recursive https://github.com/SBG-Systems/sbg_ros2_driver.git src/sbg_ros2_driver && \
    apt-get update && \
    rosdep update && \
    rosdep install --from-path src --ignore-src -y && \
    . /opt/ros/humble/setup.sh && \
    colcon build --packages-select sbg_driver --cmake-args -DCMAKE_BUILD_TYPE=Release && \
    rm -rf /var/lib/apt/lists/*

# 6. Install Hesai ROS2 Driver
RUN git clone --recursive https://github.com/HesaiTechnology/HesaiLidar_ROS_2.0.git src/HesaiLidar_ROS_2.0 && \
    apt-get update && \
    rosdep install --from-path src --ignore-src -y && \
    . /opt/ros/humble/setup.sh && \
    colcon build --packages-select hesai_ros_driver --cmake-args -DCMAKE_BUILD_TYPE=Release && \
    rm -rf /var/lib/apt/lists/*

# 7. Install evo for trajectory evaluation
RUN apt-get update && apt-get install -y python3-pip && \
    pip3 install evo --upgrade --no-binary evo && \
    rm -rf /var/lib/apt/lists/*

# 8. Setup Entrypoint
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
# Set the entrypoint to run our script on container startup
ENTRYPOINT ["/entrypoint.sh"]
```

```
CMD ["bash"]
```

Appendix A.4: run_full_analysis.py

```
import subprocess
import argparse
import os
import sys
import time
import signal
import re
import csv
import threading
import psutil
import yaml
import matplotlib
matplotlib.use('Agg')
import pandas as pd
import matplotlib.pyplot as plt
import shutil
from pathlib import Path

# ==========================================
# CONFIGURATION
# ==========================================
RESULTS_BASE = Path("/mnt/usb/results/full_analysis_results")
# The map name defined
EXPECTED_PCD_NAME = "Current_map.pcd"
FAST_LIO_LOG_PATH = Path("/root/ros2_ws/src/FAST_LIO_ROS2/Log/fast_lio_time_log.csv")
CONFIG_DIR = Path("/root/ros2_ws/src/FAST_LIO_ROS2/config")
BAG_TO_TUM_SCRIPT = Path(__file__).parent / "bag_to_tum.py"
BAG_TO_PCD_SCRIPT = Path(__file__).parent / "bag_to_pcd.py"

class FastLioAnalyzer:
    def __init__(self, bag_path, config_file, use_decoder=False, start_offset=0.0, duration=None,
record_mode='outputs', skip_pcd=False):
        self.bag_path = Path(bag_path)
        self.bag_name = self.bag_path.stem
        self.config_file = config_file
        self.use_decoder = use_decoder
        self.start_offset = start_offset
        self.duration = duration
        self.record_mode = record_mode  # 'odometry', 'outputs', or 'inputs'
        self.skip_pcd = skip_pcd
        self.output_dir = RESULTS_BASE / f"{self.bag_name}_FULL_ANALYSIS"

        # Data Containers
        self.latencies = []
        self.resource_stats = []
        self.mapping_pid = None
        self.stop_event = threading.Event()
        self.total_duration = 0

        # Create Directory
        if self.output_dir.exists():
            shutil.rmtree(self.output_dir)
        self.output_dir.mkdir(parents=True, exist_ok=True)

    def get_bag_duration(self):
        try:
            res = subprocess.run(['ros2', 'bag', 'info', str(self.bag_path)], capture_output=True, text=True)
            match = re.search(r"Duration:\s+(\d+\.\d+)s", res.stdout)
            return float(match.group(1)) if match else 100.0
        except:
            return 100.0

    def find_mapping_pid(self):
        # Only accept a fastlio_mapping process that was created AFTER we launched it.
        # This prevents attaching to a stale process left over from a previous run.
        for _ in range(15):
            for proc in psutil.process_iter(['pid', 'cmdline', 'name', 'create_time']):
                try:
                    cmdline = ' '.join(proc.info['cmdline'] or [])
                    if 'fastlio_mapping' in cmdline:
                        age = time.time() - proc.info['create_time']
                        if proc.info['create_time'] >= self.mapping_launch_time:
                            print(f"   -> Monitoring PID {proc.info['pid']} ({proc.info['name']}):
{cmdline[:100]}")
                            return proc.info['pid']
                        else:
                            print(f"   -> Skipping stale fastlio_mapping PID {proc.info['pid']} (created {age:.1f}s
ago, before this run)")
                except (psutil.NoSuchProcess, psutil.AccessDenied):
                    pass
            time.sleep(1)
        print("   -> WARNING: fastlio_mapping process not found after 15s!")
        return None

    def get_input_topics(self):
```

```
        """Parse lid_topic and imu_topic from the FAST-LIO config YAML."""
        config_path = CONFIG_DIR / self.config_file
        try:
            with open(config_path) as f:
                cfg = yaml.safe_load(f)
            common = cfg['/**']['ros__parameters']['common']
            lid_topic = common['lid_topic'].strip()
            imu_topic = common['imu_topic'].strip()
            print(f"   -> Input topics from config: {lid_topic}, {imu_topic}")
            return [lid_topic, imu_topic]
        except Exception as e:
            print(f"   -> Warning: Could not parse input topics from {config_path}: {e}")
            return []

    def task_log_parser(self, process):
        """Thread 1: Reads stdout line-by-line for Latency metrics"""
        log_path = self.output_dir / "process_log.txt"
        csv_path = self.output_dir / "latency_data.csv"

        with open(log_path, "w") as f_log, open(csv_path, "w", newline="") as f_csv:
            writer = csv.writer(f_csv)
            writer.writerow(["match", "solve", "ICP", "total"]) # Simplified header

            # Regex for the timing line
            pattern = r"match:\s*([\d\.]+).*solve:\s*([\d\.]+).*ICP:\s*([\d\.]+).*total:\s*([\d\.]+)"

            while not self.stop_event.is_set():
                line = process.stdout.readline()
                if not line: break

                f_log.write(line) # Save full log

                if "ave total:" in line:
                    match = re.search(pattern, line)
                    if match:
                        vals = [float(x) for x in match.groups()]
                        writer.writerow(vals)
                        self.latencies.append(vals[3]) # Index 3 is total time

    def task_resource_monitor(self):
        """Thread 2: Polls CPU/RAM every 0.5s"""
        while self.mapping_pid is None and not self.stop_event.is_set():
            time.sleep(0.5)

        if not self.mapping_pid: return

        try:
            proc = psutil.Process(self.mapping_pid)
            print(f"   -> Resource monitor attached to: '{proc.name()}' (PID {self.mapping_pid})")
            start_t = time.time()

            while not self.stop_event.is_set():
                try:
                    cpu = proc.cpu_percent(interval=None)
                    ram = proc.memory_info().rss / (1024 * 1024) # MB
                    elapsed = time.time() - start_t
                    self.resource_stats.append([elapsed, cpu, ram])
                except:
                    break # Process died
                time.sleep(0.5)
        except psutil.NoSuchProcess:
            return

    def generate_report(self):
        print("\n\n Generating Final Report...")

        # Resource Plot
        if self.resource_stats:
            df_res = pd.DataFrame(self.resource_stats, columns=['Time', 'CPU', 'RAM'])
            df_res.to_csv(self.output_dir / "resources.csv", index=False)

            fig, ax1 = plt.subplots(figsize=(10, 6))
            ax2 = ax1.twinx()
            ax1.plot(df_res['Time'], df_res['CPU'], 'g-', alpha=0.6, label='CPU %')
            ax2.plot(df_res['Time'], df_res['RAM'], 'b-', linewidth=2, label='RAM (MB)')

            ax1.set_ylabel('CPU (%)', color='g')
            ax2.set_ylabel('RAM (MB)', color='b')
            ax1.set_xlabel('Time (s)')
            plt.title(f"Resource Usage: {self.bag_name}")
            plt.grid(True, alpha=0.3)
            plt.savefig(self.output_dir / "resource_plot.png")

            peak_ram = df_res['RAM'].max()
            avg_cpu = df_res['CPU'].mean()
            peak_cpu = df_res['CPU'].max()
        else:
            peak_ram = 0
            avg_cpu = 0
            peak_cpu = 0

        #  Latency Stats (Prefer C++ Log if available)
```

```
        cpp_log_path = self.output_dir / "fast_lio_time_log.csv"
        source_type = "STDOUT (Approximate)"

        if cpp_log_path.exists():
            try:
                df = pd.read_csv(cpp_log_path, skipinitialspace=True)
                # 'math_time' is usually the total processing time in the C++ log
                if 'math_time' in df.columns:
                    lat_data = df['math_time']
                    if 'io_time' in df.columns:
                        lat_data = lat_data + df['io_time']
                    total_frames = len(lat_data)
                    avg_lat = lat_data.mean()
                    max_lat = lat_data.max()
                    source_type = "C++ LOG (Accurate)"
                else:
                    # Fallback to stdout data
                    total_frames = len(self.latencies)
                    avg_lat = sum(self.latencies)/total_frames if total_frames > 0 else 0
                    max_lat = max(self.latencies) if total_frames > 0 else 0
            except:
                pass
        else:
            total_frames = len(self.latencies)
            avg_lat = sum(self.latencies)/total_frames if total_frames > 0 else 0
            max_lat = max(self.latencies) if total_frames > 0 else 0

        summary = (
            f"========================================\n"
            f" FINAL RESULTS: {self.bag_name}\n"
            f"========================================\n"
            f" Total Processed Frames: {total_frames}\n"
            f" Avg Processing Time:    {avg_lat*1000:.2f} ms\n"
            f" Max Processing Time:    {max_lat*1000:.2f} ms\n"
            f" Data Source:            {source_type}\n"
            f"----------------------------------------\n"
            f" Peak CPU Usage:         {peak_cpu:.2f} %\n"
            f" Peak RAM Usage:         {peak_ram:.2f} MB\n"
            f" Avg CPU Usage:          {avg_cpu:.2f} %\n"
            f"========================================\n"
        )

        print(summary)
        with open(self.output_dir / "summary.txt", "w") as f:
            f.write(summary)

        self.update_global_history(total_frames, avg_lat, max_lat, peak_ram, peak_cpu, avg_cpu)

    def update_global_history(self, frames, avg_lat, max_lat, peak_ram, peak_cpu, avg_cpu):
        """Appends the results of this run to a master CSV for easy comparison."""
        master_csv = RESULTS_BASE / "benchmark_comparison.csv"
        file_exists = master_csv.exists()

        try:
            with open(master_csv, "a", newline="") as f:
                writer = csv.writer(f)
                # Write Header if new file
                if not file_exists:
                    writer.writerow(["Timestamp", "Bag Name", "Config", "Frames", "Avg Latency (ms)", "Max Latency
(ms)", "Peak RAM (MB)", "Peak CPU (%)", "Avg CPU (%)"])

                # Write Data
                writer.writerow([
                    time.strftime("%Y-%m-%d %H:%M:%S"),
                    self.bag_name,
                    self.config_file,
                    frames,
                    f"{avg_lat*1000:.2f}",
                    f"{max_lat*1000:.2f}",
                    f"{peak_ram:.2f}",
                    f"{peak_cpu:.2f}",
                    f"{avg_cpu:.2f}"
                ])
            print(f"   -> Added entry to master comparison log: {master_csv}")
        except Exception as e:
            print(f"   -> Warning: Could not update master log: {e}")

    def run(self):
        self.total_duration = self.get_bag_duration()
        print(f"Starting Full Analysis for {self.bag_name} ({self.total_duration:.1f}s)")
        print(f"Output: {self.output_dir}")

        # Start Decoder
        proc_decoder = None
        if self.use_decoder:
            print("   -> Launching Velodyne Decoder...")
            decoder_cmd = ['ros2', 'launch', 'fast_lio', 'decoder.launch.py']
            proc_decoder = subprocess.Popen(decoder_cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
            time.sleep(2) # Give it a moment to initialize

        # Start Recording (Background)
        bag_out = self.output_dir / "recorded_bag"
```

```
print("   -> Starting Recorder...")
if self.record_mode == 'outputs':
    topics = ['/Odometry', '/cloud_registered', '/path']
elif self.record_mode == 'inputs':
    topics = self.get_input_topics()
    if not topics:
        print("   -> Warning: Falling back to /Odometry only.")
        topics = ['/Odometry']
elif self.record_mode == 'all':
    topics = None  # Use -a flag
else:  # 'odometry'
    topics = ['/Odometry']
if topics is None:
    print("   -> Recording ALL topics")
    rec_cmd = ['ros2', 'bag', 'record', '-a', '-o', str(bag_out)]
else:
    print(f"   -> Recording topics: {', '.join(topics)}")
    rec_cmd = ['ros2', 'bag', 'record'] + topics + ['-o', str(bag_out)]
proc_rec = subprocess.Popen(rec_cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

 # Start FAST-LIO
print(f"   -> Launching Node ({self.config_file})...")
# stdbuf -oL forces line buffering so we can read logs instantly
launch_cmd = ['stdbuf', '-oL', 'ros2', 'launch', 'fast_lio', 'mapping.launch.py',
              f'config_file:={self.config_file}',
              'rviz:=false'
              ]
self.mapping_launch_time = time.time()
proc_mapping = subprocess.Popen(launch_cmd, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)

# Find PID
self.mapping_pid = self.find_mapping_pid()

#Start Monitoring Threads
t_log = threading.Thread(target=self.task_log_parser, args=(proc_mapping,))
t_res = threading.Thread(target=self.task_resource_monitor)
t_log.start()
t_res.start()

# Start Playback
print("Waiting 5 seconds for node to initialise...")
time.sleep(5)
print("   -> Playing Bag...")

# Construct Bag Play Command
play_cmd = ['ros2', 'bag', 'play', str(self.bag_path), '--clock']
if self.start_offset > 0:
    play_cmd.extend(['--start-offset', str(self.start_offset)])

# Note: We handle duration manually in the loop to ensure clean shutdown
proc_play = subprocess.Popen(play_cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

#Progress Bar Loop
start_t = time.time()
# If user specified a duration, use that. Otherwise use bag length.
target_duration = self.duration if self.duration else self.total_duration

try:
    while proc_play.poll() is None:
        elapsed = time.time() - start_t
        percent = min(100, (elapsed / target_duration) * 100)

        # Dynamic Status Line
        ram_str = f"{self.resource_stats[-1][2]:.0f}" if self.resource_stats else "0"
        frames_str = f"{len(self.latencies)}"

        bar = "" * int(percent // 2) + "-" * (50 - int(percent // 2))
        sys.stdout.write(f"\r|{bar}| {percent:.1f}% | RAM: {ram_str} MB | Frames: {frames_str}")
        sys.stdout.flush()
        time.sleep(0.5)

        # Manual Duration Check
        if self.duration and elapsed >= self.duration:
            break

except KeyboardInterrupt:
    print("\n Interrupted!")

# Cleanup
print("\n\n Finishing Up...")

# Trigger Map Save
print("   -> Triggering Map Save...")
try:
    subprocess.run(['ros2', 'service', 'call', '/map_save', 'std_srvs/srv/Trigger'],
                   stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=10)
except: pass

# Stop Threads
self.stop_event.set()

# Kill Processes
```

```
        os.kill(proc_play.pid, signal.SIGINT) if proc_play.poll() is None else None
        os.kill(proc_rec.pid, signal.SIGINT)
        os.kill(proc_mapping.pid, signal.SIGINT)
        if proc_decoder: os.kill(proc_decoder.pid, signal.SIGINT)

        # Wait for the mapping node to fully exit so it doesn't linger as a stale
        try:
            proc_mapping.wait(timeout=15)
        except subprocess.TimeoutExpired:
            print("   -> WARNING: mapping process did not exit cleanly, force-killing.")
            proc_mapping.kill()

        # Wait for threads
        t_log.join()
        t_res.join()

        if self.record_mode == 'outputs':
            if Path(EXPECTED_PCD_NAME).exists():
                shutil.move(EXPECTED_PCD_NAME, self.output_dir / "final_map.pcd")
                print("   -> Map Saved successfully.")
            elif self.skip_pcd:
                print("   -> Map reconstruction skipped (--skip-pcd).")
            elif BAG_TO_PCD_SCRIPT.exists() and bag_out.exists():
                print("   -> Reconstructing map from recorded bag (RAM Saving Mode)...")
                pcd_out = self.output_dir / "final_map.pcd"
                subprocess.run([
                    'python3', str(BAG_TO_PCD_SCRIPT),
                    str(bag_out),
                    '--topic', '/cloud_registered',
                    '--output', str(pcd_out),
                    '--voxel', '0.05'
                ], check=True)
                print(f"   -> Map reconstructed to {pcd_out.name}")
        else:
            print(f"   -> Map reconstruction skipped (record mode: {self.record_mode}).")
            if Path(EXPECTED_PCD_NAME).exists():
                Path(EXPECTED_PCD_NAME).unlink()  # Cleanup native save if present

        # Extract Trajectory (TUM format) for Evo
        if BAG_TO_TUM_SCRIPT.exists() and bag_out.exists():
            tum_out = self.output_dir / f"{self.bag_name}_trajectory.tum"
            print(f"   -> Extracting trajectory to {tum_out.name}...")
            subprocess.run(['python3', str(BAG_TO_TUM_SCRIPT), str(bag_out), str(tum_out)],
                           stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

        # Copy C++ Log CSV and Generate Plot
        if FAST_LIO_LOG_PATH.exists():
            dest_csv = self.output_dir / "fast_lio_time_log.csv"
            shutil.copy(FAST_LIO_LOG_PATH, dest_csv)
            print(f"   -> Copied detailed C++ time log to {dest_csv}")

            # Call the separate plotting script
            plot_script = Path(__file__).parent / "plot_latency.py"
            if plot_script.exists():
                plot_out = self.output_dir / "latency_plot.png"
                subprocess.run(['python3', str(plot_script), str(dest_csv), str(plot_out)])

        self.generate_report()
        print(f"DONE. All data in {self.output_dir}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Run Full FAST-LIO Analysis")
    parser.add_argument("bag_path", help="Path to input rosbag")
    parser.add_argument("config_file", nargs="?", default="velodyne.yaml", help="Config file name (default:
velodyne.yaml)")
    parser.add_argument("--decoder", action="store_true", help="Launch Velodyne decoder (for raw packet bags like
DARPA)")
    parser.add_argument("--start", type=float, default=0.0, help="Start offset in seconds")
    parser.add_argument("--duration", type=float, default=None, help="Duration to run in seconds (optional)")
    parser.add_argument("--record", choices=['odometry', 'outputs', 'inputs', 'all'], default='outputs',
                        help="Recording mode: 'odometry' (just /Odometry), "
                             "'outputs' (odom + cloud_registered + path, default), "
                             "'inputs' (LiDAR + IMU topics parsed from config), "
                             "'all' (every active topic)")
    parser.add_argument("--skip-pcd", action="store_true", help="Skip running bag_to_pcd.py map reconstruction at
the end")

    args = parser.parse_args()

    analyzer = FastLioAnalyzer(args.bag_path, args.config_file, args.decoder, args.start, args.duration,
args.record, args.skip_pcd)
    analyzer.run()
```

Appendix B – AI use disclosure

During this research AI was used in multiple phases of this thesis. artificial intelligence was used to assist with technical problem-solving and conceptual clarification. Specifically, the Gemini AI model was employed as an advanced search and explanation engine. The applications of AI in this project were strictly limited to the following supportive tasks:

Syntax and Command Retrieval: Functioning as a search tool to quickly get information on the format of Linux system commands and Robot Operating System 2 (ROS 2) middleware configurations and information.

Hardware Concept Clarification: Explaining complex systems architecture and hardware-level interactions, specifically regarding the CPU, RAM, and memory management behaviour on the Raspberry Pi 5 platform.

Software Development Support: AI-integrated tools within Visual Studio Code (VS Code) during the software engineering phase were used to provide code assistance with syntax and error checking, and to accelerate the debugging process.

All research, core architectural decisions, physical hardware assembly, algorithmic parameter tuning, data collection, and analytical conclusions presented in this thesis are my own.